CS336 作业 2（系统）：系统与并行

版本 26.1.3 | CS336 教学团队 | 2026 年春季

1 作业概述

本次作业将让你通过实践，学习如何提高单 GPU 训练速度，以及如何将训练扩展到多个 GPU。

需要实现的内容

1. 基准测试与性能分析框架。
2. 激活检查点。
3. FlashAttention-2 的 Triton 内核。
4. 分布式数据并行训练。
5. 优化器状态分片。
6. 全分片数据并行训练。

代码结构

作业代码与本说明文档位于 GitHub：

github.com/stanford-cs336/assignment2-systems

请使用 Git 克隆仓库。如果有更新，我们会通知你；你可以通过 git pull 获取最新版本。

1. cs336-basics/：本次作业需要分析作业 1 中构建的一些组件。此目录包含教学团队提供的作业 1 参考实现，其中有 cs336-basics/pyproject.toml 和 cs336-basics/cs336_basics 包。如果希望使用自己实现的模型，可以修改根目录的 pyproject.toml，使其指向自己的包。

2. /：cs336-systems 根目录。我们创建了一个名为 cs336_systems 的空模块。其中没有代码，你可以从头自行组织实现。

3. tests/*.py：包含必须通过的全部测试。测试调用 tests/adapters.py 中定义的接口钩子。你需要实现这些适配器，将自己的代码接入测试。编写更多测试或修改测试代码有助于调试，但最终实现必须通过最初提供的测试套件。

4. README.md：提供预期目录结构的更多细节，以及环境配置的基本说明。

提交方式

向 Gradescope 提交以下文件：

• writeup.pdf：回答全部书面问题。请对回答进行排版。

• code.zip：包含你编写的全部代码。

运行 test_and_make_submission.sh 中的脚本生成 code.zip。

译稿说明：每个对照页左侧为对应原页的中文译文，右侧为完整英文原页。代码、命令、公式、图表和标识符保留在原页中；中文栏翻译正文、题目、提交要求、图注及代码注释的含义。跨页内容按原文页码接续。本译稿不补充题目答案，也不更改原文的技术要求或数值。

CS336 Assignment 2 (systems): Systems and Parallelism

Version 26.1.3

CS336 Staff

Spring 2026

1 Assignment Overview

In this assignment, you will gain some hands-on experience with improving single-GPU training speed and scaling training to multiple GPUs.

What you will implement

1. Benchmarking and profiling harness
2. Activation checkpointing
3. Flash Attention 2 Triton kernel
4. Distributed data parallel training
5. Optimizer state sharding
6. Fully sharded data parallel training

What the code looks like

The assignment code and this write-up are available on GitHub at:

`github.com/stanford-cs336/assignment2-systems`

Please clone the repository using Git. If there are any updates, we will notify you and you can `git pull` to get the latest.

1. `cs336-basics/`: In this assignment, you'll be profiling some of the components that we built in assignment 1. This folder contains the staff solution code for assignment 1, so you will find `cs336-basics/pyproject.toml` and the `cs336-basics/cs336_basics` package here. If you want to use your own implementation of the model, you can modify the `pyproject.toml` file in the base directory to point to your own package.
2. `/`: The cs336-systems base directory. We created an empty module named `cs336_systems`. Note that there's no code in here, so you should be able to do whatever you want from scratch.
3. `tests/*.py`: This directory contains all the tests you must pass. These tests invoke the hooks defined in `tests/adapters.py`. You'll implement the adapters to connect your code to the tests. Writing more tests and/or modifying the test code can be helpful for debugging your code, but your implementation is expected to pass the original provided test suite.
4. `README.md`: This file contains more details about the expected directory structure, as well as some basic instructions on setting up your environment.

How to submit

You will submit the following files to Gradescope:

- `writeup.pdf`: Answer all the written questions. Please typeset your responses.
- `code.zip`: Contains all the code you've written.

Run the script in `test_and_make_submission.sh` to create the `code.zip` file.

2 性能分析与基准测试

在作业的第一部分，我们将探索如何优化 Transformer 模型的性能，以更高效地使用 GPU。我们会分析模型，了解前向传播和反向传播分别将时间和显存花在哪里；随后使用自定义 GPU 内核优化自注意力运算，使其速度超过常规 PyTorch 实现。在后续部分，我们将使用多个 GPU，了解如何在集群上训练模型。

2.1 性能分析

在实施优化之前，先分析程序的资源开销（例如时间和显存）很有帮助。否则，我们可能优化了时间或显存占比很小的部分，因而无法获得可测量的端到端改善。

我们将实现三条性能评估路径：

1. 使用 Python 标准库进行简单的端到端基准测试，测量前向与反向传播时间。
2. 使用 NVIDIA Nsight Systems 分析计算性能，了解 CPU 和 GPU 上各项操作如何分摊总时间。
3. 显存分析。

2.1.1 环境准备：导入 basics Transformer 模型

首先确认你能够加载上一次作业的模型。在作业 1 中，我们将模型组织为 Python 包，以便后续导入。我们已在 ./cs336-basics 目录中加入模型的参考实现，并在 pyproject.toml 中指向它。照常调用 uv run [command] 时，uv 会自动找到这个本地 cs336-basics 包。如果希望使用自己的模型实现，可以修改 pyproject.toml，使其指向自己的包。

你可以按右侧终端示例启动 Python，并通过 import cs336_basics 检查模型能否导入。示例展示了 CPython 3.13.13、虚拟环境创建、本地包构建以及依赖安装的输出。

现在应当可以使用作业 1 中的相关模块。例如，对于 model.py，可以通过 import cs336_basics.model 导入。

2.1.2 模型规模

本次作业会对模型进行基准测试与性能分析，以更好地了解其性能。为了解规模变化带来的影响，我们将采用并引用下页列出的模型配置。除排行榜任务外，所有模型均使用 10,000 的词表大小和 4 的批大小，上下文长度则会变化。

本次及后续作业都需要用表格和曲线展示大量结果。我们强烈建议你通过代码自动生成报告中的表格，因为手工排版 LaTeX 或 Typst 表格可能非常繁琐。

2 Profiling and Benchmarking

In the first part of the assignment, we will explore how to optimize the performance of our Transformer model to make the most efficient use of the GPU. We will profile our model to understand where it spends time and memory during the forward and backward passes, then optimize the self-attention operation with custom GPU kernels, making it faster than is possible with regular PyTorch. In the subsequent parts of the assignment, we will leverage multiple GPUs and understand how to train a model across a cluster.

2.1 Profiling

Before implementing any optimizations, it is helpful to first profile our program to understand where it spends resources (e.g., time and memory). Otherwise, we risk optimizing parts of the model that don't account for significant time or memory, and therefore not seeing measurable end-to-end improvements.

We will implement three performance evaluation paths:

1. Simple end-to-end benchmarking using the Python standard library to time our forward and backward passes
2. Compute profiling with the NVIDIA Nsight Systems tool to understand how that time is distributed across operations on both the CPU and GPU
3. Memory profiling

2.1.1 Setup - Importing your `basics` Transformer Model

Let's start by making sure that you can load the model from the previous assignment. In the previous assignment, we set up our model in a Python package, so that it could be easily imported later. We have added a reference implementation of the model in the `./cs336-basics` folder, and have pointed to it in the `pyproject.toml` file. By calling `uv run [command]` as usual, `uv` will automatically locate this local `cs336-basics` package. If you would like to use your own implementation of the model, you can modify the `pyproject.toml` file to point to your own package.

You can test that you can import your model with:

```
$ uv run python
Using CPython 3.13.13
Creating virtual environment at: /path/to/uv/env/dir
      Built cs336-systems @ file:///path/to/systems/dir
      Built cs336-basics @ file:///path/to/basics/dir
Installed 78 packages in 168ms
Python 3.13.13 (main, Apr  7 2026, 20:49:46) [Clang 22.1.1 ] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> import cs336_basics
...
```

The relevant modules from assignment 1 should now be available (e.g., for `model.py`, you can import it with `import cs336_basics.model`).

2.1.2 Model Sizing

Throughout this assignment, we will be benchmarking and profiling models to better understand their performance. To get a sense of how things change at scale, we will work with and refer to the following model configurations. For all models except in the leaderboard, we'll use a vocabulary size of 10,000 and a batch size of 4, with varying context lengths. This assignment (and later ones) will require a lot of results to be presented in tables and plots. We strongly recommend that you automate constructing tables for

可参考 pandas.DataFrame.to_latex() 与 pandas.DataFrame.to_typst()，也可以自行编写函数，将偏好的表格数据表示转换为排版文本。

表 1：不同模型规模的配置，主要基于 GPT-2 配置。

规模 | d_model | d_ff | num_layers | num_heads

small | 768 | 3072 | 12 | 12

medium | 1024 | 4096 | 24 | 16

large | 1280 | 5120 | 36 | 20

xl | 2560 | 10240 | 32 | 32

10B | 4608 | 12288 | 50 | 36

除非另有说明，上下文长度使用 512。

2.1.3 端到端基准测试

接下来实现一个简单的性能评估脚本。我们会测试模型的许多变体，例如改变精度、替换层等。因此，最好通过命令行参数控制这些变体，以便后续运行。

首先进行最简单的性能评估：测量前向传播、反向传播和优化器更新步骤的时间。由于只测量速度和显存，使用随机权重和随机数据即可。

准确测量性能需要留意一些常见陷阱，否则测得的可能并非目标指标。对于 GPU 代码，一个注意点是 CUDA 调用是异步的。调用 CUDA 内核时，例如执行 torch.matmul，PyTorch 函数不会等待矩阵乘法完成，而是直接将控制权返回给你的代码。GPU 执行矩阵乘法的同时，CPU 可以继续执行并调度新的操作，这能显著提高性能。

但这也意味着，仅测量 torch.matmul 调用多久返回，无法得知 GPU 实际完成矩阵乘法的时间。在 PyTorch 中，可以调用 torch.cuda.synchronize() 等待所有已调度的 GPU 内核完成，从而更准确地测量 CUDA 内核运行时间。这里的同步指 CPU 执行与 GPU 执行之间的同步。考虑到这一点，开始编写基础性能评估框架。

题目（benchmarking_script）：基准测试脚本（4 分）

(a) 编写脚本，对模型的前向传播、反向传播与优化器更新进行基本的端到端基准测试。脚本应支持：

• 根据超参数（例如层数）初始化模型。

• 生成一个随机数据批次。

• 测量前先运行 w 个预热步骤，然后测量 n 个步骤的执行时间。根据参数选择：仅前向、前向加反向，或前向加反向再加优化器更新。

• 可以使用 Python 的 timeit 模块计时，例如 timeit 函数或 timeit.default_timer()。后者使用系统中分辨率最高的时钟，因此作为基准测试的默认计时器，比 time.time() 更合适。

your writeup in code, since formatting tables in LaTeX or Typst can be very tedious. See `pandas.DataFrame.to_latex()` and `pandas.DataFrame.to_typst()` or write your own function to generate them from your preferred tabular representation.

Size	d_model	d_ff	num_layers	num_heads
small	768	3072	12	12
medium	1024	4096	24	16
large	1280	5120	36	20
xl	2560	10240	32	32
10B	4608	12288	50	36

Table 1: Specifications of different model sizes. These are mostly based on GPT-2 configs.

Use context length 512 unless otherwise specified.

2.1.3 End-to-End Benchmarking

We will now implement a simple performance evaluation script. We will be testing many variations of our model (changing precision, swapping layers, etc.), so **it will pay off to have your script enable these variations via command-line arguments** to make them easy to run later on.

To start off, let's do the simplest possible profiling of our model by timing the forward pass, backward pass, and optimizer step. Since we will only be measuring speed and memory, it's fine to use random weights and data.

Measuring performance is subtle — some common traps can cause us to not measure what we want. For benchmarking GPU code, one caveat is that CUDA calls are *asynchronous*. When you call a CUDA kernel, such as when you invoke `torch.matmul`, the PyTorch function call returns control to your code without waiting for the matrix multiplication to finish. In this way, the CPU can continue running ahead and scheduling new operations while the GPU finishes the matrix multiplication, which is a major performance win. On the other hand, this means that naïvely measuring how long the `torch.matmul` call takes to return does not tell us how long the GPU takes to actually run the matrix multiplication. In PyTorch, we can call `torch.cuda.synchronize()` to wait for all scheduled GPU kernels to complete, allowing us to get more accurate measurements of CUDA kernel runtime. The synchronization in this operation refers to synchronizing the CPU runtime with the GPU runtime. With this in mind, let's write our basic profiling infrastructure.

Problem (`benchmarking_script`): Benchmarking Script (4 points)

(a) Write a script to perform basic end-to-end benchmarking of the forward pass, backward pass, and optimizer step in your model. Specifically, your script should support the following:

- Given hyperparameters (e.g., number of layers), initialize a model.
- Generate a random batch of data.
- Run w warm-up steps (before you start measuring time), then time the execution of n steps (either only forward, forward and backward, or forward and backward with optimizer step, depending on an argument). For timing, you can use the Python `timeit` module (e.g., either using the `timeit` function, or using `timeit.default_timer()`, which gives you the system's highest resolution clock, thus a better default for benchmarking than `time.time()`).

• 每个步骤结束后调用 torch.cuda.synchronize()。

提交内容：一个脚本，根据指定超参数初始化 basics Transformer 模型、生成随机数据批次，并测量仅前向、前向加反向以及包含优化器更新的完整训练步骤。

(b) 对第 2.1.2 节中的模型规模，测量前向传播、反向传播与优化器更新的时间。使用 5 个预热步骤，并根据 10 个测量步骤计算耗时的均值和标准差。一次前向传播需要多久？一次反向传播呢？测量值波动较大，还是标准差较小？

提交内容：用 1-2 句话回答，并给出测得的时间。

(c) 基准测试中的一个常见问题是省略预热步骤。不进行预热，重复上述分析。这如何影响结果？你认为原因是什么？再尝试只运行 1 或 2 个预热步骤。为什么结果仍可能不同？

提交内容：用 2-3 句话回答。

2.1.4 Nsight Systems 性能分析器

端到端基准测试无法说明模型在前向与反向传播中把时间和显存花在了哪里，因此不能指出具体的优化机会。为了了解程序各组件（例如函数）的耗时，可以使用性能分析器。执行性能分析器在函数开始和结束处插入监测点，对代码进行插桩，从而提供函数级的详细执行统计，例如调用次数、平均耗时及累计耗时等。

标准 Python 性能分析器（例如 CProfile）无法分析 CUDA 内核，因为这些内核在 GPU 上异步执行。NVIDIA 提供了可通过 nsys 命令行使用的性能分析器。建议通过包管理器或下载页面中的安装程序获取较新的版本。本部分作业将使用 nsys 分析 Transformer 模型的运行时间。

使用方式很直接：在上一节 Python 脚本的运行命令前加上 nsys profile。例如，对于 benchmark.py，基本命令为：

uv run nsys profile -- python benchmark.py

随后可在本机使用 NVIDIA Nsight Systems 桌面应用查看分析结果。在时间线的 CUDA API 行中选择某个 CPU 端 CUDA API 调用，会在 CUDA HW 行中突出显示其对应的 GPU 内核执行。

右侧原页还给出了更全面的分析命令，包含 --trace、--pytorch、--cudabacktrace、--python-backtrace 与 --gpu-metrics-devices 等选项。完整命令保持原样。

- Call `torch.cuda.synchronize()` after each step.

Deliverable: A script that will initialize a `basics` Transformer model with the given hyperparameters, create a random batch of data, and time forward-only, forward-and-backward, and full training steps that include the optimizer step.

(b) Time the forward, backward, and optimizer step for the model sizes described in Section 2.1.2. Use 5 warmup steps and compute the average and standard deviation of timings over 10 measurement steps. How long does a forward pass take? How about a backward pass? Do you see high variability across measurements, or is the standard deviation small?

Deliverable: A 1-2 sentence response with your timings.

(c) One caveat of benchmarking is not performing the warm-up steps. Repeat your analysis without the warm-up steps. How does this affect your results? Why do you think this happens? Also try to run the script with 1 or 2 warm-up steps. Why might the result still be different?

Deliverable: A 2-3 sentence response.

2.1.4 Nsight Systems Profiler

End-to-end benchmarking does not tell us where our model spends time and memory during forward and backward passes, and so does not expose specific optimization opportunities. To know how much time our program spends in each component (e.g., function), we can use a *profiler*. An execution profiler instruments the code by inserting guards when functions begin and finish running, and thus can give detailed execution statistics at the function level (such as number of calls, how long they take on average, cumulative time spent on this function, etc).

Standard Python profilers (e.g., `CProfile`) are not able to profile CUDA kernels since these kernels are executed asynchronously on the GPU. Fortunately, NVIDIA ships a profiler that we can use via the CLI `nsys`. We recommend that you get an up-to-date version either from your package manager, or using the installers from their download page. In this part of the assignment, you will use `nsys` to analyze the runtime of your Transformer model.

Using `nsys` is straightforward: run your Python script from the previous section with `nsys profile` prepended. For example, you can run a basic profile for the script `benchmark.py` with:

```
$ uv run nsys profile -- python benchmark.py
```

You can then view the profile on your local machine with the NVIDIA Nsight Systems desktop application. Selecting a particular CUDA API call (on the CPU) in the `CUDA API` row of the profile will highlight all corresponding kernel executions (on the GPU) in the `CUDA HW` row.

A more comprehensive profiling run may look like:

```
$ uv run nsys profile --trace=cuda,cudnn,cublas,osrt,nvtx --pytorch=functions-trace,autograd-shapes-nvtx --cudabacktrace=all --python-backtrace=cuda --gpu-metrics-devices=0 -- python benchmark.py
```

图 1：详细的 Nsight Systems 执行轨迹。

在该示例中，--trace 指定记录哪些 API；--pytorch 在模块调用和自动微分过程中插入 NVTX 标签；--cudabacktrace 和 --python-backtrace 提供更完善的调用回溯，帮助判断某个内核是从代码的何处调用的；--gpu-metrics-devices 指定测量哪块 GPU 的利用率。

性能分析本身有开销，会使整个运行变慢。通常每次只启用需要的功能更合适。尤其是不需要调用回溯时，可以去掉 --cudabacktrace=all 与 --python-backtrace=cuda，因为它们的开销较大。

建议尝试 nsys profile 的不同命令行选项，了解其能力。还可以使用 NVTX 区间标注代码；这些标注会在时间线的 NVTX 行中显示为区块，覆盖其中的全部 CUDA API 调用及相应的内核执行。

具体而言，你应当使用 NVTX 区间在基准测试中排除预热步骤：利用 --nvtx-capture 对时间线中的 NVTX 标签进行过滤。你也可以识别模型前向与反向传播对应的内核，甚至可以通过标注自注意力的实现，区分注意力层不同部分对应的内核。

右侧代码示例的注释与标签含义：

- 导入 torch.cuda.nvtx，并将其命名为 nvtx。
- 用“缩放点积注意力”区间装饰整个函数。
- 函数输入包括 Q、K、V 和 mask。
- “计算注意力分数”区间：计算 Q 与 K 之间的注意力分数。
- “计算 softmax”区间：计算注意力分数的 softmax。
- “最后的矩阵乘法”区间：计算输出投影。

函数随后返回计算结果，代码在下页继续。



Figure 1: A detailed Nsight Systems trace

In this example, `--trace` specifies which APIs to log, `--pytorch` inserts `nvtx` labels during module calls and autograd, `--cudabacktrace` and `--python-backtrace` give better backtraces to understand where in your code a given kernel was invoked from, and `--gpu-metrics-devices` specifies which GPU's utilization to measure.

Adding profiling to a run is not free, and will overall slow down your runs. It's often worth only enabling features you're looking for in a given run. Specifically, you might want to remove `--cudabacktrace=all` and `--python-backtrace=cuda` when tracebacks aren't needed, since they have outsize overhead.

We encourage you to experiment with various command-line options for `nsys profile` to get a sense of what it can do. You can also annotate your code with NVTX ranges, which will appear as blocks in the `NVTX` row of the profile capturing all CUDA API calls and associated kernel executions. In particular, you should use NVTX ranges to **ignore the warm-up steps in your benchmarking script** (by applying an `--nvtx-capture` filter on the nvtx label in the profile). You can also isolate which kernels are responsible for the forward and backward passes of your model, and you can even isolate which kernels are responsible for different parts of a self-attention layer by annotating your implementation as follows:

```
...
import torch.cuda.nvtx as nvtx

@nvtx.range("scaled dot product attention")
def annotated_scaled_dot_product_attention(
    ... # Q, K, V, mask
):
    ...
    with nvtx.range("computing attention scores"):
        ... # compute attention scores between Q and K

    with nvtx.range("computing softmax"):
        ... # compute softmax of attention scores

    with nvtx.range("final matmul"):
        ... # compute output projection
```

在基准测试脚本中，可以通过将 cs336_basics.model.scaled_dot_product_attention 赋值为 annotated_scaled_dot_product_attention，用带标注的版本替换原实现。具体语句见右侧。

最后，torch.compile 可能使时间和资源难以归因到代码的具体部分。你可能需要调整代码各处的 torch.compile 包装及 NVTX 标注，才能正确判断源代码不同部分的时间与资源开销。

题目（nsys_profile）：Nsight Systems 性能分析（5 分）

使用 nsys 分析前向传播、反向传播和优化器更新。自行从表 1 中选择两个模型规模，并选择三个大于 128 的、为 2 的幂的上下文长度。其中最大长度应为显存中能够容纳的最长上下文。选择你认为最值得观察的配置组合。对每份分析结果回答：

(a) 前向传播总共耗时多少？是否与先前使用 Python 标准库测得的时间一致？

提交内容：用 1-2 句话回答。

(b) 前向传播中，哪个 CUDA 内核的 GPU 累计耗时最多？模型一次前向传播中，该内核被调用多少次？同时运行前向和反向传播时，耗时最多的是否仍是这个内核？提示：查看“Stats System View”中的“CUDA GPU Kernel Summary”，并利用 NVTX 区间过滤，判断模型哪些部分对应哪些内核。

提交内容：用 1-2 句话回答。

(c) 虽然绝大多数 FLOPs 来自矩阵乘法，但你会发现其他几个内核仍占据不可忽略的总运行时间。前向传播中，除矩阵乘法之外，还有哪些内核消耗了不可忽略的 CUDA 执行时间？

提交内容：用 1-2 句话回答。

(d) 使用你实现的 AdamW 分析一个完整训练步骤，即前向传播、计算损失、反向传播以及最后的优化器更新，与实际训练一致。相较于仅前向的推理，矩阵乘法耗时的占比如何变化？其他内核呢？

提交内容：用 1-2 句话回答。

(e) 比较模型一次前向传播时，自注意力层内 softmax 运算与矩阵乘法运算的耗时。它们的耗时差异与 FLOPs 差异相比如何？

提交内容：用 1-2 句话回答。


```
    return ...
```

You can swap your original implementation with the annotated version in your benchmarking script via:

```
cs336_basics.model.scaled_dot_product_attention = annotated_scaled_dot_product_attention
```

Finally, it's worth noting that `torch.compile` can make it hard to attribute time and resources to specific parts of your code. You will likely have to wrap and strip various parts of your code in `torch.compile` and `nvtx` annotations to correctly attribute time and resource usage to various parts of your source.

Problem (`nsys_profile`): Nsight Systems Profiling (5 points)

Profile your forward pass, backward pass, and optimizer step using `nsys` with two model sizes from Table 1 of your choice as well as three power-of-two context lengths larger than 128, where the largest available size should be the longest context length you can fit in memory. Pick the combinations you think would be the most interesting to look at. For each profile answer the following questions:

(a) What is the total time spent on your forward pass? Does it match what we had measured before with the Python standard library?

Deliverable: A 1-2 sentence response.

(b) What CUDA kernel takes the most cumulative GPU time during the forward pass? How many times is this kernel invoked during a single forward pass of your model? Is it the same kernel that takes the most runtime when you do both forward and backward passes? (Hint: look at the “CUDA GPU Kernel Summary” under “Stats System View”, and filter using NVTX ranges to identify which parts of the model are responsible for which kernels.)

Deliverable: A 1-2 sentence response.

(c) Although the vast majority of FLOPs take place in matrix multiplications, you will notice that several other kernels still take a non-trivial amount of the overall runtime. What other kernels besides matrix multiplies do you see accounting for non-trivial CUDA runtime in the forward pass?

Deliverable: A 1-2 sentence response.

(d) Profile running one complete training step with your implementation of AdamW (i.e., the forward pass, computing the loss and running a backward pass, and finally an optimizer step, as you'd do during training). How does the fraction of time spent on matrix multiplication change, compared to doing inference (forward pass only)? How about other kernels?

Deliverable: A 1-2 sentence response.

(e) Compare the runtime of the softmax operation versus the matrix multiplication operations within the self-attention layer of your model during a forward pass. How does the difference in runtimes compare to the difference in FLOPs?

Deliverable: A 1-2 sentence response.

2.1.5 混合精度

目前为止，我们一直使用 FP32 精度，所有模型参数和激活的数据类型均为 torch.float32。但现代 NVIDIA GPU 包含用于加速低精度矩阵乘法的专用核心，即 Tensor Cores。例如，NVIDIA B200 规格表给出的 FP32 最大吞吐量是 80 TFLOPS，而 FP16（半精度浮点）或 BF16（bfloat16）的最大吞吐量高达 2500 TFLOPS。因此，使用较低精度的数据类型应当有助于加快训练和推理。

然而，简单地将模型转换为低精度格式可能降低模型准确性。例如，实际中许多梯度过小，无法用 FP16 表示；直接使用 FP16 训练时，它们会变成零。为解决这一问题，FP16 训练通常使用损失缩放：将损失乘以一个缩放因子，提高梯度的幅度，避免其下溢为零。另外，FP16 的动态范围小于 FP32，可能发生溢出，表现为损失变成 NaN。纯 bfloat16 训练通常更稳定，因为 BF16 与 FP32 的动态范围相同，但最终模型表现仍可能受影响。

为利用低精度的速度优势，通常采用混合精度训练。PyTorch 通过 torch.autocast 上下文管理器实现这一功能：某些运算（例如矩阵乘法）使用低精度数据类型，而需要 FP32 完整动态范围的其他运算（例如累加和归约）保持原有精度。

右侧示例会自动识别前向传播中哪些运算应使用低精度，并将其转换为指定类型。示例中 model 是模型（例如 Transformer），dtype 是目标精度（例如 torch.bfloat16），x 是输入数据；在 torch.autocast(device_type="cuda", dtype=dtype) 区间内调用模型。

如上所述，即使待累加的张量已经转换为低精度，累加本身通常仍应保持较高精度。下面的练习有助于理解原因。

题目（mixed_precision_accumulation）：混合精度累加（1 分）

运行右侧代码，并评价结果的准确性。

代码依次比较四种累加方式：FP32 累加器加 FP32 的 0.01；FP16 累加器加 FP16 的 0.01；FP32 累加器加 FP16 的 0.01；以及先将 FP16 的 0.01 转换为 FP32，再加入 FP32 累加器。每种方式都累加 1000 次并打印结果。代码在下一页接续。

2.1.5 Mixed Precision

Up to this point in the assignment, we've been running with FP32 precision—all model parameters and activations have the `torch.float32` datatype. However, modern NVIDIA GPUs contain specialized GPU cores (Tensor Cores) for accelerating matrix multiplies at lower precisions. For example, the NVIDIA B200 spec sheet says that its maximum throughput with FP32 is 80 TFLOPS, while its maximum throughput with FP16 (half-precision floats) or BF16 (bfloat16) is significantly higher at a whopping 2500 TFLOPS. As a result, using lower-precision datatypes should help us speed up training and inference.

However, naïvely casting our model into a lower-precision format may come with reduced model accuracy. For example, many gradient values in practice are often too small to be representable in FP16, and thus become zero when naïvely training with FP16 precision. To combat this, it's common to use loss scaling when training with FP16—the loss is simply multiplied by a scaling factor, increasing gradient magnitudes so they don't flush to zero. Furthermore, FP16 has a lower dynamic range than FP32, which can lead to overflows that manifest as a NaN loss. Full bfloat16 training is generally more stable (since BF16 has the same dynamic range as FP32), but can still affect final model performance compared to FP32.

To take advantage of the speedups from lower-precision datatypes, it's common to use *mixed-precision* training. In PyTorch, this is implemented with the `torch.autocast` context manager. In this case, certain operations (e.g., matrix multiplies) are performed in lower-precision datatypes, while other operations that require the full dynamic range of FP32 (e.g., accumulations and reductions) are kept as-is. For example, the following code will automatically identify which operations to perform in lower-precision during the forward pass and cast these operations to the specified data type:

```
model : torch.nn.Module = ... # e.g. your Transformer model
dtype : torch.dtype = ... # e.g. torch.bfloat16
x : torch.Tensor = ... # input data

with torch.autocast(device_type="cuda", dtype=dtype):
    y = model(x)
```

As alluded to above, it is generally a good idea to keep accumulations in higher precision even if the tensors themselves being accumulated have been downcast. The following exercise will help build your intuition as to why this is the case.

Problem (`mixed_precision_accumulation`): Mixed-Precision Accumulation (1 point)

Run the following code and comment on the accuracy of the results.

```
s = torch.tensor(0,dtype=torch.float32)
for i in range(1000):
    s += torch.tensor(0.01,dtype=torch.float32)
print(s)

s = torch.tensor(0,dtype=torch.float16)
for i in range(1000):
    s += torch.tensor(0.01,dtype=torch.float16)
print(s)

s = torch.tensor(0,dtype=torch.float32)
for i in range(1000):
```

上一页混合精度累加题的代码在右侧继续。

提交内容：用 2-3 句话回答。

接下来先在一个小模型上使用混合精度，建立直观理解，再将其应用到基准测试脚本中。

题目（benchmarking_mixed_precision）：混合精度基准测试（2 分）

(a) 考虑右侧给出的 ToyModel。它包含：从 in_features 到 10 维的无偏置线性层 fc1，10 维 LayerNorm 层 ln，从 10 维到 out_features 的无偏置线性层 fc2，以及 ReLU。前向传播依次执行 fc1、ReLU、ln 和 fc2。

假设在 GPU 上训练该模型，且模型参数最初为 FP32。我们希望使用 FP16 自动转换混合精度。以下各项的数据类型分别是什么？

- autocast 上下文中的模型参数。
- 第一个前馈层 ToyModel.fc1 的输出。
- 层归一化 ToyModel.ln 的输出。
- 模型预测的 logits。
- 损失。
- 模型梯度。

提交内容：给出以上各项的数据类型。

(b) 你应当已经观察到，FP16 混合精度自动转换对层归一化层和前馈层的处理不同。层归一化的哪些部分对混合精度敏感？如果使用 BF16 而非 FP16，仍需要区别对待层归一化吗？为什么？

提交内容：用 2-3 句话回答。


```
    s += torch.tensor(0.01,dtype=torch.float16)
print(s)

s = torch.tensor(0,dtype=torch.float32)
for i in range(1000):
    x = torch.tensor(0.01,dtype=torch.float16)
    s += x.type(torch.float32)
print(s)
```

Deliverable: A 2-3 sentence response.

We will now apply mixed precision first to a toy model to build intuition and then to our benchmarking script.

Problem (`benchmarking_mixed_precision`): Benchmarking Mixed Precision (2 points)

(a) Consider the following model:

```
class ToyModel(nn.Module):
    def __init__(self, in_features: int, out_features: int):
        super().__init__()
        self.fc1 = nn.Linear(in_features, 10, bias=False)
        self.ln = nn.LayerNorm(10)
        self.fc2 = nn.Linear(10, out_features, bias=False)
        self.relu = nn.ReLU()

    def forward(self, x):
        x = self.relu(self.fc1(x))
        x = self.ln(x)
        x = self.fc2(x)
        return x
```

Suppose we are training the model on a GPU and that the model parameters are originally in FP32. We'd like to use autocasting mixed precision with FP16. What are the data types of:

- the model parameters within the autocast context?
- the output of the first feed-forward layer (`ToyModel.fc1`)?
- the output of layer norm (`ToyModel.ln`)?
- the model's predicted logits?
- the loss?
- the model's gradients?

Deliverable: The data types for each of the components listed above.

(b) You should have seen that FP16 mixed precision autocasting treats the layer normalization layer differently than the feed-forward layers. What parts of layer normalization are sensitive to mixed precision? If we use BF16 instead of FP16, do we still need to treat layer normalization differently? Why or why not?

Deliverable: A 2-3 sentence response.

中文译文 / 对应原文第 9 页

(c) 修改基准测试脚本，使其可以选择用 BF16 混合精度运行模型。对第 2.1.2 节中的每种语言模型规模，分别测量使用和不使用混合精度时的前向与反向传播时间。比较全精度与混合精度的结果，并讨论模型规模变化时的趋势。无操作的上下文管理器 nullcontext 可能有用。

提交内容：用 2-3 句话回答，给出时间与评论。

2.1.6 显存分析

此前我们关注计算性能。现在将注意力转向显存，这是语言模型训练与推理中的另一项重要资源。PyTorch 提供强大的显存分析器，可以跟踪随时间变化的显存分配。

使用时，可以按右侧示例修改基准测试脚本：在预热阶段结束后，用 torch.cuda.memory._record_memory_history(max_entries=1000000) 开始记录；执行需要分析的工作；用 torch.cuda.memory._dump_snapshot("memory_snapshot.pickle") 保存快照；最后用 torch.cuda.memory._record_memory_history(enabled=None) 停止记录。

这会输出 memory_snapshot.pickle，可以加载到在线工具 pytorch.org/memory_viz。该工具展示整体显存使用时间线，以及每次独立分配的大小和对应的代码调用栈。在浏览器中打开上述地址，将 pickle 文件拖入页面即可。

接下来使用 PyTorch 分析器研究模型的显存开销。

题目（memory_profiling）：显存分析（4 分）

使用表 1 的 xl 模型，上下文长度分别为 128 和 2048，分析由前向传播、反向传播和优化器更新组成的完整训练步骤。

(a) 在性能分析脚本中添加一个选项，通过显存分析器运行模型。可以复用已有功能，例如启用混合精度、加载指定规模模型等。然后运行脚本，得到 xl 模型仅推理（只前向）与完整训练步骤的显存分析结果。显存时间线是什么样的？是否能根据峰值判断正在执行哪个阶段？

提交内容：memory_viz 工具中 xl 模型的两张“Active memory timeline”截图，一张为前向传播，一张为完整训练步骤（前向、反向及优化器更新）；另用 2-3 句话回答。

(c) Modify your benchmarking script to optionally run the model using mixed precision with BF16. Time the forward and backward passes with and without mixed-precision for each language model size described in Section 2.1.2. Compare the results of using full precision versus mixed precision, and comment on any trends as model size changes. You may find the `nullcontext` no-op context manager to be useful.

Deliverable: A 2-3 sentence response with your timings and commentary.

2.1.6 Profiling Memory

So far, we have been looking at compute performance. We'll now shift our attention to *memory*, another major resource in language model training and inference. PyTorch also ships with a powerful memory profiler, which can keep track of allocations over time.

To use the memory profiler, you can modify your benchmarking script as follows:

```
... # warm-up phase in your benchmarking script

# Start recording memory history.
torch.cuda.memory._record_memory_history(max_entries=1000000)

... # what you want to profile in your benchmarking script

# Save a pickle file to be loaded by PyTorch's online tool.
torch.cuda.memory._dump_snapshot("memory_snapshot.pickle")

# Stop recording history.
torch.cuda.memory._record_memory_history(enabled=None)
```

This will output a file `memory_snapshot.pickle` that you can load into the following online tool: `pytorch.org/memory_viz`. This tool will let you see the overall memory usage timeline as well as each individual allocation that was made, with its size and a stack trace leading to the code where it originates. To use this tool, you should open the link above in a web browser, and then drag and drop your pickle file onto the page.

You will now use the PyTorch profiler to analyze the memory usage of your model.

Problem (`memory_profiling`): Memory Profiling (4 points)

Profile your complete training step of forward pass, backward pass, and optimizer step of the `xl` model from Table 1 with context lengths of 128 and 2048.

(a) Add an option to your profiling script to run your model through the memory profiler.

It may be helpful to reuse some of your previous infrastructure (e.g., to activate mixed-precision, load specific model sizes, etc). Then, run your script to get a memory profile of the `xl` model when either doing inference only (just forward pass) or a full training step. What do your memory timelines look like? Can you tell which stage is running based on the peaks you see?

Deliverable: Two images of the "Active memory timeline" of an `xl` model, from the `memory_viz` tool: one for the forward pass, and one for running a full training step (forward and backward passes, then optimizer step), and a 2-3 sentence response.

(b) 每种上下文长度在前向传播时的峰值显存是多少？完整训练步骤呢？

提交内容：一张表，每种上下文长度对应两个数值。

(c) 使用混合精度，分别测量 xl 模型前向传播和完整训练步骤的峰值显存。混合精度是否显著影响显存开销？

提交内容：用 2-3 句话回答。

(d) 考虑 xl 模型。按照参考超参数，Transformer 残差流中的一个单精度激活张量有多大？以 MiB 表示，即将字节数除以 1024^2。

提交内容：用 1-2 句话回答，给出推导。

(e) 仔细查看 pytorch.org/memory_viz 中 xl 模型前向传播快照的“Active Memory Timeline”。降低“Detail”级别时，工具会隐藏对应范围内较小的分配。例如，设为 10% 时只显示最大的 10% 分配。显示的最大分配有多大？根据调用栈，能否判断这些分配来自哪里？

提交内容：用 1-2 句话回答。

(f) Nsight Systems 也有显存分析选项。将其与此前的 Nsight 选项结合，可以了解模型生命周期中不同步骤的显存分配。使用 PyTorch 提供的 NVTX 标签，判断模型中的单个 TransformerBlock 为反向传播保存了多少显存（这些张量通常称为 residuals）。记录贡献最大的 5 项运算，以及它们占总体显存的百分比。

反向传播中，这些张量会被释放，但同时也会产生新的梯度张量。根据前向传播中的分配量，以及反向传播中每个 TransformerBlock 引起的显存变化，计算一个 TransformerBlock 产生的梯度张量占用多少显存。结果是否符合预期？

提交内容：Nsight Systems 截图，以及 1-2 段回答。

3 单 GPU 显存

后续部分将探索如何把张量分片到多个 GPU，但也有一些技巧可以用于单 GPU 训练。最常见的是梯度检查点，也称激活检查点。

3.1 自动微分保存的中间张量

回顾：要执行模型的反向传播，需要保存前向传播产生的激活。对某些运算，这显而易见；但默认情况下，保存激活的运算往往比你预想的更多。为反向传播保存的张量称为“residuals”，或简称“saved tensors”（保存的张量）。

先了解网络保存了什么。从一个看似简单的 RMSNorm 函数开始，为简化讨论使用纯 FP32，并在自动微分保存或读取张量时添加钩子。

(b) What is the peak memory usage of each context length when doing a forward pass? What about when doing a full training step?

Deliverable: A table with two numbers per context length.

(c) Find the peak memory usage of the `xl` model when using mixed-precision, for both a forward pass and a full training step. Does mixed-precision significantly affect memory usage?

Deliverable: A 2-3 sentence response.

(d) Consider the `xl` model. Given our reference hyperparameters, what is the size of a tensor of activations in the Transformer residual stream, in single-precision? Give this size in MiB (i.e., divide the number of bytes by 1024^2).

Deliverable: A 1-2 sentence response with your derivation.

(e) Now look closely at the “Active Memory Timeline” from `pytorch.org/memory_viz` of a memory snapshot of the `xl` model doing a forward pass. When you reduce the “Detail” level, the tool hides the smallest allocations to the corresponding level (e.g., putting “Detail” at 10% only shows the 10% largest allocations). What is the size of the largest allocations shown? Looking through the stack trace, can you tell where those allocations come from?

Deliverable: A 1-2 sentence response.

(f) Nsight Systems also has flags for memory profiling. You can combine these with the Nsight flags from before to understand what allocations are happening at different steps in your model's lifespan. Use the PyTorch-provided NVTX labels to determine how much memory is saved for backward (these tensors are often called residuals) by a single `TransformerBlock` in your model. Note the 5 largest contributing operations, and what percentage of the overall memory they contribute.

During the backward pass, all these tensors will be freed, but new gradient tensors are emitted at the same time. Based on your profiles showing how much memory was allocated during the forward pass, and how much memory usage changes for every `TransformerBlock` in the backward pass, calculate how much memory the produced gradient tensors for a `TransformerBlock` take. Does the result match what you expect?

Deliverable: Screenshots from Nsight Systems and a 1-2 paragraph response.

3 Single-GPU Memory

The later parts of this assignment will explore tricks to shard your tensors across multiple GPUs, but there are also tricks that can be applied even to single-GPU training. The most common of these is gradient checkpointing (also known as activation checkpointing).

3.1 Autograd Residuals

Recall that in order to perform a backward pass through your model, we need to save the activations that were produced in the forward pass. While this is obviously the case for some operations, by default it'll happen for many more than you might expect. The tensors saved for the backward pass are called “residuals”, or simply “saved tensors”.

Let's build some understanding of what's being saved in our network. Starting with our unassuming RMSNorm function (pure FP32 for simplicity), let's add some hooks for when tensors are being saved or retrieved by autograd.

中文译文 / 对应原文第 11 页

右侧完整保留 RMSNorm 与自动微分钩子的示例代码。

示例创建形状为 (4, 512, 2560)、需要梯度的随机输入 x。RMSNorm 的可学习权重初始化为全 1，eps 默认为 1e-5；前向传播计算均方根的倒数，将输入乘以该值，再乘以权重。

pack_hook 在张量被保存时打印其 shape、dtype 和 grad_fn，并返回该张量；unpack_hook 在张量被读取时打印相同信息并返回张量。随后在 torch.autograd.graph.saved_tensors_hooks(pack_hook, unpack_hook) 上下文内，运行 RMSNorm 前向传播，并对 y.sum() 进行反向传播。

输出显示，被保存的张量数量多得令人担忧，其中几个的大小与完整激活相同！

右侧输出中，“Saving residual”表示“保存供反向传播使用的张量”；“Loading residual”表示“读取供反向传播使用的张量”。

保存记录依次包含：

• (4, 512, 2560)，FP32，grad_fn=None。

• 两个 (4, 512, 1)，FP32，关联 RsqrtBackward0。

• (4, 512, 2560)，FP32，grad_fn=None。

• (4, 512, 2560)，FP32，关联 MulBackward0。

• (2560)，FP32，grad_fn=None。

读取记录从关联 MulBackward0 的完整激活张量开始，其余记录在下一页继续。原始运行命令与完整终端输出保留在右侧。


```python
import torch
from torch import nn

x = torch.randn((4, 512, 2560), requires_grad=True)


class RMSNorm(nn.Module):
    def __init__(
        self,
        hidden_size: int,
        eps: float = 1e-5,
        device=None,
    ):
        super().__init__()
        self.weight = nn.Parameter(torch.ones(hidden_size, device=device))
        self.eps = eps

    def forward(self, x):
        rms = torch.rsqrt(x.pow(2).mean(-1, keepdim=True) + self.eps)
        x = x * rms
        return self.weight * x


def pack_hook(t):
    shape, dtype, grad_fn = t.shape, t.dtype, t.grad_fn
    print(f"Saving residual: {shape=}, {dtype=}, {grad_fn=}")
    return t

def unpack_hook(t):
    shape, dtype, grad_fn = t.shape, t.dtype, t.grad_fn
    print(f"Loading residual: {shape=}, {dtype=}, {grad_fn=}")
    return t

ln = RMSNorm(x.shape[-1])

with torch.autograd.graph.saved_tensors_hooks(pack_hook, unpack_hook):
    y = ln(x)
    y.sum().backward()
```

The output shows a worrying amount of tensors being written out, several of them at full activation size!

```
$ uv run scripts/autograd_experiment.py
Saving residual: shape=torch.Size([4, 512, 2560]), dtype=torch.float32, grad_fn=None
Saving residual: shape=torch.Size([4, 512, 1]), dtype=torch.float32, grad_fn=<RsqrtBackward0
object at 0x7f7dd319b5e0>
Saving residual: shape=torch.Size([4, 512, 1]), dtype=torch.float32, grad_fn=<RsqrtBackward0
object at 0x7f7dd319b5e0>
Saving residual: shape=torch.Size([4, 512, 2560]), dtype=torch.float32, grad_fn=None
Saving residual: shape=torch.Size([4, 512, 2560]), dtype=torch.float32, grad_fn=<MulBackward0
object at 0x7f7dd319b5e0>
Saving residual: shape=torch.Size([2560]), dtype=torch.float32, grad_fn=None

Loading residual: shape=torch.Size([4, 512, 2560]), dtype=torch.float32,
grad_fn=<MulBackward0 object at 0x7f7cf14e6740>
```

右侧顶部接续上一页的张量读取记录。

3.1.1 算子融合

在此例中，显然拆分出的操作过多、粒度过细。我们希望只有一个操作：接收 RMSNorm 权重和激活，输出结果；反向传播中也将其作为一个整体。这是进行内核融合的一个动机。由于 RMSNorm 的计算结构较规整，甚至可以使用 torch.compile 自动融合。

右侧示例用 torch.compile 包装 RMSNorm，再在同样的保存/读取钩子上下文中执行前向与反向传播。

新的输出明显更好：仅保存并读取一个 (4, 512, 2560) 的输入激活张量、一个 (2560) 的权重张量以及一个 (4, 512, 1) 的张量，均为 FP32，grad_fn=None。

反向传播只需保存一个完整大小的激活张量，即 RMSNorm 函数的输入。还要注意，读取次序不再是保存次序的逆序，每个保存的张量也不再有 grad_fn 依赖：PyTorch 正将整个 RMSNorm 视为单个函数。

3.2 激活检查点

融合确实有用，但它对显存节省的帮助有限。例如，我们对 xl 规模的单个 TransformerBlock 进行融合。

右侧示例导入 RotaryEmbedding 和 TransformerBlock，并设置 d_model=2560、d_ff=10240、num_heads=16、context_length=2048。代码注释指出，这个模型的层数为 32。构建块及旋转位置编码器后，使用 torch.compile(block, fullgraph=True)，尽可能进行融合；输入为需要梯度、形状 (4, context_length, d_model) 的随机张量。

代码在下一页接续。


```
Loading residual: shape=torch.Size([2560]), dtype=torch.float32, grad_fn=None
Loading residual: shape=torch.Size([4, 512, 1]), dtype=torch.float32, grad_fn=<RsqrtBackward0
object at 0x7f7cf14e6740>
Loading residual: shape=torch.Size([4, 512, 2560]), dtype=torch.float32, grad_fn=None
Loading residual: shape=torch.Size([4, 512, 1]), dtype=torch.float32, grad_fn=<RsqrtBackward0
object at 0x7f7cf14e6740>
Loading residual: shape=torch.Size([4, 512, 2560]), dtype=torch.float32, grad_fn=None
```

3.1.1 Operator Fusion

In this case, it's clear that the granularity of the operations used is too high. We want a single op that takes in the RMSNorm weights and the activation, and spits out the output, as well as for that operation to be unitary in the backward pass. This is one motivation for kernel fusion. Since the RMSNorm is fairly well behaved, we can even automatically fuse it using `torch.compile`.

```
...
ln = torch.compile(RMSNorm(x.shape[-1]))

with torch.autograd.graph.saved_tensors_hooks(pack_hook, unpack_hook):
    y = ln(x)
    y.sum().backward()
```

The new output is significantly better:

```
Saving residual: shape=torch.Size([4, 512, 2560]), dtype=torch.float32, grad_fn=None
Saving residual: shape=torch.Size([2560]), dtype=torch.float32, grad_fn=None
Loading residual: shape=torch.Size([4, 512, 2560]), dtype=torch.float32, grad_fn=None
Loading residual: shape=torch.Size([2560]), dtype=torch.float32, grad_fn=None
Loading residual: shape=torch.Size([4, 512, 1]), dtype=torch.float32, grad_fn=None
```

We only need to save a single full-size activation tensor for the backward pass — namely, the input to the RMSNorm function. Notice also how the order of loading is no longer the reverse of saving, and each residual no longer has a `grad_fn` dependency — PyTorch is treating the entirety of our RMSNorm as a single function.

3.2 Activation Checkpointing

While fusion is undoubtedly useful, it can only get us so far in saving memory. For instance, let's fuse a single TransformerBlock at size `xl`.

```python
import torch
from cs336_basics.model import RotaryEmbedding, TransformerBlock

# num_layers for this model is 32
d_model, d_ff, num_heads, context_length = 2560, 10240, 16, 2048
block = TransformerBlock(d_model=d_model, d_ff=d_ff, num_heads=num_heads,
positional_encoder=RotaryEmbedding(dim=d_model // num_heads, context_length=context_length))

# Fuse as much torch.compile will allow
block = torch.compile(block, fullgraph=True)
x = torch.randn((4, context_length, d_model), requires_grad=True)
```

中文译文 / 对应原文第 13 页

右侧代码接续上一页。pack_hook 现在还会累计保存张量的字节数：遇到 torch.nn.Parameter 时直接返回，避免重复统计参数；其他张量按元素数乘每元素字节数计入 total_size_bytes，然后打印形状、类型和 grad_fn。

在保存/读取钩子上下文中运行前向传播，为反向传播保存张量；最后打印单个 TransformerBlock 保存张量的总大小，单位为 MiB。

脚本输出：单个 TransformerBlock 保存张量的总大小为 3651.31 MiB。

每层约 3.6 GiB。如果所有层都如此，仅为反向传播保存的激活就达到 114 GiB！注意力运算保存的中间张量中存在相当大的浪费，第 4 节将对此改进；但即使改进后，显存使用量仍会随批大小、序列长度和嵌入维度线性增长。

3.2.1 重计算

与其保留每个生成的张量，不如只周期性地保存计算结果的检查点，并重新计算检查点之间的值。PyTorch 提供了简单接口：torch.utils.checkpoint.checkpoint 接收一个函数及该函数的参数，然后改变其行为：

1. 前向传播中：

① 保存函数的输入值。

② 禁止保存该函数前向传播中的张量。

2. 反向传播中：

① 先增加一个重计算步骤，从此前保存的输入重新执行前向传播，并保存反向传播所需的值。

② 执行反向传播，随后所有这些张量均可释放。

在连续运行 4 个 TransformerBlock 的简单情形中，显存开销会按预期累加。右侧给出的 four_blocks 函数连续调用四次 block；后续代码与结果见下一页。


```
...
# Now logs the number of bytes saved
total_size_bytes = 0
def pack_hook(t):
    if isinstance(t, torch.nn.Parameter):  # Skip logging parameters to avoid double counting
        return t
    global total_size_bytes
    shape, dtype, grad_fn = t.shape, t.dtype, t.grad_fn
    total_size_bytes += t.numel() * t.element_size()
    print(f"Saving residual: {shape=}, {dtype=}, {grad_fn=}")
    return t

...

# Run forward pass, saving for backward
with torch.autograd.graph.saved_tensors_hooks(pack_hook, unpack_hook):
    y = block(x)

print(f"Total size of saved tensors in single TransformerBlock: {total_size_bytes /
(1024**2):.2f} MiB")
```

The script shows us how much memory we're saving for backward:

```
...
Total size of saved tensors in single TransformerBlock: 3651.31 MiB
```

3.6 GiB for every layer. If we do this for all layers, we get 114 GiB of activations, just saved for backward! There's a nontrivial amount of waste in the attention operation's residuals, which we will fix in Section 4, but even with this fix, the memory use will grow linearly with batch size, sequence length and embedding size.

3.2.1 Recomputation

Instead of holding on to every tensor we generate, it's possible to save only periodic checkpoints of our results, and recompute the values in-between. PyTorch has an interface we can call to handle this in a simple fashion. `torch.utils.checkpoint.checkpoint` takes in a function, and arguments to that function. It then modifies the behavior of the function passed by:

1. In the forward pass:
    1. Saving the input values to the function
    2. Suppressing the saving of tensors in the forward pass
2. In the backward pass:
    1. Prepending a recomputation step where the forward pass is recomputed from the previously saved inputs, and values are saved for backward
    2. The backward pass is run and all tensors can be freed

In the simple case of running through 4 transformer blocks, we see that our memory adds up as we would expect.

```
...
def four_blocks(x):
    x = block(x)
```

中文译文 / 对应原文第 14 页

右侧先给出四个 TransformerBlock 不使用检查点时的代码及输出：保存张量的总大小为 14605.25 MiB。

随后给出梯度检查点版本：two_blocks 函数连续执行两个块；four_blocks_checkpoint 对 two_blocks 调用两次 checkpoint，均指定 use_reentrant=False。

代码注释含义：checkpoint 会丢弃保存的张量，直到反向传播；反向传播到达检查点覆盖的块时，重新执行一次前向传播，生成需要保存的张量，然后完成正常的反向传播。

保存记录包含两个空张量以及两个形状为 (4, 2048, 2560) 的 FP32 张量，其中后一个关联 CompiledFunctionBackward。使用检查点后，四个 TransformerBlock 保存张量的总大小为 160.00 MiB。

注意，显存使用并没有消失，而是分成两类：一类是在每次 checkpoint 调用入口处，为后续重计算长期保留的检查点；另一类是在被检查点覆盖的块内部，重计算时临时生成、用于完成反向传播的张量。

由于主要关注峰值显存，我们要平衡保存检查点的开销与完整生成一个块所需中间张量的开销。检查点越多，单个检查点区块内需要生成的张量越少；但检查点自身占用的显存就越大。


```
    x = block(x)
    x = block(x)
    x = block(x)
    return x


with torch.autograd.graph.saved_tensors_hooks(pack_hook, unpack_hook):
    y = four_blocks(x)

print(f"Total size of saved tensors in four TransformerBlocks: {total_size_bytes /
(1024**2):.2f} MiB")
```

```
Total size of saved tensors in four TransformerBlocks: 14605.25 MiB
```

But we can employ gradient checkpointing as follows:

```
from torch.utils.checkpoint import checkpoint
def two_blocks(x):
    x = block(x)
    x = block(x)
    return x


def four_blocks_checkpoint(x):
    # checkpoint throws out all the saved tensors until the backward pass
    # when getting to the checkpointed block in the backward pass,
    # it reruns a forward pass to produce the saved tensors,
    # then completes normal backward pass.
    x = checkpoint(two_blocks, x, use_reentrant=False)
    x = checkpoint(two_blocks, x, use_reentrant=False)
    return x

with torch.autograd.graph.saved_tensors_hooks(pack_hook, unpack_hook):
    y = four_blocks_checkpoint(x)

print(f"Total size of saved tensors in four TransformerBlocks with checkpointing:
{total_size_bytes / (1024**2):.2f} MiB")
```

```
Saving residual: shape=torch.Size([0]), dtype=torch.float32, grad_fn=None
Saving residual: shape=torch.Size([4, 2048, 2560]), dtype=torch.float32, grad_fn=None
Saving residual: shape=torch.Size([0]), dtype=torch.float32, grad_fn=None
Saving residual: shape=torch.Size([4, 2048, 2560]), dtype=torch.float32,
grad_fn=<torch.autograd.function.CompiledFunctionBackward object at 0x7aa0657a19d0>
Total size of saved tensors in four TransformerBlocks with checkpointing: 160.00 MiB
```

Keep in mind this hasn't eliminated the memory use. Rather, it's factored our memory use into two categories: the longer term storage we save to prepare for recomputation at the entry point of each `checkpoint` call (the checkpoint itself), and the short term memory generated in the recomputation pass within the checkpointed block to facilitate a backward pass through it. Since our main consideration is the peak memory usage, we want to balance the memory cost of the saved checkpoints with the memory cost of materializing a full block worth of residuals. The more checkpoints we use, the smaller the amount of materialized memory within a single block, but the more memory we need for the checkpoints themselves.

在上述例子中，重计算产生的中间张量显然占主导，部分原因是检查点很小。因此，将更多显存预算用于检查点会有益，也就是缩小单个检查点区块的范围。

检查点区块大小本身不会改变重计算的计算开销。不过，可以通过递归检查点继续降低显存需求，代价是增加计算量；递归检查点指在 checkpoint 调用内部嵌套其他 checkpoint 调用。

题目（gradient_checkpointing）：显存最优的梯度检查点（4 分）

考虑一个由 N 个相同块顺序堆叠而成的 Transformer。不使用检查点时，N 个块的全部中间张量会同时存活，因而激活峰值显存为 O(N)。我们可以自由地将前向传播的任意部分包装在 checkpoint 中，包括相互嵌套的调用。

(a) 忽略计算成本，什么检查点策略能使激活峰值显存最小？说明如何安排 checkpoint 调用（可以给代码草图），并给出该策略的峰值激活显存与计算量关于 N 的渐近复杂度。假设单个块保存的中间张量开销远大于每个检查点的管理开销。

提交内容：用 3-5 句话描述策略及其渐近峰值显存，并附简短代码草图。

(b) 考虑上述 xl 配置，批大小为 4、序列长度为 2048。如果时间/计算预算只允许一层重计算，即不能嵌套 checkpoint 调用，怎样设置检查点最能降低峰值显存？分析运行中的峰值显存，验证你的假设。并与相邻的更小、更大的检查点区块大小比较，确认结论。

提交内容：用 3-5 句话描述推理，并给出策略的实测峰值显存。

4 GPU 内核

4.1 使用 FlashAttention-2 优化注意力

4.1.1 PyTorch 注意力基准测试

分析结果很可能表明，注意力层在显存和计算两方面都有优化机会。概括来说，注意力由矩阵乘法、softmax 和另一矩阵乘法组成，表达式见右侧公式（1）。

朴素实现需要为每个批次/注意力头元素保存形状为 seq_len × seq_len 的注意力分数矩阵。序列较长时矩阵会非常大，导致长输入或长输出任务显存不足。我们将依据 FlashAttention-2 论文实现一个注意力内核，通过分块计算，避免显式生成完整的 seq_len × seq_len 矩阵，从而支持长得多的序列。

In our example above, it's clear that the recomputed residuals' memory is dominating (in part because our checkpoints are tiny), so it would be beneficial to shift the balance more toward checkpointed memory. Thus we would want to decrease the scope of a checkpointed block.

Having larger or smaller checkpointed blocks doesn't affect the computational cost of recomputation, but we can continue to reduce the required memory at the cost of compute by using *recursive checkpointing*, meaning we nest `checkpoint` calls inside other `checkpoint` calls.

Problem (`gradient_checkpointing`): Memory-Optimal Gradient Checkpointing (4 points)

Consider a Transformer with N identical blocks stacked sequentially. Without any checkpointing, all N blocks' worth of residuals are kept alive simultaneously, giving $O(N)$ peak activation memory. We have a free hand to wrap any subset of the forward pass in `checkpoint`, including nesting `checkpoint` calls inside one another.

(a) What checkpointing strategy minimizes peak activation memory, ignoring the compute cost? Describe how you would arrange the `checkpoint` calls (a code sketch is fine), and give the asymptotic peak activation memory and compute of your strategy as a function of N. Assume the residuals saved by a single block dominate any per-checkpoint bookkeeping.

Deliverable: A 3-5 sentence description of the strategy and its asymptotic peak memory, plus a short code sketch.

(b) Consider the `xl` model config with batch size 4 and sequence length 2048 as above. If you only have the time/compute budget to run one step of recomputation (meaning you may not nest `checkpoint` calls), what is the best checkpointing strategy to reduce peak memory? Profile your run's peak memory to validate your hypothesis. Compare the peak memory of the next smaller and larger checkpointing block sizes to be sure.

Deliverable: A 3-5 sentence description of your reasoning along with the measured peak memory for your strategy.

4 GPU Kernels

4.1 Optimizing Attention with FlashAttention-2

4.1.1 Benchmarking PyTorch Attention

Your profiling likely suggests that there is an opportunity for optimization, both in terms of memory and compute, in your attention layers. At a high level, the attention operation consists of a matrix multiplication followed by softmax, then another matrix multiplication:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\text{mask}\left(\frac{QK^T}{\sqrt{d_k}}\right)\right)V \tag{1}$$

The naïve attention implementation needs to save attention score matrices of shape `seq_len` × `seq_len` for each batch/head element, which can grow very large with long sequence lengths, causing out-of-memory errors for any tasks with long inputs or outputs. We will implement an attention kernel following the FlashAttention-2 paper, which computes attention by tiles and avoids ever explicitly materializing the `seq_len` × `seq_len` attention score matrices, enabling scaling to much longer sequence lengths.

题目（pytorch_attention）：PyTorch 注意力基准测试（2 分）

(a) 在不同规模下测试你的注意力实现。编写脚本：

① 固定批大小为 8，不使用多头注意力，即去掉头维度。

② 遍历头嵌入维度 d_model=[16, 32, 64, 128] 与序列长度 [256, 1024, 4096, 8192, 16384] 的笛卡尔积。

③ 创建相应大小的随机输入 Q、K、V。

④ 测量使用这些输入执行 100 次注意力前向传播的时间。

⑤ 测量反向传播开始前的显存使用量，并测量 100 次反向传播的时间。

⑥ 确保预热，并在每次前向/反向传播后调用 torch.cuda.synchronize()。

根据 GPU 的不同，部分配置预计会显存不足。报告这些配置的耗时或显存不足错误。在什么规模下发生显存不足？选一个导致显存不足的较小配置，核算其注意力显存开销，可以使用作业 1 中 Transformer 显存开销的公式。为反向传播保存的显存如何随序列长度变化？你会怎样消除这项显存成本？

提交内容：耗时表、显存开销计算，以及 1-2 段回答。

4.2 即时编译注意力的基准测试

从 2.0 版开始，PyTorch 提供强大的即时编译器，自动尝试对 PyTorch 函数应用多种优化。入门介绍：

https://pytorch.org/tutorials/intermediate/torch_compile_tutorial.html

尤其是，它会动态分析计算图，尝试自动生成融合的 Triton 内核。

编译接口很简单。例如，对某一层应用编译器，可以先创建 layer = SomePyTorchModule(...)，再调用 compiled_layer = torch.compile(layer)。

compiled_layer 在功能上与 layer 一样，包括前向与反向传播。还可以通过 torch.compile(model) 编译整个 PyTorch 模型，甚至编译调用 PyTorch 运算的 Python 函数。

题目（torch_compile）：Torch Compile（2 分）

(a) 扩展注意力基准测试脚本，加入已编译的 PyTorch 注意力实现，并使用上方 pytorch_attention 题目相同的配置，与未编译实现比较性能。

Problem (`pytorch_attention`): PyTorch Attention Benchmarking (2 points)

(a) Benchmark your attention implementation at different scales. Write a script that will:

(i) Fix the batch size to 8 and don't use multihead attention (i.e. remove the head dimension).

(ii) Iterate through the cartesian product of `[16, 32, 64, 128]` for the head embedding dimension d_{model}, and `[256, 1024, 4096, 8192, 16384]` for the sequence length.

(iii) Create random inputs Q, K, V for the appropriate size.

(iv) Time 100 forward passes through attention using the inputs.

(v) Measure how much memory is in use before the backward pass starts, and time 100 backward passes.

(vi) Make sure to warm up, and to call `torch.cuda.synchronize()` after each forward/backward pass.

Depending on your GPU, some of these configurations are expected to run out of memory. Report the timings (or out-of-memory errors) you get for these configurations. At what size do you get out-of-memory errors? Do the accounting for the memory usage of attention in one of the smallest configurations you find that runs out of memory (you can use the equations for memory usage of Transformers from Assignment 1). How does the memory saved for backward change with the sequence length? What would you do to eliminate this memory cost?

Deliverable: A table with your timings, your calculations for the memory usage, and a 1-2 paragraph response.

4.2 Benchmarking JIT-Compiled Attention

Since version 2.0, PyTorch also ships with a powerful just-in-time compiler that automatically tries to apply a number of optimizations to PyTorch functions: see `https://pytorch.org/tutorials/intermediate/torch_compile_tutorial.html` for an intro. In particular, it will try to automatically generate fused Triton kernels by dynamically analyzing your computation graph. The interface for using the PyTorch compiler is very simple. For instance, if we wanted to apply it to a single layer of our model, we can use:

```
layer = SomePyTorchModule(...)
compiled_layer = torch.compile(layer)
```

Now, `compiled_layer` functionally behaves just like `layer` (e.g., with its forward and backward passes). We can also compile our entire PyTorch model with `torch.compile(model)`, or even a Python function that calls PyTorch operations.

Problem (`torch_compile`): Torch Compile (2 points)

(a) Extend your attention benchmarking script to include a compiled version of your PyTorch implementation of attention, and compare its performance to the uncompiled version with the same configuration as the `pytorch_attention` problem above.

提交内容：一张表，对比编译后的注意力模块与 pytorch_attention 题中未编译版本的前向、反向传播时间。

(b) 在端到端基准测试脚本中编译整个 Transformer 模型。前向传播性能如何变化？前向、反向与优化器更新合起来的性能呢？

提交内容：对比普通与编译后 Transformer 模型的表格。

根据此前观察到的序列长度扩展规律，要处理长序列，需要显著改进。即使使用 torch.compile，当前实现面对长序列时仍存在很差的显存访问模式。为此，我们将用 Triton 实现 FlashAttention-2，以更精细地控制显存访问，以及各项计算的时机。

4.2.1 示例：加权求和

为介绍所需的 Triton 知识及其与 PyTorch 的互操作，我们将逐步讲解一个“加权求和”内核。更多入门资源可见 Triton 教程。不过这些教程没有使用新的、更方便的块指针抽象；下面将对此进行介绍。

给定输入矩阵 X，用按列对应的权重向量 w 乘各元素，然后对每行求和，即得到 X 与 w 的矩阵-向量乘积。先讲解前向传播，再编写反向传播的 Triton 内核。

前向传播

前向传播就是右侧所示的广播内积：假设 x 是形状 [..., D] 的 n 维张量，weight 是形状 [D] 的一维张量，返回 (weight * x).sum(axis=-1)。

每个 Triton 程序实例（可能并行运行）计算 x 中一个行块的加权和，把对应标量写入输出张量。程序实例是运行同一程序的一组线程，这些线程块可以在 GPU 上并行运行。内核参数不是张量本身，而是张量首元素的指针及各轴步长，用来说明如何沿轴移动。

利用步长加载当前实例需要求和的行块，并用程序 ID 分配工作，即第 i 个实例处理第 i 个行块。本例中 Triton 与 PyTorch 前向传播的主要区别，是需要指针运算和显式加载/存储。tl.make_block_ptr 的块指针抽象可显著简化指针运算，但需要先做初始化设置。

Deliverable: A table comparing your forward and backward pass timings for your compiled attention module with the uncompiled version from the `pytorch_attention` problem above.

(b) Now, compile your entire Transformer model in your end-to-end benchmarking script. How does the performance of the forward pass change? What about the combined forward and backward passes and optimizer steps?

Deliverable: A table comparing your vanilla and compiled Transformer model.

Given the scaling behaviors we've seen with respect to the sequence length, we need significant improvements to handle large sequences. Even with `torch.compile`, the current implementation suffers from very poor memory access patterns at long sequence length. For that, we will write a Triton implementation of FlashAttention-2, where we'll have significantly more control over how memory is accessed and when to compute what.

4.2.1 Example - Weighted Sum

To introduce what you'll need to know about Triton and how it interoperates with PyTorch, we will work through an example kernel for a "weighted sum" operation. For further resources on getting up to speed with Triton, see Triton's tutorials. We note that these tutorials do not use the new, convenient block pointer abstraction, which we will walk through below.

Given an input matrix X, we'll multiply its entries by a column-wise weight vector w, and sum each row, giving us the matrix-vector product of X and w. We are going to work through the forward pass of this operation first, and then write the Triton kernel for the backward pass.

Forward pass

The forward pass of our kernel is just the following broadcasted inner product.

```
def weighted_sum(x, weight):
    # Here, assume that x has n-dim shape [..., D], and weight has 1D shape [D]
    return (weight * x).sum(axis=-1)
```

When writing our Triton kernel, we'll have each program instance (potentially running in parallel) compute the weighted sum of a *tile* of rows of x, and write the corresponding scalar outputs to the output tensor. In Triton, a program instance is a block of threads all running the same program, and these *thread blocks* can be run in parallel on the GPU. Instead of taking *tensors* as arguments, we take *pointers* to their first elements, as well as *strides* for each tensor that tell us how to move along axes.

We can use the strides to load a tensor corresponding to the tile of rows of x that we're summing in the running instance, using the program ID to divide up the work (i.e., instance i will process the i-th tile of rows of x). The main difference between the forward pass in Triton and PyTorch in this simple case is the need to do pointer arithmetic and explicit loads/stores. We will use the block pointer abstraction with `tl.make_block_ptr` to greatly simplify the pointer arithmetic, although this means we need to do some setup to prepare the block pointers.

图 2：加权求和内核示例（第 4.2.1 节）中的分块与块指针移动。

图中 ROWS 为总行数，D 为列/特征维度，ROWS_TILE 和 D_TILE 为块大小；.advance((0, D_TILE)) 沿列移动，.advance((ROWS_TILE, 0)) 沿行移动。

关于分块方式和块指针推进，参见图 2。前述加权求和函数对应的内核实现见右侧，下一页接续。

代码注释译文（按出现顺序）：

- x_ptr、weight_ptr：输入指针；output_ptr：输出指针。
- x_stride_row、x_stride_dim：步长说明沿张量各轴移动一个元素时指针的移动量。
- 权重维度步长及输出行步长通常为 1。
- ROWS_TILE_SIZE、D_TILE_SIZE 的块形状必须在编译时已知，因此使用 tl.constexpr。
- 每个实例计算 x 中一个行块的加权和。
- tl.program_id 用于检查当前运行的是哪个线程块。
- 块指针可以选择一个多维显存区域，并移动选区。
- 块指针需要知道：张量首元素指针；用于处理越界的完整张量形状；用于正确解释内存布局的各维步长；起始块的多维坐标 offsets；每次加载/存储的块形状；以及内存维度的排序（注释给出 np.argsort(strides)），用于优化，并是 Hopper 及之后架构的 TMA 支持所需的信息。

公式、代码与图表见右侧对应原页

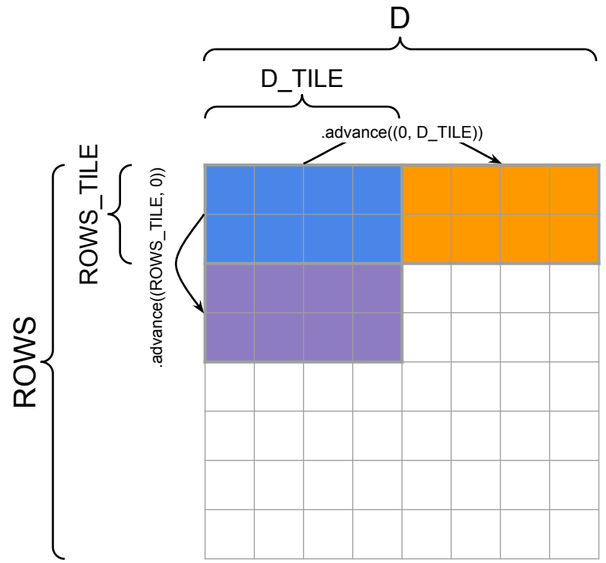


Figure 2: Tiling and advancing block pointers in the weighted sum kernel example (Section 4.2.1).

Refer to Figure 2 for a schematic of tiling and how block pointers are advanced. The weighted sum function from above looks like the following:

```
import triton
import triton.language as tl

@triton.jit
def weighted_sum_fwd(
    x_ptr, weight_ptr,  # Input pointers
    output_ptr,  # Output pointer
    x_stride_row, x_stride_dim,  # Strides tell us how to move one element in each axis of a tensor
    weight_stride_dim,  # Likely 1
    output_stride_row,  # Likely 1
    NUM_ROWS, D,
    ROWS_TILE_SIZE: tl.constexpr, D_TILE_SIZE: tl.constexpr,  # Tile shapes must be known at compile time
):
    # Each instance will compute the weighted sum of a tile of rows of x.
    # `tl.program_id` gives us a way to check which thread block we're running in
    row_tile_idx = tl.program_id(0)

    # Block pointers give us a way to select from an ND region of memory
    #  and move our selection around.
    # The block pointer must know:
    # - The pointer to the first element of the tensor
    # - The overall shape of the tensor to handle out-of-bounds access
    # - The strides of each dimension to use the memory layout properly
    # - The ND coordinates of the starting block, i.e., "offsets"
    # - The block shape to load/store at a time
    # - The order of the dimensions in memory from major to minor
    #    axes (= np.argsort(strides)) for optimizations, needed for
    #    TMA support on >=Hopper
```

中文译文 / 对应原文第 19 页

右侧接续加权求和前向内核，并开始封装 PyTorch 自动微分函数。

代码注释译文（按出现顺序）：

• 初始化输出缓冲区：大小为 ROWS_TILE_SIZE，类型为 tl.float32。

• 遍历 D 维上的块，加载当前块指针。

• ROWS_TILE_SIZE 可能不能整除 NUM_ROWS，D_TILE_SIZE 也可能不能整除 D，因此两个维度都需边界检查。

• row 形状为 (ROWS_TILE_SIZE, D_TILE_SIZE)，weight 形状为 (D_TILE_SIZE,)，越界区域填零。

• 计算每行加权和，累加到 output。

• 移动指针到下一块。advance 参数为（行、列）坐标增量，输入与权重指针均沿最后一维移动 D_TILE_SIZE。

• 写入输出块指针，每行一个标量。因 ROWS_TILE_SIZE 可能不能整除 NUM_ROWS，存储也需边界检查。

现在把内核封装到 PyTorch Autograd 函数中，与 PyTorch 互操作：输入为 Tensor，输出也为 Tensor，之后还支持反向传播的自动微分引擎。

WeightedSumFunc.forward 中的注释含义：

• 缓存 x 和 weight，供反向传播使用。反向时只收到损失对输出的梯度，需用它们计算对 x 和 weight 的梯度。

• 根据输入最后一维确定 D，其余维度确定输出形状。

• 记录输入原形状，将输入重排为二维。

• 用 ctx.save_for_backward 保存 x 和 weight。

代码在下一页接续。


```
    x_block_ptr = tl.make_block_ptr(
        x_ptr,
        shape=(NUM_ROWS, D,),
        strides=(x_stride_row, x_stride_dim),
        offsets=(row_tile_idx * ROWS_TILE_SIZE, 0),
        block_shape=(ROWS_TILE_SIZE, D_TILE_SIZE),
        order=(1, 0),
    )

    weight_block_ptr = tl.make_block_ptr(
        weight_ptr,
        shape=(D,),
        strides=(weight_stride_dim,),
        offsets=(0,),
        block_shape=(D_TILE_SIZE,),
        order=(0,),
    )

    output_block_ptr = tl.make_block_ptr(
        output_ptr,
        shape=(NUM_ROWS,),
        strides=(output_stride_row,),
        offsets=(row_tile_idx * ROWS_TILE_SIZE,),
        block_shape=(ROWS_TILE_SIZE,),
        order=(0,),
    )

    # Initialize a buffer to write to
    output = tl.zeros((ROWS_TILE_SIZE,), dtype=tl.float32)

    for i in range(tl.cdiv(D, D_TILE_SIZE)):
        # Load the current block pointer
        # Since ROWS_TILE_SIZE might not divide NUM_ROWS, and D_TILE_SIZE might not divide D,
        # we need boundary checks for both dimensions
        row = tl.load(x_block_ptr, boundary_check=(0, 1), padding_option="zero")  # (ROWS_TILE_SIZE, D_TILE_SIZE)
        weight = tl.load(weight_block_ptr, boundary_check=(0,), padding_option="zero")  # (D_TILE_SIZE,)

        # Compute the weighted sum of the row.
        output += tl.sum(row * weight[None, :], axis=1)

        # Move the pointers to the next tile.
        # These are (rows, columns) coordinate deltas
        x_block_ptr = x_block_ptr.advance((0, D_TILE_SIZE))  # Move by D_TILE_SIZE in the last dimension
        weight_block_ptr = weight_block_ptr.advance((D_TILE_SIZE,))  # Move by D_TILE_SIZE

    # Write output to the output block pointer (a single scalar per row).
    # Since ROWS_TILE_SIZE might not divide NUM_ROWS, we need boundary checks
    tl.store(output_block_ptr, output, boundary_check=(0,))
```

Let's now wrap this kernel in a PyTorch Autograd function that will interoperate with PyTorch (i.e., take `Tensors` as inputs, output a `Tensor`, and later also work with the autograd engine during the backward pass):

```
class WeightedSumFunc(torch.autograd.Function):
    @staticmethod
    def forward(ctx, x, weight):
        # Cache x and weight to be used in the backward pass, when we
        # only receive the gradient wrt. the output tensor, and
        # need to compute the gradients wrt. x and weight.
        D, output_dims = x.shape[-1], x.shape[:-1]

        # Reshape input tensor to 2D
        input_shape = x.shape
        x = rearrange(x, "... d -> (...) d")

        ctx.save_for_backward(x, weight)
```

右侧顶部接续 WeightedSumFunc.forward。

断言检查维度一致、输入均为 CUDA 张量，以及 x 连续，因为指针运算假设 x 连续。D_TILE_SIZE 为 triton.next_power_of_2(D) // 16，约为沿嵌入维度循环 16 次；ROWS_TILE_SIZE 为 16，注释说明每次处理 16 个批次元素。

初始化空结果张量时注意：torch.empty 的元素不一定为零。根据 n_rows，在一维网格中启动相应数量的内核实例，最后恢复输出形状。

调用 weighted_sum_fwd[(triton.cdiv(n_rows, ctx.ROWS_TILE_SIZE),)] 时，元组 (triton.cdiv(n_rows, ctx.ROWS_TILE_SIZE),) 定义线程块的“启动网格”。内核中用 tl.program_id(0) 访问线程块索引。

反向传播

因为定义了自有内核，也需要编写自己的反向函数。

前向传播接收输入并计算输出；反向传播接收目标函数对输出的梯度，需计算对各输入的梯度。本例输入为矩阵 x∈R^(n×h) 和权重向量 w∈R^h。简记运算为 f(x,w)，值域为 R^n。给定损失 L 对层输出的梯度 ∇f(x,w)L，应用多元链式法则得到对 x、w 的梯度，见公式（2）、（3）。

这些式子给出简单的反向计算方式。对 x，应用公式（2），取 w 与输出梯度的外积；对 w，需要将传入的梯度与对应行相乘。

反向内核先定义所有块指针，再计算 ∇xL。右侧底部给出 weighted_sum_backward 的函数开头，参数包括输入、传入梯度、输出梯度的指针及各步长。下页接续。


```
        assert len(weight.shape) == 1 and weight.shape[0] == D, "Dimension mismatch"
        assert x.is_cuda and weight.is_cuda, "Expected CUDA tensors"
        assert x.is_contiguous(), "Our pointer arithmetic will assume contiguous x"

        ctx.D_TILE_SIZE = triton.next_power_of_2(D) // 16  # Roughly 16 loops through the embedding dimension
        ctx.ROWS_TILE_SIZE = 16  # Each thread processes 16 batch elements at a time
        ctx.input_shape = input_shape

        # Need to initialize empty result tensor. Note that these elements are not necessarily 0!
        y = torch.empty(output_dims, device=x.device)

        # Launch our kernel with n instances in our 1D grid.
        n_rows = y.numel()
        weighted_sum_fwd[(triton.cdiv(n_rows, ctx.ROWS_TILE_SIZE),)](
            x, weight,
            y,
            x.stride(0), x.stride(1),
            weight.stride(0),
            y.stride(0),
            NUM_ROWS=n_rows, D=D,
            ROWS_TILE_SIZE=ctx.ROWS_TILE_SIZE, D_TILE_SIZE=ctx.D_TILE_SIZE,
        )

        return y.view(input_shape[:-1])
```

Notice that when we invoke the Triton kernel with `weighted_sum_fwd[(triton.cdiv(n_rows, ctx.ROWS_TILE_SIZE),)]`, we define a so-called “launch grid” of thread blocks by passing the tuple `(triton.cdiv(n_rows, ctx.ROWS_TILE_SIZE),)`. Then, we can access the thread block index with `tl.program_id(0)` in our kernel.

Backward pass

Since we are defining our own kernel, we will also need to write our own backward function.

In the forward pass, we were given the inputs to our layer, and needed to compute its outputs. In the backward pass, recall that we will be given the gradients of the objective with respect to our outputs, and need to compute the gradient with respect to each of our inputs. In our case, our operation has as inputs a matrix $x : \mathbb{R}^{n \times h}$ and a weight vector $w : \mathbb{R}^h$. For brevity, let's call our operation $f(x, w)$, whose range is $\mathbb{R}^n$. Then, assuming we are given $\nabla_{f(x,w)}\mathcal{L}$, the gradient of the loss $\mathcal{L}$ with respect to the output of our layer, we can apply the multivariate chain rule to obtain the following expressions for the gradients with respect to x and w:

$$\left(\nabla_x \mathcal{L}\right)_{ij} = \sum_{k=1}^{n} \frac{\partial f(x,w)_k}{\partial x_{ij}} \left(\nabla_{f(x,w)}\mathcal{L}\right)_k = w_j \cdot \left(\nabla_{f(x,w)}\mathcal{L}\right)_i \tag{2}$$

$$\left(\nabla_w \mathcal{L}\right)_j = \sum_{i=1}^{n} \frac{\partial f(x,w)_i}{\partial w_j} \left(\nabla_{f(x,w)}\mathcal{L}\right)_i = \sum_{i=1}^{n} x_{ij} \cdot \left(\nabla_{f(x,w)}\mathcal{L}\right)_i \tag{3}$$

This gives a simple formula for computing the backward pass. To obtain the backward step with respect to x, we apply Equation 2 and take the outer product of w and $\nabla_{f(x,w)}$. To compute the backward step with respect to w (i.e. $\left(\nabla_w \mathcal{L}\right)_j$), we must multiply our input gradient by the corresponding output row.

Our kernel for the backward pass will start by defining all the block pointers and then computing $\nabla_x \mathcal{L}$:

```
@triton.jit
def weighted_sum_backward(
    x_ptr, weight_ptr,  # Input
    grad_output_ptr,  # Grad input
    grad_x_ptr, partial_grad_weight_ptr,  # Grad outputs
    stride_xr, stride_xd,
```

中文译文 / 对应原文第 21 页

本页右侧完整保留加权求和反向内核。
代码注释译文（按出现顺序）：
• 获取行块索引 row_tile_idx 和行块总数 n_row_tiles。
• 输入包括输出梯度 grad_output、原输入 x 及权重 weight 的块指针。
• 为 grad_x 建立形状 (NUM_ROWS, D) 的块指针。
• 部分权重梯度指针的形状为 (n_row_tiles, D)，当前实例写入对应行块结果，块形状为 (1, D_TILE_SIZE)。
• 遍历 D 维上的块，加载形状 (ROWS_TILE_SIZE,) 的输出梯度。
• 对 grad_x 计算外积：输出梯度列向量乘权重行向量，写入对应块，两个维度均检查边界。
• 为 grad_weight 尽可能在当前行块内归约：加载输入行块，与输出梯度相乘后沿行维求和并保留该维。
• 写入部分权重梯度时，只需检查维度 1 的边界；维度 0 不会越界。
• 将 x、weight、partial_grad_weight 和 grad_x 的指针沿 D 推进到下一块。
全部变量和运算保留在右侧代码中。


```
    stride_wd,
    stride_gr,
    stride_gxr, stride_gxd,
    stride_gwb, stride_gwd,
    NUM_ROWS, D,
    ROWS_TILE_SIZE: tl.constexpr, D_TILE_SIZE: tl.constexpr,
):
    row_tile_idx = tl.program_id(0)
    n_row_tiles = tl.num_programs(0)

    # Inputs
    grad_output_block_ptr = tl.make_block_ptr(
        grad_output_ptr,
        shape=(NUM_ROWS,), strides=(stride_gr,),
        offsets=(row_tile_idx * ROWS_TILE_SIZE,),
        block_shape=(ROWS_TILE_SIZE,),
        order=(0,),
    )

    x_block_ptr = tl.make_block_ptr(
        x_ptr,
        shape=(NUM_ROWS, D,), strides=(stride_xr, stride_xd),
        offsets=(row_tile_idx * ROWS_TILE_SIZE, 0),
        block_shape=(ROWS_TILE_SIZE, D_TILE_SIZE),
        order=(1, 0),
    )

    weight_block_ptr = tl.make_block_ptr(
        weight_ptr,
        shape=(D,), strides=(stride_wd,),
        offsets=(0,), block_shape=(D_TILE_SIZE,),
        order=(0,),
    )

    grad_x_block_ptr = tl.make_block_ptr(
        grad_x_ptr,
        shape=(NUM_ROWS, D,), strides=(stride_gxr, stride_gxd),
        offsets=(row_tile_idx * ROWS_TILE_SIZE, 0),
        block_shape=(ROWS_TILE_SIZE, D_TILE_SIZE),
        order=(1, 0),
    )

    partial_grad_weight_block_ptr = tl.make_block_ptr(
        partial_grad_weight_ptr,
        shape=(n_row_tiles, D,), strides=(stride_gwb, stride_gwd),
        offsets=(row_tile_idx, 0),
        block_shape=(1, D_TILE_SIZE),
        order=(1, 0),
    )

    for i in range(tl.cdiv(D, D_TILE_SIZE)):
        grad_output = tl.load(grad_output_block_ptr, boundary_check=(0,), padding_option="zero")  # (ROWS_TILE_SIZE,)

        # Outer product for grad_x
        weight = tl.load(weight_block_ptr, boundary_check=(0,), padding_option="zero")  # (D_TILE_SIZE,)
        grad_x_row = grad_output[:, None] * weight[None, :]
        tl.store(grad_x_block_ptr, grad_x_row, boundary_check=(0, 1))

        # Reduce as many rows as possible for the grad_weight result
        row = tl.load(x_block_ptr, boundary_check=(0, 1), padding_option="zero")  # (ROWS_TILE_SIZE, D_TILE_SIZE)
        grad_weight_row = tl.sum(row * grad_output[:, None], axis=0, keep_dims=True)
        tl.store(partial_grad_weight_block_ptr, grad_weight_row, boundary_check=(1,))  # Never out of bounds for dim 0

        # Move the pointers to the next tile along D
        x_block_ptr = x_block_ptr.advance((0, D_TILE_SIZE))
        weight_block_ptr = weight_block_ptr.advance((D_TILE_SIZE,))
        partial_grad_weight_block_ptr = partial_grad_weight_block_ptr.advance((0, D_TILE_SIZE))
        grad_x_block_ptr = grad_x_block_ptr.advance((0, D_TILE_SIZE))
```

计算 ∇x 很简单，结果写入输出张量的相应块即可。但 ∇w 稍复杂：每个内核实例负责 x 的一个行块，而需要跨行求和。我们不在反向内核中直接完成全部求和，而是假设 partial_grad_weight_ptr 指向 n_row_tiles×H 的矩阵，其第一维对应只在一个 x 行块内部归约的结果。先在当前行块内求和再写入；内核外用 torch.sum 汇总各行块结果，得到 ∇w。[脚注 1]

autograd.Function 的最后部分因此比较简单，见右侧代码。

代码注释含义：前向已在此前定义；反向取出保存的 x、weight 及块大小，前后向的块大小不必一致。每个线程块先写入部分结果缓冲区，再归约获得最终梯度。分配部分权重梯度及 grad_x 空间，启动反向内核，沿第 0 维求和得到 grad_weight，并返回两个梯度。

最后，通过 f_weightedsum = WeightedSumFunc.apply 获得类似 torch.nn.functional 的函数。对 x、w 两个 PyTorch 张量调用它，将得到右侧示例所示的 CUDA 张量。

注意张量附带的 grad_fn：这说明该张量出现在计算图时，PyTorch 知道反向传播应调用什么。加权求和的 Triton 实现至此完成。

4.2.2 FlashAttention-2 前向传播

依据 FlashAttention-2 [T. Dao，2023]，你将用显著改进的 Triton 实现替换 PyTorch 注意力。FlashAttention-2 用一些技巧分块计算前向传播，从而高效访问显存，并避免在全局显存中生成完整注意力矩阵。

脚注 1：当然，也可以为这一步自行编写内核。

Computing the gradient ∇_x is simple, and we write the result to the appropriate tile of the output tensor. However, computing ∇_w is a bit more challenging. Each kernel instance is responsible for one row tile of x, but we now need to sum *across* rows of x. Instead of doing this sum directly in our backward pass, we will assume that `partial_grad_weight_ptr` contains an `n_row_tiles` $\times H$ matrix, where the first dimension is only reduced within a row tile from x. We reduce within the current row tile before writing to this tensor. Outside of the kernel, we reduce ∇_w using `torch.sum` to sum up the results from each row tile.[1] The final part of the `autograd.Function` is then relatively simple:

```
class WeightedSumFunc(torch.autograd.Function):
    @staticmethod
    def forward(ctx, x, weight):
        # ... (defined earlier)

    @staticmethod
    def backward(ctx, grad_out):
        x, weight = ctx.saved_tensors
        ROWS_TILE_SIZE, D_TILE_SIZE = ctx.ROWS_TILE_SIZE, ctx.D_TILE_SIZE  # These don't have to be the same
        n_rows, D = x.shape

        # Our strategy is for each thread block to first write to a partial buffer,
        # then we reduce over this buffer to get the final gradient.
        partial_grad_weight = torch.empty((triton.cdiv(n_rows, ROWS_TILE_SIZE), D), device=x.device, dtype=x.dtype)
        grad_x = torch.empty_like(x)

        weighted_sum_backward[(triton.cdiv(n_rows, ROWS_TILE_SIZE),)](
            x, weight,
            grad_out,
            grad_x, partial_grad_weight,
            x.stride(0), x.stride(1),
            weight.stride(0),
            grad_out.stride(0),
            grad_x.stride(0), grad_x.stride(1),
            partial_grad_weight.stride(0), partial_grad_weight.stride(1),
            NUM_ROWS=n_rows, D=D,
            ROWS_TILE_SIZE=ROWS_TILE_SIZE, D_TILE_SIZE=D_TILE_SIZE,
        )
        grad_weight = partial_grad_weight.sum(axis=0)
        return grad_x, grad_weight
```

Finally, we can now obtain a function that works much like those implemented in `torch.nn.functional`:

```
f_weightedsum = WeightedSumFunc.apply
```

Now, calling `f_weightedsum` on two PyTorch tensors x and w will give a tensor such as the following:

```
tensor([ 90.8563, -93.6815, -80.8884,  ..., 103.4840, -21.4634, -24.0192],
       device='cuda:0', grad_fn=<WeightedSumFuncBackward>)
```

Note the `grad_fn` attached to the tensor — this shows that PyTorch knows what to call in the backward pass when this tensor appears in the computation graph. This completes our Triton implementation of the weighted sum operation.

4.2.2 FlashAttention-2 Forward Pass

You will replace your PyTorch attention implementation with a significantly improved Triton implementation following FlashAttention-2 [T. Dao, 2023]. FlashAttention-2 employs some tricks to compute the forward pass in tiles, which allows for efficient memory access patterns and avoids the need to materialize the full attention matrix on global memory.

[1] Or, of course, we could write our own kernel for that.

开始本节前，强烈建议至少阅读最初的 FlashAttention 论文 [T. Dao 等，2022]，理解其高效注意力的核心技术：跨块在线计算 softmax。这项技术由 [M. Milakov 等，2018] 提出。也建议阅读 H. He [4]，更直观地了解 GPU 如何实际执行 PyTorch 代码。

理解普通注意力的低效之处

暂不考虑掩码的前向传播为：$S=QK^T/\sqrt{d}$，$P=\text{softmax}(S)$，$O=PV$。完整表达式见公式（4）-（6）。标准反向传播见公式（7）-（11），依次计算 dV、dP、dS、dQ 和 dK。

反向传播依赖前向中的一些巨大激活。例如公式（7）计算 dV 需要 P，形状为 (batch_size, n_heads, seq_len, seq_len)。其大小随序列长度二次增长，解释了长序列注意力测试中的显存问题。

普通注意力的前后向均需在片上 SRAM 与 GPU HBM 之间传输 P 和其他大激活，带来显著 I/O 开销。标准实现中有多次传输，例如反向计算公式（7）和（9）都要从 HBM 读取 P。

FlashAttention 的主要目标是避免在 HBM 中读写注意力矩阵，降低 I/O 和峰值显存成本。使用三项技术：分块、重计算、算子融合。

分块

为避免在 HBM 中读写注意力矩阵，需要在无法访问完整输入的情况下完成 softmax 归约。重新组织计算，将输入分块并多次遍历输入块，逐步完成归约。

重计算

不将形状 (batch_size, n_heads, seq_len, seq_len) 的大中间注意力矩阵存入 HBM，而是保存某些“激活检查点”；反向时重计算部分前向传播，获得计算梯度所需的其他激活。FlashAttention-2 还保存注意力分数的 logsumexp 值 L，简化反向计算。L 的表达式见公式（12）。

Before jumping into this section, we highly recommend reading at least the original FlashAttention paper [T. Dao et al., 2022], which will give you intuition for the core technique that enables efficient attention with FlashAttention: computing the softmax in an online fashion across tiles (a technique proposed in [M. Milakov et al., 2018]). We also recommend checking out H. He [4] for some more intuition on how GPUs actually execute PyTorch code.

Understanding inefficiencies in vanilla attention

Recall that the forward pass for attention (ignoring masking for now) can be written as:

$$\boldsymbol{S} = \boldsymbol{Q}\boldsymbol{K}^\top/\sqrt{d} \tag{4}$$

$$P_{ij} = \text{softmax}_j(\boldsymbol{S})_{ij} \tag{5}$$

$$\boldsymbol{O} = \boldsymbol{P}\boldsymbol{V} \tag{6}$$

The standard backward pass is

$$\boldsymbol{dV} = \boldsymbol{P}^\top \boldsymbol{dO} \tag{7}$$

$$\boldsymbol{dP} = \boldsymbol{dO}\boldsymbol{V}^\top \tag{8}$$

$$\boldsymbol{dS}_i = \text{dsoftmax}(\boldsymbol{dP}_i) = (\text{diag}(\boldsymbol{P}_i) - \boldsymbol{P}_i\boldsymbol{P}_i^\top)\boldsymbol{dP}_i \tag{9}$$

$$\boldsymbol{dQ} = \boldsymbol{dS}\boldsymbol{K}/\sqrt{d} \tag{10}$$

$$\boldsymbol{dK} = \boldsymbol{dS}^\top\boldsymbol{Q}/\sqrt{d} \tag{11}$$

As we can see, the backward pass depends on some very large activations from the forward pass. For example, computing $\boldsymbol{dV}$ in Equation 7 requires $\boldsymbol{P}$, which are the attention scores of shape `(batch_size, n_heads, seq_len, seq_len)`—the size of this activation matrix depends *quadratically* on the sequence length, explaining the memory issues we encountered above when benchmarking attention at large sequence lengths. During both the forward and backward pass of vanilla attention, we pay significant memory IO costs to transfer $\boldsymbol{P}$ and other large activations between on-chip SRAM and GPU HBM. There are *several* such transfers made in standard implementations: for example, a standard backward pass implementation would read $\boldsymbol{P}$ from HBM in the computations of both Equation 7 and Equation 9.

The main goal of FlashAttention is to avoid reading and writing the attention matrix to and from HBM, to reduce IO and peak memory costs. We accomplish this using three techniques: tiling, recomputation, and operator fusion.

Tiling

To avoid reading and writing the attention matrix to and from HBM, we compute the softmax reduction without access to the whole input. Specifically, we restructure the attention computation to split the input into tiles and make several passes over input tiles, thus incrementally performing the softmax reduction.

Recomputation

We avoid storing the large intermediate attention matrices of shape `(batch_size, n_heads, seq_len, seq_len)` in HBM. Instead, we will save certain “activation checkpoints” in HBM and then recompute part of the forward pass during the backward pass, to get the other activations we need for computing gradients. FlashAttention-2 also stores the logsumexp of the attention scores, L, which will be used to simplify the backward pass computation. The expression for L is:

$$L_i = \log\left(\sum_j \exp(\boldsymbol{S}_{ij})\right) \tag{12}$$

最终内核将在线计算 L，但结果应与公式（12）一致。

结合分块和重计算后，显存 I/O 和峰值开销不再依赖 sequence_length²，因此可使用更长序列。

算子融合

在单个内核中执行全部操作，可避免注意力矩阵及其他中间激活的重复 I/O，这称为算子或内核融合。我们会编写单个 Triton 前向内核，完成注意力全部运算，限制 HBM 与 SRAM 之间的传输。重计算也使融合成为可能，因为无需将每个中间激活存到 HBM。

更多直观解释见 FlashAttention 论文 [T. Dao 等，2022；T. Dao，2023]。

带重计算的反向传播

使用 L，可以完成重计算并高效反向传播。开始前，在全局显存中预计算 D=rowsum(O∘dO)，其中∘为逐元素乘法。它等于 rowsum(P∘dP)，因为 P(dP)ᵀ=P(dO Vᵀ)ᵀ=(PV)(dO)ᵀ=O(dO)ᵀ；对任意 A、B，rowsum(A∘B)=diag(ABᵀ)。准确原式见右侧。

借助 L、D 两个向量，反向传播无需 softmax，完整流程见公式（13）-（19）。它不要求前向将 P 保存在 HBM：在公式（13）、（14）中，从 Q、K、L 重计算 P。

FlashAttention 前向传播的细节

了解上述技术后，深入讨论需要实现的 FA2 前向内核。为避免在 HBM 中读写注意力矩阵，希望通过分块独立计算各输出块。这要求可以计算 P 的块，最好 query 和 key 两维都分块。

但对 S 使用 softmax，需要对整行归约以获得分母，因此无法直接分块计算 P。FlashAttention-2 用在线 softmax 解决。

以下下标 i 表示当前 query 块，上标 (j) 表示当前 key 块。query 块大小为 Bq，key 块大小为 Bk，不对隐藏维度 d 分块。

还维护逐行运行量 m_i^(j)、l_i^(j)，均属于 R^Bq。m 是运行最大值，用于稳定地计算 softmax，回顾作业 1 中这一技巧。每加入一个新的 S 行块，即 j 增大时，就更新 m。

In our final kernel we will compute this in an online manner, but the final result should be the same. With tiling and recomputation together, our memory IO and peak usage no longer depend on sequence_length^2 and therefore we may use larger sequence lengths.

Operator fusion

Lastly, we avoid repeated memory IO for the attention matrix and other intermediate activations by performing all our operations in a single kernel—this is referred to as operator or kernel fusion. We will write a single Triton kernel for the forward pass that performs all the operations involved in attention with limited data transfer between HBM and SRAM. Operator fusion is partly enabled by recomputation, since we can avoid the usual memory IO we would pay to store every intermediate activation to HBM.

For more intuition on these techniques, check out the FlashAttention papers [T. Dao et al., 2022; T. Dao, 2023].

Backward pass with recomputation

Using L, we can do the appropriate recomputation and compute the backward pass efficiently. Before we start the backward pass, we precompute the value $D = \text{rowsum}(\boldsymbol{O} \circ \boldsymbol{dO})$ in global memory (where $\circ$ is element-wise multiplication), which is equal to $\text{rowsum}(\boldsymbol{P} \circ \boldsymbol{dP})$ since $\boldsymbol{PdP}^\top = \boldsymbol{P}(\boldsymbol{dOV}^\top)^\top = (\boldsymbol{PV})\boldsymbol{dO}^\top = \boldsymbol{OdO}^\top$ (and $\text{rowsum}(\boldsymbol{A} \circ \boldsymbol{B}) = \text{diag}(\boldsymbol{AB}^\top)$ for any matrices $\boldsymbol{A}$ and $\boldsymbol{B}$). With the L and D vectors, the backward pass can be computed without softmax. The full calculation for the backward pass is now:

$$\boldsymbol{S} = \boldsymbol{QK}^\top / \sqrt{d} \tag{13}$$

$$P_{ij} = \exp(S_{ij} - L_i) \tag{14}$$

$$\boldsymbol{dV} = \boldsymbol{P}^\top \boldsymbol{dO} \tag{15}$$

$$\boldsymbol{dP} = \boldsymbol{dOV}^\top \tag{16}$$

$$dS_{ij} = P_{ij}(dP_{ij} - D_i) \tag{17}$$

$$\boldsymbol{dQ} = \boldsymbol{dSK} / \sqrt{d} \tag{18}$$

$$\boldsymbol{dK} = \boldsymbol{dS}^\top \boldsymbol{Q} / \sqrt{d} \tag{19}$$

We can see that the sequence of operations does not require us to have stored the attention scores $\boldsymbol{P}$ in HBM during the forward pass—we recompute them from the activations $\boldsymbol{Q}$, $\boldsymbol{K}$, and L in Equation 13 and Equation 14.

Details of the FlashAttention forward pass

Now that we have a high-level idea of the techniques used in FlashAttention-2, we will dive into the details of the FA2 forward pass kernel that you will implement. In order to avoid reading and writing the attention matrix to and from HBM, we wish to use tiling, i.e., computing each tile of the output independently of the others. This requires us to be able to compute tiles of P, ideally tiled in both dimensions (for queries and for keys).

However, when we apply softmax to S, we require entire rows of S to be reduced to compute the softmax denominator, meaning we cannot compute P in tiles directly. FlashAttention-2 solves this problem using *online softmax*. In the following text, we will use subscript index i to denote the current query tile, and superscript index (j) to denote the current key tile. The tiles along the query dimension will be of size B_q, and those along the key dimension will be of size B_k. We will not tile along the hidden dimension d.

We also keep some row-wise running values, $m_i^{(j)} \in \mathbb{R}^{B_q}$ and $l_i^{(j)} \in \mathbb{R}^{B_q}$. The row-wise $m_i^{(j)}$ value is a running maximum, which is tracked so we can compute softmax in a numerically stable manner (recall this trick from our softmax implementation in Assignment 1). We will update $m_i^{(j)}$ with each new row-

中文译文 / 对应原文第 25 页

利用运行最大值，可计算未归一化 softmax 分子 P。l 是分母的运行代理值，利用未归一化值更新。准确表达式见右侧。最终写输出时，用处理完全部 key 块后的 l 值归一化。算法 1 展示 GPU 前向传播。

算法 1：FlashAttention-2 前向传播

数学表达式、维度和行号完整保留在右侧；以下逐行翻译操作说明。

1. 输入要求：Q、K、V 及块大小 Bq、Bk。
2. 将 Q 分成 Tq 个大小 Bq×d 的块。
3. 将 K、V 分成 Tk 个大小 Bk×d 的块。
4. 对 i=1,...,Tq 循环。
5. 从全局显存加载 Qi。
6. 初始化 O 为零，l 为零，m 为负无穷。
7. 对 j=1,...,Tk 循环。
8. 从全局显存加载当前 K、V 块。
9. 计算 softmax 前的注意力分数块 S。
10. 计算新的逐行最大值 m。
11. 计算未归一化概率块 P。
12. 计算新的分母代理值 l。
13. 缩放此前的 O，再累加当前 P 与 V 的乘积。
14. 结束内层循环。
15. 使用最终 l 对 O 归一化。
16. 计算最终 logsumexp 值 L。
17. 将 Oi 作为 O 的第 i 个块写入全局显存。
18. 将 Li 作为 L 的第 i 个块写入全局显存。
19. 结束外层循环。
20. 返回输出 O 与 logsumexp 值 L。

实现前先汇总通用 Triton 技巧：

• 用 tl.device_print 打印以调试：
https://triton-lang.org/main/python-api/generated/triton.language.device_print.html
• TRITON_INTERPRET=1 可在 CPU 上运行解释器，但我们发现它有一些问题。
• 定义块指针时，确保偏移正确，且块偏移乘以对应块大小。
• 线程块的启动网格设置方法在下一页继续。

wise tile of S (when j increases). Using the running maximum, we can compute the unnormalized softmax values (numerators) as $\tilde{\boldsymbol{P}}_i^{(j)} = \exp\left(\boldsymbol{S}_{ij} - m_i^{(j)}\right)$. $l_i^{(j)}$ is a running proxy for the softmax denominator, and will be updated using the unnormalized softmax values as $l_i^{(j)} = \exp\left(m_i^{(j-1)} - m_i^{(j)}\right) \cdot l_i^{(j-1)} + \text{rowsum}\left(\tilde{\boldsymbol{P}}_i^{(j)}\right)$. When we finally write the output, we will need to finish normalizing it by using $l_i^{(T_k)}$, which is the final value of $l_i^{(j)}$ after processing all key tiles. Algorithm 1 shows the forward pass on the GPU.

Algorithm 1: FlashAttention-2 forward pass

Require: $\boldsymbol{Q} \in \mathbb{R}^{N_q \times d}$, $\boldsymbol{K}, \boldsymbol{V} \in \mathbb{R}^{N_k \times d}$, tile sizes B_q, B_k
Split $\boldsymbol{Q}$ into $T_q = \left\lceil \frac{N_q}{B_q} \right\rceil$ tiles $\boldsymbol{Q}_1, \ldots, \boldsymbol{Q}_{T_q}$ of size $B_q \times d$
Split $\boldsymbol{K}, \boldsymbol{V}$ into $T_k = \left\lceil \frac{N_k}{B_k} \right\rceil$ tiles $\boldsymbol{K}^{(1)}, \ldots, \boldsymbol{K}^{(T_k)}$ and $\boldsymbol{V}^{(1)}, \ldots, \boldsymbol{V}^{(T_k)}$ of size $B_k \times d$
for $i = 1, \ldots, T_q$ **do**
  Load $\boldsymbol{Q}_i$ from global memory
  Initialize $\boldsymbol{O}_i^{(0)} = \boldsymbol{0} \in \mathbb{R}^{B_q \times d}, l_i^{(0)} = 0 \in \mathbb{R}^{B_q}, m_i^{(0)} = -\infty \in \mathbb{R}^{B_q}$
  for $j = 1, \ldots, T_k$ **do**
    Load $\boldsymbol{K}^{(j)}, \boldsymbol{V}^{(j)}$ from global memory
    Compute tile of pre-softmax attention scores $\boldsymbol{S}_i^{(j)} = \frac{\boldsymbol{Q}_i (\boldsymbol{K}^{(j)})^\top}{\sqrt{d}} \in \mathbb{R}^{B_q \times B_k}$
    Compute $m_i^{(j)} = \max\left(m_i^{(j-1)}, \text{rowmax}\left(\boldsymbol{S}_i^{(j)}\right)\right) \in \mathbb{R}^{B_q}$
    Compute $\tilde{\boldsymbol{P}}_i^{(j)} = \exp\left(\boldsymbol{S}_i^{(j)} - m_i^{(j)}\right) \in \mathbb{R}^{B_q \times B_k}$
    Compute $l_i^{(j)} = \exp\left(m_i^{(j-1)} - m_i^{(j)}\right) l_i^{(j-1)} + \text{rowsum}\left(\tilde{\boldsymbol{P}}_i^{(j)}\right) \in \mathbb{R}^{B_q}$
    Compute $\boldsymbol{O}_i^{(j)} = \text{diag}\left(\exp\left(m_i^{(j-1)} - m_i^{(j)}\right)\right) \boldsymbol{O}_i^{(j-1)} + \tilde{\boldsymbol{P}}_i^{(j)} \boldsymbol{V}^{(j)}$
  end for
  Compute $\boldsymbol{O}_i = \text{diag}\left(l_i^{(T_k)}\right)^{-1} \boldsymbol{O}_i^{(T_k)}$
  Compute $L_i = m_i^{(T_k)} + \log\left(l_i^{(T_k)}\right)$
  Write $\boldsymbol{O}_i$ to global memory as the i-th tile of $\boldsymbol{O}$.
  Write L_i to global memory as the i-th tile of L.
end for
Return the output $\boldsymbol{O}$ and the logsumexp L.

Before we get into implementing the forward pass in Triton, we collect here a few general tips and tricks for writing Triton kernels.

Triton Tips and Tricks

- You can use print statements in Triton with `tl.device_print` to debug: `https://triton-lang.org/main/python-api/generated/triton.language.device_print.html`. There is a setting `TRITON_INTERPRET=1` to run the Triton interpreter on CPU, though we have found it buggy.
- When defining block pointers, make sure they have the correct offsets, and that block offsets are multiplied by the appropriate tile sizes.
- The launch grid of thread blocks is set with

• 在 torch.autograd.Function 子类的方法中，用 kernel_fn[(launch_grid_d1, launch_grid_d2, ...)](...arguments...) 设置启动网格，加权求和例子已展示。

• 使用 tl.dot 做矩阵乘法。

• 用 *_block_ptr = *_block_ptr.advance(...) 推进块指针。

题目（flash_forward）：FlashAttention-2 前向传播（15 分）

(a) 编写纯 PyTorch（不使用 Triton）的 autograd.Function，实现 FlashAttention-2 前向传播。它比常规实现慢很多，但可帮助调试 Triton 内核。

接收 Q、K、V 和 is_causal 标志，生成 O 和 logsumexp 值 L。本小题可忽略 is_causal。forward 应为反向保存 L、Q、K、V、O 并返回 O。注意 forward 的第一个参数总是上下文。所有 autograd.Function 类需实现 backward，目前可只抛出 NotImplementedError。需要对照时，可在 PyTorch 中实现公式（4）-（6）及（12）并比较输出。

接口为 forward(ctx, Q, K, V, is_causal=False)。自行选择块大小，至少 16×16。测试维度总是至少 16 的 2 的幂，不必担心越界。

提交内容：实现 FlashAttention-2 前向的 torch.autograd.Function 子类。实现 [adapters.get_flashattention_autograd_function_pytorch]，再运行 uv run pytest -k test_flash_forward_pass_pytorch，确保通过。

(b) 按算法 1 编写前向 Triton 内核。再写另一个 torch.autograd.Function 子类，在前向调用这个融合内核，而不是用 PyTorch 计算。提示：

• 调试时，将每个 Triton 运算结果与 (a) 的分块 PyTorch 实现比较。

• 启动网格设为 (Tq, batch_size)：每个实例只加载一个批次索引，也只读写 Q、O、L 的一个 query 块。

• 内核只有一个循环，遍历 key 块 1≤j≤Tk。

• 循环末尾推进块指针。

• 使用右侧起始、下页接续的函数声明；根据给出的块指针推断其他指针的设置。


```
kernel_fn[(launch_grid_d1, launch_grid_d2, ...)](...arguments...)
```

in the methods of the `torch.autograd.Function` subclass, as we saw in the weighted sum example.

- Perform matrix multiplications with `tl.dot`.
- To advance a block pointer, use `*_block_ptr = *_block_ptr.advance(...)`

Problem (`flash_forward`): FlashAttention-2 Forward Pass (15 points)

(a) Write a pure PyTorch (no Triton) `autograd.Function` that implements the FlashAttention-2 forward pass. This will be a lot slower than the regular PyTorch implementation, but will help you debug your Triton kernel.

Your implementation should take input $\boldsymbol{Q}$, $\boldsymbol{K}$, and $\boldsymbol{V}$ as well as a flag `is_causal` and produce the output $\boldsymbol{O}$ and the logsumexp value L. You can ignore the `is_causal` flag for this task. The `autograd.Function` forward should then save L, Q, K, V, O for the backward pass and return O. Remember that the implementation of the `forward` method of `autograd.Function` always takes the context as its first parameter. Any `autograd.Function` class needs to implement a `backward` method, but for now you can make it just raise `NotImplementedError`. If you need something to compare against, you can implement Equation 4 to Equation 6 and Equation 12 in PyTorch and compare your outputs.

The interface is then `def forward(ctx, Q, K, V, is_causal=False)`. Determine your own tile sizes, but make sure they are at least of size 16×16. We will always test your code with dimensions that are powers of 2 and at least 16, so you don't need to worry about out-of-bounds accesses.

Deliverable: A `torch.autograd.Function` subclass that implements FlashAttention-2 in the forward pass. To test your code, implement [`adapters.get_flashattention_autograd_function_pytorch`]. Then, run the test with `uv run pytest -k test_flash_forward_pass_pytorch` and make sure your implementation passes it.

(b) Write a Triton kernel for the forward pass of FlashAttention-2 following Algorithm 1. Then, write another subclass of `torch.autograd.Function` that calls this (fused) kernel in the forward pass, instead of computing the result in PyTorch. A few problem-specific tips:

- To debug, we suggest comparing the results of each Triton operation you perform with the tiled PyTorch implementation you wrote in part (a).
- Your launch grid should be set as $(T_q, \text{batch_size})$, meaning each Triton program instance will load only elements from a single batch index, and only read/write to a single query tile of $\boldsymbol{Q}$, $\boldsymbol{O}$, and L.
- The kernel should only have a single loop, which will iterate key tiles $1 \le j \le T_k$.
- Advance block pointers at the end of the loop.
- Use the function declaration below (using the block pointer we give you, you should be able to infer the setup of the rest of the pointers):

```
@triton.jit
def flash_fwd_kernel(
```

中文译文 / 对应原文第 27 页

右侧顶部接续 flash_fwd_kernel 声明与 Q_block_ptr 示例。

注释含义：获取 query_tile_index、batch_index；每个指针加上批次索引乘该张量批次步长的偏移。Q 块指针形状为 (N_QUERIES, D)，偏移为 (query_tile_index * Q_TILE_SIZE, 0)，块形状 (Q_TILE_SIZE, D)，order=(1, 0)。

scale 为 1/√d；Q_TILE_SIZE、K_TILE_SIZE 分别为 Bq、Bk，之后可调优。

避免精度问题的补充要求：

• 片上缓冲区 Oi、l、m 使用 tl.float32。向输出缓冲区累加时用 acc 参数，如 acc = tl.dot(..., acc=acc)。

• P 与 V 相乘前，先将 P 转为 V 的类型；Oi 写入全局显存前，转换为适当类型。用 tensor.to 转换，用 tensor.dtype 获取张量类型，用 *_block_ptr.type.element_ty 获取块指针/指针的元素类型。

提交内容：使用 Triton 内核实现 FlashAttention-2 前向的 torch.autograd.Function 子类。实现 [adapters.get_flash_autograd_function_triton]，运行 uv run pytest -k test_flash_forward_pass_triton，确保通过。

(c) 在 autograd.Function 的最后一个参数位置添加因果掩码标志。它为布尔值，True 时启用索引比较进行因果掩码。Triton 内核添加对应参数 is_causal: tl.constexpr，此类型标注必需。

构造 query/key 的适当索引向量，比较得到 Bq×Bk 的掩码。对被屏蔽元素，在 S 对应位置加 -1e6。务必用 ctx.is_causal = is_causal 保存标志供反向使用。

提交内容：为 Triton 前向实现添加因果掩码标志。参数可选且默认 False，使原测试仍通过。


```
    Q_ptr, K_ptr, V_ptr,
    O_ptr, L_ptr,
    stride_qb, stride_qq, stride_qd,
    stride_kb, stride_kk, stride_kd,
    stride_vb, stride_vk, stride_vd,
    stride_ob, stride_oq, stride_od,
    stride_lb, stride_lq,
    N_QUERIES, N_KEYS,
    scale,
    D: tl.constexpr,
    Q_TILE_SIZE: tl.constexpr,
    K_TILE_SIZE: tl.constexpr,
):
    # Program indices
    query_tile_index = tl.program_id(0)
    batch_index = tl.program_id(1)

    # Offset each pointer with the corresponding batch index
    # multiplied with the batch stride for each tensor
    Q_block_ptr = tl.make_block_ptr(
        Q_ptr + batch_index * stride_qb,
        shape=(N_QUERIES, D),
        strides=(stride_qq, stride_qd),
        offsets=(query_tile_index * Q_TILE_SIZE, 0),
        block_shape=(Q_TILE_SIZE, D),
        order=(1, 0),
    )

    ...
```

where `scale` is $\frac{1}{\sqrt{d}}$ and `Q_TILE_SIZE` and `K_TILE_SIZE` are B_q and B_k respectively. You can tune these later.

These additional guidelines may help you avoid precision issues:

- The on chip buffers $(\boldsymbol{O}_i, l, m)$ should have `dtype tl.float32`. If you're accumulating into an output buffer, use the `acc` argument (`acc = tl.dot(..., acc=acc)`).
- Cast $\tilde{\boldsymbol{P}}_i^{(j)}$ to the `dtype` of $\boldsymbol{V}^{(j)}$ before multiplying them, and cast $\boldsymbol{O}_i$ to the appropriate `dtype` before writing it to global memory. Casting is done with `tensor.to`. You can get the `dtype` of a tensor with `tensor.dtype`, and the dtype of a block pointer/pointer with `*_block_ptr.type.element_ty`.

Deliverable: A `torch.autograd.Function` subclass that implements FlashAttention-2 in the forward pass using your Triton kernel. Implement `[adapters.get_flash_autograd_function_triton]` . Then, run the test with `uv run pytest -k test_flash_forward_pass_triton` and make sure your implementation passes it.

(c) Add a flag as the last argument to your `autograd.Function` implementation for causal masking. This should be a boolean flag that, when set to `True`, enables an index comparison for causal masking. Your Triton kernel should have a corresponding additional parameter `is_causal: tl.constexpr` (this is a required type annotation). In Triton, construct appropriate index vectors for queries and keys, and compare them to form a square mask of size $B_q \times B_k$. For elements that are masked out, add the constant value of `-1e6` to the corresponding elements of the attention score matrix $\boldsymbol{S}_i^{(j)}$. Make sure to save the mask flag for backward using `ctx.is_causal = is_causal`.

Deliverable: An additional flag for your `torch.autograd.Function` subclass that implements the FlashAttention-2 forward pass with causal masking using your Triton kernel. Make sure that the flag is optional and defaults to `False` so the previous tests still pass.

实现带重计算的反向传播

不同于公式（7）-（11）的标准反向，公式（13）-（19）通过重计算避免 softmax。因此简单内核即可执行反向，无需在线技巧。本部分可对普通 PyTorch 函数使用 torch.compile，不用 Triton。

题目（flash_backward）：FlashAttention-2 反向传播（5 分）

使用 PyTorch（非 Triton）与 torch.compile 实现 FlashAttention-2 autograd.Function 的反向传播。输入 Q、K、V、O、dO、L，返回 dQ、dK、dV。记得计算并使用 D 向量，可按照公式（13）-（19）。

提交内容：运行 uv run pytest -k test_flash_backward 测试。

下面比较部分使用 Triton 的 FlashAttention-2 与常规 PyTorch 注意力的性能。

题目（flash_benchmarking）：FlashAttention-2 基准测试（5 分）

(a) 用 triton.testing.do_bench 编写脚本，比较部分使用 Triton 的 FlashAttention-2 前向与反向，以及常规 PyTorch 实现（不使用 FlashAttention）。

报告表格，分别列出 Triton 和 PyTorch 的前向、反向、端到端前向加反向延迟。在计时前生成随机输入，使用单个 B200，始终使用批大小 1 与因果掩码。

遍历以下笛卡尔积：128 到 65536 的各个 2 的幂序列长度；16 到 128 的各个 2 的幂嵌入维度；torch.bfloat16 与 torch.float32。可能需按输入大小调整块大小。

提交内容：采用上述设置，对比实现并报告前向、反向及端到端延迟的结果表。

4.2.3 可选：Triton 反向传播

若希望进一步练习 Triton 或获得快速排行榜提交，下页提供可用 Triton 实现的分块 FlashAttention-2 反向算法。算法 2 展示应在 Triton 中实现的流程。关键是计算 P 两次，一次用于 dQ，另一次用于 dK、dV；从而跳过线程块间同步，避免缓慢的原子操作。

Implementing the backward pass with recomputation

Notice that unlike the standard backward pass in Equation 7 to Equation 11, we can use recomputation to avoid the softmax operation in the backward pass shown in Equation 13 to Equation 19. This means that we can compute the backward pass using a trivial kernel, and no online tricks are required. Thus, for this part, you can implement backward by calling `torch.compile` on a regular PyTorch function (not Triton).

Problem (`flash_backward`): FlashAttention-2 Backward Pass (5 points)

Implement the backward pass for your FlashAttention-2 `autograd.Function` using PyTorch (not Triton) and `torch.compile`. Your implementation should take the $\boldsymbol{Q}$, $\boldsymbol{K}$, $\boldsymbol{V}$, $\boldsymbol{O}$, $\boldsymbol{dO}$, and L tensors as inputs, and return $\boldsymbol{dQ}$, $\boldsymbol{dK}$ and $\boldsymbol{dV}$. Remember to compute and use the D vector. You may follow along the computations of Equation 13 to Equation 19.

Deliverable: To test your implementation, run `uv run pytest -k test_flash_backward`.

Let's now compare the performance of your (partially) Triton implementation of FlashAttention-2 with your PyTorch implementation of regular Attention.

Problem (`flash_benchmarking`): FlashAttention-2 Benchmarking (5 points)

(a) Write a benchmarking script using `triton.testing.do_bench` that compares the performance of your (partially) Triton implementation of FlashAttention-2 forward and backward passes with a regular PyTorch implementation (i.e., not using FlashAttention).

Specifically, you will report a table that includes latencies for forward, backward, and the end-to-end forward-backward pass, for both your Triton and PyTorch implementations. Randomly generate any necessary inputs before you start benchmarking, and run the benchmark on a single B200. Always use batch size 1 and causal masking. Sweep over the cartesian product of sequence lengths of various powers of 2 from 128 up to 65536, embedding dimension sizes of various powers of 2 from 16 up to size 128, and precisions of `torch.bfloat16` and `torch.float32`. You will likely need to adjust tile sizes depending on the input sizes.

Deliverable: A table of results comparing your implementation of FlashAttention-2 with the PyTorch implementation, using the settings above and reporting forward, backward, and end-to-end latencies.

4.2.3 OPTIONAL: Triton backward pass

If you're interested in getting more practice with Triton and/or having a fast leaderboard submission, we provide the tiled FlashAttention-2 backward pass below which you can implement in Triton. Algorithm 2 shows the FlashAttention-2 backward pass as it should be implemented in Triton. A key trick here is to compute $\boldsymbol{P}$ twice, once for $\boldsymbol{dQ}$ and again for $\boldsymbol{dK}$ and $\boldsymbol{dV}$. This lets us skip synchronization across thread blocks, meaning we can avoid slow atomics.

中文译文 / 对应原文第 29 页

算法 2：分块 FlashAttention-2 反向传播

数学表达式、维度及行号完整保留在右侧；以下逐行翻译操作说明。

1. 输入要求：Q、O、dO、K、V、L 及块大小 Bq、Bk。
2. 计算 D=rowsum(O∘dO)。
3. Q、O、dO 各拆为 Tq 个 Bq×d 的块。
4. K、V 各拆为 Tk 个 Bk×d 的块。
5. L、D 各拆为 Tq 个大小 Bq 的块。
6. 对 j=1,...,Tk 循环。
7. 从全局显存加载当前 K、V 块。
8. 初始化当前块 dK、dV 为零。
9. 对 i=1,...,Tq 循环。
10. 从全局显存加载 Qi、dOi。
11. 计算注意力分数块 S。
12. 使用 S、Li 计算概率块 P。
13. 更新 dV 累加值。
14. 计算 dP。
15. 使用 P、dP、Di 计算 dS。
16. 更新 dK 累加值。
17. 结束内层循环。
18. 将最终 dK、dV 写入全局显存，作为第 j 个块。
19. 结束外层循环。
20. 对 i=1,...,Tq 循环。
21. 从全局显存加载 Qi、dOi。
22. 初始化当前 query 块 dQ 为零。
23. 对 j=1,...,Tk 循环。
24. 从全局显存加载当前 K、V 块。
25. 计算注意力分数块 S。
26. 使用 S、Li 计算概率块 P。
27. 计算 dP。
28. 使用 P、dP、Di 计算 dS。
29. 更新 dQ 累加值。
30. 结束内层循环。
31. 将最终 dQi 写入全局显存，作为第 i 个块。
32. 结束外层循环。
33. 返回 dQ、dK、dV。

5 分布式数据并行训练

下面探索多 GPU 训练语言模型的方法，重点是数据并行。先介绍 PyTorch 分布式通信，再研究朴素 DDP 实现，最后实现并测试通信效率的改进。

Algorithm 2: Tiled FlashAttention-2 backward pass

Require: $\boldsymbol{Q}, \boldsymbol{O}, \boldsymbol{dO} \in \mathbb{R}^{N_q \times d}$, $\boldsymbol{K}, \boldsymbol{V} \in \mathbb{R}^{N_k \times d}$, $L \in \mathbb{R}^{N_q}$, tile sizes B_q, B_k

Compute $D = \mathrm{rowsum}(\boldsymbol{O} \circ \boldsymbol{dO}) \in \mathbb{R}^{N_q}$

Split $\boldsymbol{Q}, \boldsymbol{O}, \boldsymbol{dO}$ into $T_q = \left\lceil \frac{N_q}{B_q} \right\rceil$ tiles $\boldsymbol{Q}_1, ..., \boldsymbol{Q}_{T_q}$, $\boldsymbol{O}_1, ..., \boldsymbol{O}_{T_q}$, $\boldsymbol{dO}_1, ..., \boldsymbol{dO}_{T_q}$, each of size $B_q \times d$

Split $\boldsymbol{K}, \boldsymbol{V}$ into $T_k = \left\lceil \frac{N_k}{B_k} \right\rceil$ tiles $\boldsymbol{K}^{(1)}, ..., \boldsymbol{K}^{(T_k)}$ and $\boldsymbol{V}^{(1)}, ..., \boldsymbol{V}^{(T_k)}$, each of size $B_k \times d$

Split L, D into T_q tiles $L_1, ..., L_{T_q}$ and $D_1, ..., D_{T_q}$, each of size B_q

for $j = 1, ..., T_k$ **do**

  Load $\boldsymbol{K}^{(j)}, \boldsymbol{V}^{(j)}$ from global memory

  Initialize $\boldsymbol{dK}_0^{(j)} = \boldsymbol{dV}_0^{(j)} = \boldsymbol{0} \in \mathbb{R}^{B_k \times d}$

  for $i = 1, ..., T_q$ **do**

    Load $\boldsymbol{Q}_i, \boldsymbol{dO}_i$ from global memory

    Compute tile of attention scores $\boldsymbol{S}_i^{(j)} = \frac{\boldsymbol{Q}_i (\boldsymbol{K}^{(j)})^\top}{\sqrt{d}} \in \mathbb{R}^{B_q \times B_k}$

    Compute attention probabilities $\boldsymbol{P}_i^{(j)} = \exp\left(\boldsymbol{S}_i^{(j)} - L_i\right) \in \mathbb{R}^{B_q \times B_k}$

    Compute $\boldsymbol{dV}_i^{(j)} = \boldsymbol{dV}_{i-1}^{(j)} + \left(\boldsymbol{P}_i^{(j)}\right)^\top \boldsymbol{dO}_i \in \mathbb{R}^{B_k \times d}$

    Compute $\boldsymbol{dP}_i^{(j)} = \boldsymbol{dO}_i \left(\boldsymbol{V}^{(j)}\right)^\top \in \mathbb{R}^{B_q \times B_k}$

    Compute $\boldsymbol{dS}_i^{(j)} = \boldsymbol{P}_i^{(j)} \circ \left(\boldsymbol{dP}_i^{(j)} - D_i\right) \in \mathbb{R}^{B_q \times B_k}$

    Compute $\boldsymbol{dK}_i^{(j)} = \boldsymbol{dK}_{i-1}^{(j)} + \left(\boldsymbol{dS}_i^{(j)}\right)^\top \boldsymbol{Q}_i / \sqrt{d} \in \mathbb{R}^{B_k \times d}$.

  end for

  Write $\boldsymbol{dK}_{T_q}^{(j)}$ and $\boldsymbol{dV}_{T_q}^{(j)}$ to global memory as the j-th tiles of $\boldsymbol{dK}$ and $\boldsymbol{dV}$.

end for

for $i = 1, ..., T_q$ **do**

  Load $\boldsymbol{Q}_i, \boldsymbol{dO}_i$ from global memory

  Initialize $\boldsymbol{dQ}_i^{(0)} = \boldsymbol{0} \in \mathbb{R}^{B_q \times d}$

  for $j = 1, ..., T_k$ **do**

    Load $\boldsymbol{K}^{(j)}, \boldsymbol{V}^{(j)}$ from global memory

    Compute tile of attention scores $\boldsymbol{S}_i^{(j)} = \frac{\boldsymbol{Q}_i (\boldsymbol{K}^{(j)})^\top}{\sqrt{d}} \in \mathbb{R}^{B_q \times B_k}$

    Compute attention probabilities $\boldsymbol{P}_i^{(j)} = \exp\left(\boldsymbol{S}_i^{(j)} - L_i\right) \in \mathbb{R}^{B_q \times B_k}$

    Compute $\boldsymbol{dP}_i^{(j)} = \boldsymbol{dO}_i \left(\boldsymbol{V}^{(j)}\right)^\top \in \mathbb{R}^{B_q \times B_k}$

    Compute $\boldsymbol{dS}_i^{(j)} = \boldsymbol{P}_i^{(j)} \circ \left(\boldsymbol{dP}_i^{(j)} - D_i\right) \in \mathbb{R}^{B_q \times B_k}$

    Compute $\boldsymbol{dQ}_i^{(j)} = \boldsymbol{dQ}_i^{(j-1)} + \boldsymbol{dS}_i^{(j)} \boldsymbol{K}^{(j)} / \sqrt{d} \in \mathbb{R}^{B_q \times d}$

  end for

  Write $\boldsymbol{dQ}_i^{(T_k)}$ to global memory as the i-th tile of $\boldsymbol{dQ}$.

end for

Return $\boldsymbol{dQ}, \boldsymbol{dK}, \boldsymbol{dV}$.

5 Distributed Data Parallel Training

In this next part of the assignment, we'll explore methods for using multiple GPUs to train our language models, focusing on data parallelism. We'll start with a primer on distributed communication in PyTorch. Then, we'll study a naïve implementation of distributed data parallel training, then implement and benchmark various improvements to communication efficiency.

中文译文 / 对应原文第 30 页

5.1 PyTorch 单节点分布式通信

先看简单分布式应用，目标是生成四个随机整数张量并求和。

右侧例子启动四个工作进程，各生成随机整数张量。跨进程求和使用 all-reduce 集合通信，它将每个进程的数据张量替换为归约结果，即张量之和。

完整代码见右侧。setup 设置 MASTER_ADDR="localhost"、MASTER_PORT="29500"，用 gloo 初始化进程组。distributed_demo 初始化后生成随机数据，打印归约前数据，同步调用 dist.all_reduce，再打印归约后数据。主程序设置 world_size=4，用 mp.spawn 启动四个进程并等待结束。

运行得到右侧输出。各进程最初持有不同张量；all-reduce 求和后，所有进程的 data 被原地修改，持有相同的结果 [22, 16, 25]。[脚注 2]

回看脚本，mp.spawn 启动 nprocs 个进程，以 args 调用 fn。实际调用为 fn(rank, *args)，rank 是工作进程索引，范围 0 到 nprocs-1。因此 distributed_demo 第一个位置参数必须接收整数 rank。下一页接续。

脚注 2：多次运行会发现打印顺序不确定。分布式环境中无法控制命令执行的准确顺序；唯一保证是 all-reduce 完成后，各进程持有逐位相同的结果张量。

5.1 Single-Node Distributed Communication in PyTorch

Let's start by looking at a simple distributed application in PyTorch, where the goal is to generate four random integer tensors and compute their sum.

In the distributed case below, we will spawn four worker processes, each of which generates a random integer tensor. To sum these tensors across the worker processes, we will call the **all-reduce** collective communication operation, which replaces the original data tensor on each process with the all-reduced result (i.e., the sum).

Now let's take a look at some code.

```
import os
import torch
import torch.distributed as dist
import torch.multiprocessing as mp

def setup(rank, world_size):
    os.environ["MASTER_ADDR"] = "localhost"
    os.environ["MASTER_PORT"] = "29500"
    dist.init_process_group("gloo", rank=rank, world_size=world_size)

def distributed_demo(rank, world_size):
    setup(rank, world_size)
    data = torch.randint(0, 10, (3,))
    print(f"rank {rank} data (before all-reduce): {data}")
    dist.all_reduce(data, async_op=False)
    print(f"rank {rank} data (after all-reduce): {data}")

if __name__ == "__main__":
    world_size = 4
    mp.spawn(fn=distributed_demo, args=(world_size, ), nprocs=world_size, join=True)
```

After running the script above, we get the output below. As expected, each worker process initially holds different `data` tensors. After the all-reduce operation, which sums the tensors across all of the worker processes, `data` is modified in-place on each of the worker processes to hold the all-reduced result.[2]

```
$ uv run python distributed_hello_world.py
rank 3 data (before all-reduce): tensor([3, 7, 8])
rank 0 data (before all-reduce): tensor([4, 4, 7])
rank 2 data (before all-reduce): tensor([6, 0, 7])
rank 1 data (before all-reduce): tensor([9, 5, 3])
rank 1 data (after all-reduce): tensor([22, 16, 25])
rank 0 data (after all-reduce): tensor([22, 16, 25])
rank 3 data (after all-reduce): tensor([22, 16, 25])
rank 2 data (after all-reduce): tensor([22, 16, 25])
```

Let's now look back more closely at our script above. The command `mp.spawn` spawns `nprocs` processes that run `fn` with the provided `args`. In addition, the function `fn` is called as `fn(rank, *args)`, where rank is the index of the worker process (a value between 0 and `nprocs-1`). Thus, our `distributed_demo` function

[2] If you run this script multiple times, you'll notice that the order of the printed output is not deterministic. Since this application is running in a distributed setting, we cannot control the exact order in which commands are being run—our only guarantee is that after the all-reduce operation is complete, the separate processes will hold bitwise identical result tensors.

还传入 world_size，表示工作进程总数。

每个工作进程属于通过 dist.init_process_group 初始化的进程组。多个工作进程通过共享的主进程协调通信；主进程由 IP 地址和端口定义，运行 rank 0 进程。all-reduce 等集合通信在组内各进程上操作。

本例用 gloo 后端，也有其他后端。nccl 使用 NVIDIA NCCL 集合通信库，对 CUDA 张量通常更快，但只能用于有 GPU 的机器；Gloo 可用于纯 CPU 机器。GPU 分布式训练始终使用 NCCL，没有 GPU 的本地开发才用 Gloo。本例使用 Gloo 是为支持 CPU 机器的本地执行与开发。

多 GPU 任务确保不同 rank 使用不同 GPU。可在 setup 调用 torch.cuda.set_device(rank)，使 tensor.to("cuda") 自动移到指定设备；也可为各 rank 显式创建设备字符串，如 device = f"cuda:{rank}"，用于数据移动。

术语

后续或其他资料可能出现以下术语。本作业聚焦单节点多进程，但这些术语也用于一般分布式训练，示意图见图 3。

• node（节点）：网络上的一台机器。

• worker（工作实例）：参与训练的程序实例。本作业每个 worker 为一个进程，worker、process、worker process 互换使用。但实际 worker 可能使用多进程，例如加载训练数据，因此不总等价。

• world size：进程组的 worker 总数。

• global rank（全局编号）：唯一标识组内 worker 的整数 ID，范围 0 到 world_size-1。world size 为 2 时，主进程为 0，另一进程为 1。

• local world size：某个节点本地运行的 worker 数。两个节点各有 4 个 worker 时，world size=8，local world size=4。单节点时，本地与全局数量相同。

• local rank（本地编号）：唯一标识机器上本地 worker 的整数 ID，范围 0 到 local_world_size-1。两个节点各有 4 个进程时，每节点本地编号均为 0、1、2、3。单节点多进程时，本地与全局 rank 相同。

must accept this integer rank as its first positional argument. In addition, we pass in the `world_size`, which refers to the total number of worker processes.

Each worker process belongs to a *process group*, which is initialized via `dist.init_process_group`. The process group represents multiple worker processes that will coordinate and communicate via a shared master. The master is defined by its IP address and port, and the master runs the process with rank 0. Collective communication operations like all-reduce operate on each process in the process group.

In this case, we initialized our process group with the `"gloo"` backend, but other backends are available. In particular, the `"nccl"` backend will use the NVIDIA NCCL collective communications library, which will generally be more performant for CUDA tensors. However, NCCL can only be used on machines with GPUs, while Gloo can be run on CPU-only machines. You should always use NCCL for distributed GPU training, and only use Gloo for local development where you don't have a GPU available. We used Gloo in this example because it enables local execution and development on CPU-only machines.

When running multi-GPU jobs, make sure that different ranks use different GPUs. One method for doing this is to call `torch.cuda.set_device(rank)` in the setup function, so that `tensor.to("cuda")` will automatically move it to the specified device. Alternatively, you can explicitly create a per-rank device string (e.g., `device = f"cuda:{rank}"`), and then use this device string as the target device for any data movement (e.g., `tensor.to(f"cuda:{rank}")`).

Terminology

In the rest of the assignment (and various other resources you might see online), you may encounter the following terms in the context of PyTorch distributed communication. Though we will focus on single-node, multi-process distributed training in this assignment, the terminology is useful for understanding distributed training in general. See Figure 3 for a visual representation.

`node` a machine on the network.

`worker` an instance of a program that's participating in the distributed training. In this assignment, each worker will have a single process, so we'll use *worker*, *process*, and *worker process* interchangeably. However, a worker may use multiple processes (e.g., to load data for training), so these terms are not always equivalent in practice.

`world size` The number of total workers in a process group.

`global rank` An integer ID (between 0 and `world_size-1`) that uniquely identifies a worker in the process group. For example, for world size of two, one process will have global rank 0 (the master process) and the other process will have rank 1.

`local world size` When running applications across different nodes, the local world size is the number of workers running locally on a given node. For example, if we have an application that spawns 4 workers on 2 nodes each, the world size would be 8 and the local world size would be 4. Note that when running on a single node, the local world size of a worker is equivalent to the (global) world size.

`local rank` An integer ID (between 0 and `local_world_size-1`) that uniquely identifies the index of a local worker on the machine. For example, if we have an application that spawns 4 processes on 2 nodes each, each node would have workers with local ranks 0, 1, 2, and 3. Note that when running a single-node multi-process distributed application, the local rank of a process is equivalent to its global rank.

图 3：2 个节点、world size 为 8 的分布式应用。每个工作进程由 global rank（0-7）及 local rank（0-3）标识。图来自：

https://lightning.ai/docs/fabric/stable/advanced/distributed_communication.html

5.1.1 分布式应用基准测试的最佳实践

本部分将反复测试分布式应用，了解通信开销。建议：

• 尽可能在同一机器上测试，便于控制变量比较。

• 测量前先运行几个预热步骤，对 NCCL 通信尤其重要；通常 5 次足够。

• GPU 上计时，调用 torch.cuda.synchronize() 等待 CUDA 操作完成。即使用 async_op=False 也需要，因为函数在操作排入 GPU 队列后就返回，而非在通信完成后返回。[脚注 3]

• 不同 rank 的耗时可能略有差异，通常汇总各 rank 测量改善估计。all-gather，特别是 dist.all_gather_object，可用于收集结果。

• 通常先在本地 CPU 用 Gloo 调试，再按题目要求在 GPU 用 NCCL 测试。

切换后端只需修改 init_process_group 调用和张量设备转换。

题目（distributed_communication_single_node）：分布式通信（单节点）（5 分）

编写脚本，测试单节点多进程中 all-reduce 的运行时间。上面的示例可作为起点。改变以下设置：

• all-reduce 数据大小：float32 张量，1MB、10MB、100MB、1GB。

• GPU/进程数量：2、4、6。

资源要求：最多 6 个 GPU，每次测试应少于 5 分钟。

提交内容：对比各设置的图和/或表，以及 2-3 句话讨论结果及各因素如何相互影响。

脚注 3：更多细节见：

https://github.com/pytorch/pytorch/issues/68112#issuecomment-965932386

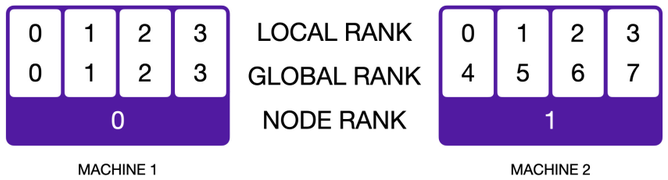


Figure 3: A schematic representation of a distributed application running on 2 nodes with a world size of 8. Each worker process is identified by a global rank (from 0 to 7) and a local rank (from 0 to 3). Figure taken from `https://lightning.ai/docs/fabric/stable/advanced/distributed_communication.html`

5.1.1 Best Practices for Benchmarking Distributed Applications

Throughout this portion of the assignment you will be benchmarking distributed applications to better understand the overhead from communication. Here are a few best practices:

- Whenever possible, run benchmarks on the same machine to facilitate controlled comparisons.
- Perform several warm-up steps before timing the operation of interest. This is especially important for NCCL communication calls. 5 iterations of warmup is generally sufficient.
- Call `torch.cuda.synchronize()` to wait for CUDA operations to complete when benchmarking on GPUs. Note that this is necessary even when calling communication operations with `async_op=False`, which returns when the operation is queued on the GPU (as opposed to when the communication actually finishes).[3]
- Timings may vary slightly across different ranks, so it's common to aggregate measurements across ranks to improve estimates. You may find the all-gather collective (specifically the `dist.all_gather_object` function) to be useful for collecting results from all ranks.
- In general, debug locally with Gloo on CPU, and then as required in a given problem, benchmark with NCCL on GPU.

Switching between the backends should be as simple as modifying your `init_process_group` call and tensor device casts.

Problem (`distributed_communication_single_node`): Distributed Communication (Single Node) (5 points)

Write a script to benchmark the runtime of the all-reduce operation in the single-node multi-process setup. The example code above may provide a reasonable starting point. Experiment with varying the following settings:

`all-reduce` **`data size`** float32 data tensors ranging over 1MB, 10MB, 100MB, 1GB.

`Number of GPUs/processes` 2, 4, or 6.

Resource requirements: Up to 6 GPUs. Each benchmarking run should take less than 5 minutes.

Deliverable: Plot(s) and/or table(s) comparing the various settings, with 2-3 sentences of commentary about your results and thoughts about how the various factors interact.

[3] See `https://github.com/pytorch/pytorch/issues/68112\#issuecomment-965932386` for more details.

5.2 朴素的分布式数据并行训练实现

了解 PyTorch 分布式应用基础后，构建一个最小的分布式数据并行（DDP）训练实现。

数据并行将批次拆分到多个设备（例如 GPU），从而支持单设备无法容纳的大批次。例如四个设备各自最多容纳批大小 32，则数据并行可支持有效批大小 128。

朴素 DDP 的过程如下。最初每个设备各自构建随机初始化的模型，通过 broadcast 集合通信，将 rank 0 的模型参数发送到其他 rank。训练开始时，各设备持有相同的模型参数和优化器状态副本，例如 Adam 累积的梯度统计量。

1. 给定 n 个样本的批次，进行分片，每个设备收到互不重叠的 n/d 个样本，d 为数据并行设备数。d 应整除 n，否则各 rank 工作量不均，步骤会受最慢者限制。

2. 各设备使用本地模型参数，对 n/d 个样本运行前向和反向传播。此时各设备持有仅由自己收到的样本计算出的梯度。

3. 用 all-reduce 对不同设备的梯度取平均，使各设备持有全部 n 个样本上的平均梯度。

4. 各设备执行优化器更新，更新自己的参数副本；对优化器而言，只是在优化本地模型。由于模型和优化器状态的初值相同，且每轮使用相同平均梯度，各设备参数与优化器状态保持同步。至此完成一轮训练，可以重复上述过程。

题目（naive_ddp）：朴素 DDP（5 分）

提交内容：实现朴素 DDP，在反向传播结束后，对各参数梯度分别执行 all-reduce。实现 [adapters.get_ddp]，以及可选的 [adapters.ddp_on_after_backward]，然后运行 uv run pytest tests/test_ddp.py 测试。

题目（naive_ddp_benchmarking）：朴素 DDP 基准测试（3 分）

此实现每次反向传播后，对各参数梯度分别跨 rank 执行 all-reduce。为了更好地理解数据并行开销，编写脚本，测试此前语言模型使用该 DDP 实现训练的性能。测量每训练步骤的总耗时，以及梯度通信耗时占比。使用单节点 2 个 GPU，以及第 2.1.2 节的 xl 模型。

提交内容：基准测试设置说明，以及每种设置下的每轮训练耗时和梯度通信耗时。

5.2 A Naïve Implementation of Distributed Data Parallel Training

Now that we've seen the basics of writing distributed applications in PyTorch, let's build a minimal implementation of distributed data parallel (DDP) training.

Data parallelism splits batches across multiple devices (e.g., GPUs), enabling training on large batch sizes that do not fit on a single device. For example, given four devices that can each handle a maximum batch size of 32, data parallel training would enable an effective batch size of 128.

Here are the steps for naïvely doing distributed data parallel training. Initially, each device constructs a (randomly-initialized) model. We use the broadcast collective communication operation to send the model parameters from rank 0 to all other ranks. At the start of training, each device holds an identical copy of the model parameters and optimizer states (e.g. the accumulated gradient statistics in Adam).

1. Given a batch with n examples, the batch is sharded and each device receives n/d disjoint examples (where d is the number of devices used for data parallel training). d should divide n, otherwise some ranks would do more work than others, and the step is bottlenecked by the slowest.
2. Each device uses its local copy of the model parameters to run a forward pass on its n/d examples and a backward pass to calculate the gradients. Note that at this point, each device holds the gradients computed from the n/d examples it received.
3. We then use the all-reduce collective communication operation to average the gradients across the different devices, so each device holds the gradients averaged across all n examples.
4. Next, each device runs an optimizer step to update its copy of the parameters—from the optimizer's perspective, it is simply optimizing a local model. The parameters and optimizer states will stay in sync on all of the different devices since they all start from the same initial model and optimizer state and use the same averaged gradients for each iteration. At this point, we've completed a single training iteration and can repeat the process.

Problem (`naive_ddp`): Naïve DDP (5 points)

Deliverable: Implement a naïve form of distributed data parallel training that all-reduces individual parameter gradients after the backward pass. To test your implementation, implement [`adapters.get_ddp`] and (optionally) [`adapters.ddp_on_after_backward`], then run `uv run pytest tests/test_ddp.py`.

Problem (`naive_ddp_benchmarking`): Naïve DDP Benchmarking (3 points)

In this naïve DDP implementation, parameter gradients are individually all-reduced across ranks after each backward pass. To better understand the overhead of data parallel training, create a script to benchmark your previously-implemented language model when trained with this naïve implementation of DDP. Measure the total time per training step and the proportion of time spent on communicating gradients. Collect measurements in the single-node setting (1 node x 2 GPUs) for the xl model size described in Section 2.1.2.

Deliverable: A description of your benchmarking setup, along with the measured time per training iteration and time spent communicating gradients for each setting.

5.3 改进最小 DDP 实现

第 5.2 节的实现有两项主要限制：

1. 每个参数张量单独执行一次 all-reduce。每次通信调用都有开销，将通信合并成批可能降低开销。

2. 等待全部反向传播完成才通信。但反向传播逐步计算梯度，因此某个参数梯度一旦就绪便可立即通信，无需等待其他参数。这让梯度通信与反向计算重叠，降低 DDP 开销。

本部分依次处理这些限制，测量对训练速度的影响。

5.3.1 减少通信调用次数

尝试把 all-reduce 合并，而不是每参数一次调用：将待归约的梯度拼接成一个张量，再跨全部 rank 对合并后的梯度执行 all-reduce。torch._utils._flatten_dense_tensors 和 torch._utils._unflatten_dense_tensors 可能有用。

题目（minimal_ddp_flat_benchmarking）：展平梯度的最小 DDP 基准测试（2 分）

修改最小 DDP，使其通信一个包含全部参数展平梯度的张量。在此前相同条件下（单节点 2 GPU，第 2.1.2 节 xl 模型），与每参数张量单独 all-reduce 的实现比较。

提交内容：使用一次合并 all-reduce 时，每轮训练耗时和梯度通信耗时；并用 1-2 句话比较合并通信和逐个通信的结果。

5.3.2 将逐参数梯度通信与计算重叠

合并调用可能降低大量小 all-reduce 的调用开销，但全部通信时间仍直接增加总开销。利用反向传播从损失向输入逐层计算梯度这一事实，可以在梯度就绪后立即 all-reduce，使反向计算与通信重叠。

首先实现并测试一个 DDP 包装器，在反向传播中，参数梯度一旦就绪，就异步归约对应张量。以下提示可能有用：

反向钩子：使用 register_post_accumulate_grad_hook，在反向传播完成某参数的梯度累积后自动调用函数。[脚注 4]

脚注 4：更多信息和示例见：

https://pytorch.org/docs/stable/generated/torch.Tensor.register_post_accumulate_grad_hook.html

5.3 Improving Upon the Minimal DDP Implementation

The minimal DDP implementation that we saw in Section 5.2 has a couple of key limitations:

1. It conducts a separate all-reduce operation for every parameter tensor. Each communication call incurs overhead, so it may be advantageous to batch communication calls to minimize this overhead.
2. It waits for the backward pass to finish before communicating gradients. However, the backward pass is incrementally computed. Thus, when a parameter gradient is ready, it can immediately be communicated without waiting for the gradients of the other parameters. This allows us to overlap communication of gradients with computation of the backward pass, reducing the overhead of distributed data parallel training.

In this part of the assignment, we'll address each of these limitations in turn and measure the impact on training speed.

5.3.1 Reducing the Number of Communication Calls

Rather than issuing a communication call for each parameter tensor, let's see if we can improve performance by batching the all-reduce. Concretely, we'll take the gradients that we want to all-reduce, concatenate them into a single tensor, and then all-reduce the combined gradients across all ranks. It might be helpful to use `torch._utils._flatten_dense_tensors` and `torch._utils._unflatten_dense_tensors`.

Problem (`minimal_ddp_flat_benchmarking`): Minimal DDP with Flat Gradients Benchmarking (2 points)

Modify your minimal DDP implementation to communicate a tensor with flattened gradients from all parameters. Compare its performance with the minimal DDP implementation that issues an all-reduce for each parameter tensor under the previously-used conditions (1 node x 2 GPUs, xl model size as described in Section 2.1.2).

Deliverable: The measured time per training iteration and time spent communicating gradients under distributed data parallel training with a single batched all-reduce call. 1-2 sentences comparing the results when batching vs. individually communicating gradients.

5.3.2 Overlapping Computation with Communication of Individual Parameter Gradients

While batching the communication calls might help lower the overhead associated with issuing a large number of small all-reduce operations, all of the communication time still directly contributes to the overhead. To resolve this, we can take advantage of the observation that the backward pass incrementally computes gradients for each layer (starting from the loss and moving toward the input)—thus, we can all-reduce parameter gradients as soon as they're ready, reducing the overhead of data parallel training by overlapping computation of the backward pass with communication of gradients.

We'll start by implementing and benchmarking a distributed data parallel wrapper that asynchronously all-reduces individual parameter tensors as they become ready during the backward pass. The following pointers may be useful:

Backward hooks To automatically call a function on a parameter after its gradient has been accumulated in the backward pass, you can use the `register_post_accumulate_grad_hook` function.[4]

[4] See `https://pytorch.org/docs/stable/generated/torch.Tensor.register_post_accumulate_grad_hook.html` for more information and usage examples.

中文译文 / 对应原文第 35 页

异步通信：所有 PyTorch 集合通信都支持同步（async_op=False）和异步（async_op=True）执行。同步调用阻塞到集合操作排入 GPU 队列；CUDA 是异步的，因此这不代表操作已完成。不过，后续使用其输出的函数调用会按预期工作。[脚注 5]

异步调用返回分布式请求句柄。函数返回时，不能保证集合操作已排入 GPU 队列，更不能保证已完成。调用返回句柄的 handle.wait()，等待操作排入 GPU 队列，使其输出可安全用于后续运算。

右侧给出对张量列表逐个 all-reduce 的同步和异步示例。

代码注释译文：

• 同步方式：阻塞到操作排入 GPU 队列。

• 异步方式：每次调用立即返回，最后等待结果。

• 中间可执行不依赖 all_reduce 结果的其他命令。

• 最后确保 all-reduce 已排队，因而依赖其输出的其他操作也可排队；等待所有句柄后清空列表。

题目（ddp_overlap_individual_parameters）：逐参数通信重叠的 DDP（5 分）

实现一个处理 DDP 训练的 Python 类。包装任意 PyTorch nn.Module，训练前广播权重，保证各 rank 初始参数相同，并发起用于梯度平均的通信。建议接口如下（原始签名保留在右侧）：

• __init__(self, module: torch.nn.Module)：给定已实例化、需要并行的模块，构建处理跨 rank 梯度同步的 DDP 容器。

• forward(self, *inputs, **kwargs)：使用给定位置和关键字参数，调用被包装模块的 forward()。

接口在下一页接续。

脚注 5：高级情形中，如果使用多个 CUDA 流，可能需要显式跨流同步，确保后续操作使用输出时已就绪。参见：

https://pytorch.org/docs/stable/notes/cuda.html#cuda-streams

Asynchronous communication All PyTorch collective communication operations support synchronous (`async_op=False`) and asynchronous execution (`async_op=True`). Synchronous calls will block until the collective operation is queued on the GPU. This does not mean that the CUDA operation is completed since CUDA operations are asynchronous. That being said, later function calls using the output will behave as expected.[5] In contrast, asynchronous calls will return a distributed request handle—as a result, when the function returns, the collective communication operation is not guaranteed to have been queued on the GPU, let alone completed. To wait for the operation to be queued on the GPU (and therefore for the output to be usable in later operations), you can call `handle.wait()` on the returned communication handle.

For example, the following two examples all-reduce each tensor in a list of tensors with either a synchronous or an asynchronous call:

```python
tensors = [torch.rand(5) for _ in range(10)]

# Synchronous, block until operation is queued on the GPU.
for tensor in tensors:
    dist.all_reduce(tensor, async_op=False)

# Asynchronous, return immediately after each call and
# wait on results at the end.
handles = []
for tensor in tensors:
    handle = dist.all_reduce(tensor, async_op=True)
    handles.append(handle)

# ...
# Possibly execute other commands that don't rely on the all_reduce results

# Ensure that all-reduce calls were queued and
# therefore other operations depending on the
# all-reduce output can be queued.
for handle in handles:
    handle.wait()
handles.clear()
```

Problem (`ddp_overlap_individual_parameters`): DDP with Overlapping Individual Parameters (5 points)

Implement a Python class to handle distributed data parallel training. The class should wrap an arbitrary PyTorch `nn.Module` and take care of broadcasting the weights before training (so all ranks have the same initial parameters) and issuing communication calls for gradient averaging. We recommend the following public interface:

`def __init__(self, module: torch.nn.Module):` Given an instantiated PyTorch `nn.Module` to be parallelized, construct a DDP container that will handle gradient synchronization across ranks.

`def forward(self, *inputs, **kwargs):` Calls the wrapped module's `forward()` method with the provided positional and keyword arguments.

[5] In advanced cases, if you are using multiple CUDA streams, you may need explicit synchronization across streams to ensure that the output is ready for later operations. See `https://pytorch.org/docs/stable/notes/cuda.html\#cuda-streams`.

• finish_gradient_synchronization(self)：调用时等待异步通信在 GPU 上结束。

使用该类进行训练时，将模块传入包装器，并在 optimizer.step() 前调用 finish_gradient_synchronization()，保证依赖梯度的优化器更新可以安全排队。完整训练使用示例见右侧。

提交内容：处理 DDP 训练的容器类，将梯度通信与反向计算重叠。先实现 [adapters.get_ddp] 和 [adapters.ddp_on_after_backward]；后者可选，具体实现可能不需要。运行 uv run pytest tests/test_ddp.py，建议重复多次，例如 5 次，确保稳定通过。

题目（ddp_overlap_individual_parameters_benchmarking）：逐参数通信重叠的 DDP 基准测试（1 分）

(a) 测试将反向计算与各参数梯度通信重叠时的 DDP 性能。与之前的两个最小实现比较：每参数一次 all-reduce，以及全部参数梯度拼接后一次 all-reduce。配置相同：单节点、2 GPU、第 2.1.2 节的 xl 模型。

提交内容：通信与反向重叠时，每轮训练的实测耗时；并用 1-2 句话比较结果。

(b) 用 Nsight 对基准代码插桩，配置仍为单节点、2 GPU、xl 模型，比较最初 DDP 与重叠实现。直观比较两份轨迹，提供截图，证明一种实现使计算与通信重叠，另一种没有。

提交内容：两张截图，分别来自最初 DDP 和通信计算重叠版 DDP，直观显示通信是否与反向传播重叠。

def finish_gradient_synchronization(self): When called, wait for asynchronous communication calls to finish on the GPU.

To use this class to perform distributed training, we'll pass it a module to wrap, and then add a call to `finish_gradient_synchronization()` before we run `optimizer.step()` to ensure that the optimizer step, an operation that depends on the gradients, can be safely queued:

```
model = ToyModel().to(device)
ddp_model = DDP(model)

for _ in range(train_steps):
    x, y = get_batch()
    logits = ddp_model(x)
    loss = loss_fn(logits, y)
    loss.backward()
    ddp_model.finish_gradient_synchronization()
    optimizer.step()
```

Deliverable: Implement a container class to handle distributed data parallel training. This class should overlap gradient communication and the computation of the backward pass. To test your DDP class, first implement the adapters [**adapters.get_ddp**] and [**adapters.ddp_on_after_backward**] (the latter is optional, depending on your implementation you may not need it).

Then, to execute the tests, run `uv run pytest tests/test_ddp.py`. We recommend running the tests multiple times (e.g., 5) to ensure that it passes reliably.

Problem (ddp_overlap_individual_parameters_benchmarking): DDP Overlapping Individual Parameters Benchmarking (1 point)

(a) Benchmark the performance of your DDP implementation when overlapping backward pass computation with communication of individual parameter gradients. Compare its performance with our previously-studied settings (the minimal DDP implementation that either issues an all-reduce for each parameter tensor, or a single all-reduce on the concatenation of all parameter tensors) with the same setup: 1 node, 2 GPUs, and the xl model size described in Section 2.1.2.

Deliverable: The measured time per training iteration when overlapping the backward pass with communication of individual parameter gradients, with 1-2 sentences comparing the results.

(b) Instrument your benchmarking code (using the 1 node, 2 GPUs, xl model size setup) with the Nsight profiler, comparing the initial DDP implementation with this overlapped implementation. Visually compare the two traces, and provide a profiler screenshot demonstrating that one implementation overlaps compute with communication while the other doesn't.

Deliverable: 2 screenshots (one from the initial DDP implementation, and another from this DDP implementation that overlaps compute with communication) that visually show that communication is or isn't overlapped with the backward pass.

6 优化器状态分片

DDP 概念简单，通常也有效，但每个 rank 都保存独立的模型参数和优化器状态副本，冗余带来显著显存成本。例如 AdamW 为每参数维护两个浮点数，因而占用相当于模型权重两倍的显存。S. Rajbhandari 等 [5] 描述了多种方法，将（1）优化器状态、（2）梯度、（3）参数划分到不同 rank，并按需通信，减少数据并行中的冗余。

本部分实现简化的优化器状态分片，降低各 rank 的显存消耗。每个 rank 的优化器只处理部分参数，约为 1/world_size，而非全部参数。执行更新时只更新自己分片的参数，再将更新后的参数广播到其他 rank，确保每次优化器更新后模型参数同步。

题目（optimizer_state_sharding）：优化器状态分片（15 分）

实现 Python 类，包装任意输入的 PyTorch optim.Optimizer，并在每次更新后同步参数。建议接口如下：

• __init__(self, params, optimizer_cls: Type[Optimizer], **kwargs: Any)：初始化状态分片优化器。params 是待优化参数的集合，或参数组，以支持对模型不同部分使用不同学习率等超参数；它们将跨所有 rank 分片。optimizer_cls 指定要包装的优化器类型，例如 optim.AdamW。其余关键字参数传递到该优化器构造器。务必在此方法中调用 torch.optim.Optimizer 父类构造器。

• step(self, closure, **kwargs)：用给定 closure 和关键字参数调用被包装优化器的 step()。更新参数后，与其他 rank 同步。

• add_param_group(self, param_group: dict[str, Any])：向分片优化器添加参数组。父类构造器在构建时会调用它，训练中也可能调用，例如逐步解冻模型层。因此它需要处理模型参数到各 rank 的分配。

提交内容：处理优化器状态分片的容器类。实现 [adapters.get_sharded_optimizer]，运行 uv run pytest tests/test_sharded_optimizer.py。建议重复多次，例如 5 次，确保稳定通过。

实现后，分析训练峰值显存与运行时间开销。

6 Optimizer State Sharding

Distributed data parallel training is conceptually simple and often very effective, but requires each rank to hold a distinct copy of the model parameters and optimizer state. This redundancy can come with significant memory costs. For example, the AdamW optimizer maintains two floats per parameter, meaning that it consumes twice as much memory as the model weights. S. Rajbhandari et al. [5] describe several methods for reducing this redundancy in data-parallel training by partitioning the (1) optimizer state, (2) gradients, and (3) parameters across ranks, communicating them between workers as necessary.

In this part of the assignment, we'll reduce per-rank memory consumption by implementing a simplified version of optimizer state sharding. Rather than keeping the optimizer states for all parameters, each rank's optimizer instance will only handle a subset of the parameters (approximately 1 / `world_size`). When each rank's optimizer takes an optimizer step, it'll only update the subset of model parameters in its shard. Then, each rank will broadcast its updated parameters to the other ranks to ensure that the model parameters remain synchronized after each optimizer step.

Problem (`optimizer_state_sharding`): Optimizer State Sharding (15 points)

Implement a Python class to handle optimizer state sharding. The class should wrap an arbitrary input PyTorch `optim.Optimizer` and take care of synchronizing updated parameters after each optimizer step. We recommend the following public interface:

`def __init__(self, params, optimizer_cls: Type[Optimizer], **kwargs: Any):` Initializes the sharded state optimizer. `params` is a collection of parameters to be optimized (or parameter groups, in case the user wants to use different hyperparameters, such as learning rates, for different parts of the model); these parameters will be sharded across all the ranks. The `optimizer_cls` parameter specifies the type of optimizer to be wrapped (e.g., `optim.AdamW`). Finally, any remaining keyword arguments are forwarded to the constructor of the `optimizer_cls`. Make sure to call the `torch.optim.Optimizer` super-class constructor in this method.

`def step(self, closure, **kwargs):` Calls the wrapped optimizer's `step()` method with the provided closure and keyword arguments. After updating the parameters, synchronize with the other ranks.

`def add_param_group(self, param_group: dict[str, Any]):` This method should add a parameter group to the sharded optimizer. This is called during construction of the sharded optimizer by the super-class constructor and may also be called during training (e.g., for gradually unfreezing layers in a model). As a result, this method should handle assigning the model's parameters among the ranks.

Deliverable: Implement a container class to handle optimizer state sharding. To test your sharded optimizer, first implement the adapter [`adapters.get_sharded_optimizer`] . Then, to execute the tests, run `uv run pytest tests/test_sharded_optimizer.py`. We recommend running the tests multiple times (e.g., 5) to ensure that they pass reliably.

Now that we've implemented optimizer state sharding, let's analyze its effect on the peak memory usage during training and its runtime overhead.

题目（optimizer_state_sharding_accounting）：优化器状态分片开销核算（5 分）

(a) 编写脚本，分析有/无优化器状态分片时训练语言模型的峰值显存。标准配置为单节点、2 GPU、xl 模型。报告初始化模型后、优化器更新前、优化器更新后三个时刻的峰值显存。是否符合预期？分解各设置的显存开销，例如参数与优化器状态各占多少。

提交内容：用 2-3 句话回答，提供峰值显存和各模型/优化器组件的显存分解。

(b) 分片实现如何影响训练速度？标准配置下分别测量有/无分片时每轮耗时。

提交内容：用 2-3 句话回答，给出耗时。

(c) 我们的方案与 ZeRO 第 1 阶段有何区别？后者在 S. Rajbhandari、J. Rasley、O. Ruwase 和 Y. He [5] 中称为 ZeRO-DP P_os。

提交内容：用 2-3 句话概述差异，特别是显存和通信量相关差异。

7 全分片数据并行

借助优化器状态分片和数据并行，可以沿数据并行轴拆分优化器状态及激活。但模型权重仍重复，每 GPU 都保存完整权重。

将数据并行（DP）轴改为全分片数据并行（FSDP）轴可以解决。每 GPU 仅保存各权重张量的一个切片，但前向/反向前需使用 all-gather，从其他 GPU 获取切片，形成完整权重。

为避免 GPU 计算等待通信，大多数 FSDP 实现会提前调度对应层的 all-gather，在需要时权重已就绪，防止通信阻塞计算。这样权重分片通信不在关键路径上；只要通信跟得上计算且调度合理，就不增加耗时。

某些层的显存和计算量很小，传输延迟开销不值得。应标记这些层不进行 FSDP 分片；本架构主要是归一化层。因此需要分片的是嵌入层和所有线性层。

主权重必须以 FP32 保存，因为反复累加的值对精度敏感；但实际计算不必用 FP32。混合精度中使用前总要转为低精度计算类型，因此可在通信前就转换，节省带宽。

题目（fsdp）：全分片数据并行（15 分）

题目在下页继续。

Problem (`optimizer_state_sharding_accounting`): Optimizer State Sharding Accounting (5 points)

(a) Create a script to profile the peak memory usage when training language models with and without optimizer state sharding. Using the standard configuration (1 node, 2 GPUs, xl model size), report the peak memory usage after model initialization, directly before the optimizer step, and directly after the optimizer step. Do the results align with your expectations? Break down the memory usage in each setting (e.g., how much memory for parameters, how much for optimizer states, etc.).

Deliverable: 2-3 sentence response with peak memory usage results and a breakdown of how the memory is divided between different model and optimizer components.

(b) How does our implementation of optimizer state sharding affect training speed? Measure the time taken per iteration with and without optimizer state sharding for the standard configuration (1 node, 2 GPUs, xl model size).

Deliverable: 2-3 sentence response with your timings.

(c) How does our approach to optimizer state sharding differ from ZeRO stage 1 (described as ZeRO-DP P_{os} in S. Rajbhandari, J. Rasley, O. Ruwase, and Y. He [5])?

Deliverable: 2-3 sentence summary of any differences, especially those related to memory and communication volume.

7 Fully-Sharded Data Parallel

With optimizer state sharding and data parallel, we're able to split the optimizer state and activations across our data-parallel axis. However, our model weights remain duplicated — we're storing a full copy of them all on each GPU.

We can solve this by turning our data parallel (DP) axis into a fully-sharded data parallel axis (FSDP). With FSDP, each GPU stores only its own slice of every weight tensor, but has to pull slices from other GPUs to form the full weight tensor using an all-gather to prepare for a forward or backward pass.

To avoid keeping GPU compute waiting around for communication to finish, most FSDP implementations schedule the layer's all-gather in advance of the operation, meaning the relevant weights are ready before they are needed, preventing communication from blocking computation. This keeps weight sharding communication off the critical path, meaning it has no cost as long as communication can keep up with compute and is scheduled well.

Some layers are small enough in memory and compute that the latency overhead of a transfer is not worth it. You should mark these layers not to be sharded by FSDP. In our architecture, this will mostly be the case for norms. This leaves us with the embedding layer and every linear layer.

While it is necessary to *store* master weights in FP32 (any values that are repeatedly accumulated into are sensitive to precision), the weights do not need to be *used* in FP32. In mixed precision, we always convert to the low-precision compute datatype before use, so we may as well convert even before the weight is communicated to save on bandwidth.

Problem (`fsdp`): Fully-Sharded Data Parallel (15 points)

实现 FSDP 训练的 Python 类，包装任意 PyTorch nn.Module（整个模型），并为其中所有 Linear 或 Embedding 层添加钩子或包装。建议接口：

• __init__(self, module: torch.nn.Module, compute_dtype: torch.dtype | None = None)：给定已实例化模块，构造 FSDP 模块，处理权重 all-gather 与梯度 reduce-scatter。钩子或包装器需及时汇集权重，供前向使用。为限制显存，只有在当前层之前第二层完成前向后，才开始汇集当前层权重。反向时同样汇集权重供计算；梯度就绪后 reduce-scatter 到对应 rank。使用后释放汇集的权重。若提供 compute_dtype，在通信或计算前转到该类型，同时保持主权重与优化器更新为 FP32。

• forward(self, *inputs, **kwargs)：用给定位置及关键字参数调用被包装模块的 forward()。

• finish_gradient_synchronization(self)：等待 GPU 上的异步通信结束。

提交内容：处理 FSDP 训练的容器类，每个分片应兼容作业 1 的标准 AdamW。实现 [adapters.get_fsdp]，运行 uv run pytest tests/test_fsdp.py。建议重复多次，例如 5 次，以捕捉竞态条件。

题目（fsdp_accounting）：FSDP 开销核算（5 分）

(a) 根据第 6 节分析，预计实现 FSDP 会节省多少峰值显存？计算中可忽略每个 GPU 为权重 all-gather 预分配的缓冲区大小。

提交内容：用 2-3 句话回答，给出结果。

(b) 在 2 GPU 上分析 xl 模型，关注权重 all-gather。通信能否及时完成以供前向传播？

提交内容：用 2-3 句话回答并给出耗时，附 Nsight 截图支持结论。

8 分析并行策略

训练还可沿其他轴并行，常见策略包括：

• 数据并行（DP）：将数据批次分到多个设备，各设备计算自己批次的梯度，再跨设备取平均。

• 全分片数据并行（FSDP）：在 DP 基础上，将优化器状态、梯度和权重也分到各设备，以节省显存。前向/反向时，各设备需从其他设备汇集权重分片。

Implement a Python class for fully-sharded data parallel training. The class should wrap an arbitrary PyTorch `nn.Module` (your full model) and hook into or wrap any `Linear` or `Embedding` layer within it. We recommend the following public interface:

`def __init__(self, module: torch.nn.Module, compute_dtype: torch.dtype | None = None):` Given an instantiated PyTorch `nn.Module` to be parallelized, construct an FSDP module that will handle weight all-gathers and gradient reduce-scatters. Make sure that your hooks or your module wrappers all-gather the weights in time for the forward pass. To limit memory use, only start gathering after the layer two before the current one has completed its forward pass. In the backward pass, your hooks or module wrappers should all-gather to have the weights available for the computation. When the gradients are available, they should be reduce-scattered to the appropriate ranks. Make sure to free the gathered weights after use. When `compute_dtype` is provided, cast the weights to that dtype before communicating or using them for compute, while keeping master weights and optimizer updates in FP32.

`def forward(self, *inputs, **kwargs):` Calls the wrapped module's `forward()` method with the provided positional and keyword arguments.

`def finish_gradient_synchronization(self):` When called, wait for asynchronous communication calls to finish on the GPU.

Deliverable: Implement a container class to handle fully sharded data parallel training. Each shard of this container should be compatible with the standard AdamW implementation from assignment 1. To test your FSDP implementation, implement the adapter [`adapters.get_fsdp`] . Run the tests with `uv run pytest tests/test_fsdp.py`. We recommend running the tests multiple times (e.g., 5) to catch any race conditions.

Problem (`fsdp_accounting`): FSDP Accounting (5 points)

(a) Given your analysis in Section 6, how much memory do you expect to save from the peak by implementing FSDP? You can ignore the size of the preallocated buffers needed to all-gather weights to each GPU in your calculation.

Deliverable: 2-3 sentence response with your findings.

(b) Profile the xl model on two GPUs and pay attention to the all-gather of weights. Does the communication finish in time for the forward pass?

Deliverable: 2-3 sentence response with your timings. Include screenshots of Nsight to back up your claims.

8 Analyzing Parallelism Strategies

There are more axes along which we can parallelize our training process. Some common strategies include:

- **Data parallelism (DP)** — Batches of data are split across multiple devices, and each device computes gradients for its own batch. Gradients are then averaged across devices.
- **Fully-Sharded Data Parallelism (FSDP)** — On top of data parallelism, we also split optimizer states, gradients, and weights across devices to reduce memory usage. Devices then need to gather weight shards from other devices during the forward and backward passes.

• 张量并行（TP）：权重矩阵沿输入或输出维度分片，各设备计算其分片对应的激活，再跨设备归约或汇集激活。

• 流水线并行（PP）：模型按层拆成多个阶段，各阶段运行在不同设备。

• 专家并行（EP）：混合专家模型的专家分到不同设备，各设备计算其专家的输出。

本节在简化设置下进行基本数学分析，选择与组合并行策略。先关注 DP、FSDP、TP 及组合，通过计算通信成本并与计算成本比较，判断扩展到多少设备后会被通信瓶颈限制。

更详细的 TPU/GPU 拓扑与并行策略可参考 TPU Scaling Book（J. Austin 等 [6]）；更详细的流水线并行讨论见 The Ultra-Scale Playbook 的附录（H. Z. P. N. M. M. L. W. T. W. Nouamane Tazi Ferdinand Mom [7]）。这些书的其他部分也有很多有用信息。

8.1 通信原语

先理解通信原语。假设 N 个设备编号 0,...,N-1，每一对设备均有链路连接；每设备出口（出站）带宽为 W，即每秒可向其他设备发送 W 字节。如何实现 gather 和 reduce？

all-gather 的常见实现是环形 all-gather。每设备 i 起初持有大小 S/N 的分块 xi，最后获得大小 S 字节的完整 x=[x0,...,xN-1]。将设备排成环，每步向右邻发送当前分块，保存左邻传来的分块；随后继续将刚收到的块向右转发，并从左侧接收新块。N-1 步后，各设备拥有完整张量。

理想情况下，每设备每步同时以 W 带宽发送 S/N 字节，共 N-1 步；耗时为 ((N-1)/N)·S/W 秒。

再分析环形 reduce-scatter。各设备 i 起初持有大小 S 的完整张量 x^(i)，目标是求和 y=Σx^(i)，最终每设备仅持有 y 的一个 S/N 分块。先排成环，再将每个完整张量拆成 N 块。与 all-gather 类似地传递分块，但每设备转发前先加上自己的贡献，块保存的是部分和。

对 t=1,...,N-1，设备 i 执行：

• 若 t=1，初始化 y←x^(i)，保存当前部分和。

• 将 y 的 (i-t) mod N 分块发送到 (i+1) mod N 设备。

• 从 (i-1) mod N 设备接收 z 的 (i-t-1) mod N 分块。

• 将收到分块加入 y 的对应分块，更新部分和。

N-1 步后，设备 i 获得 yi 分块的完整和；耗时与环形 all-gather 相同。准确索引表达式保留在右侧。

- **Tensor Parallelism (TP)** — Weight matrices are sharded across the input or output dimension. Devices compute the activations corresponding to their shard, and activations are then reduced or gathered across devices.
- **Pipeline Parallelism (PP)** — The model is split layerwise into multiple stages, where each stage is run on a different device.
- **Expert Parallelism (EP)** — The experts in a Mixture-of-Experts model are split onto different devices, and each device computes the output for its own expert.

In this section, we'll do some basic math in a simplified setting to choose between parallelism strategies and decide how to combine them. To start, we'll focus on DP, FSDP, TP, and their combinations. Our approach will be to calculate the communication cost of each strategy and compare it to the computational cost, which tells us how many devices we can scale to before communication cost becomes the bottleneck.

For a more detailed treatment of TPU/GPU topologies and parallelism strategies, the TPU Scaling Book (J. Austin *et al.* [6]) is an excellent resource. And for a more detailed pipeline parallel discussion, see The Ultra-Scale Playbook Appendix (H. Z. P. N. M. M. L. W. T. W. Nouamane Tazi Ferdinand Mom [7]). The rest of these books also have a lot of other information you might find useful.

8.1 Communication Primitives

Our first step will be to understand the communication primitives. In our simplified setting, suppose we have N devices numbered $0, ..., N-1$, and each pair of devices is connected by a link. We'll also assume each device has W egress (i.e. outgoing) bandwidth; in other words, each device can send data to another device at a rate of W bytes per second. How might we implement gather and reduce?

One common way to implement the all-gather operation is the **ring all-gather**. Recall that in an all-gather, each device i starts with a chunk x_i of size $\frac{S}{N}$, and ends up with the entire $x = [x_0, ..., x_{N-1}]$ of size S (in bytes). In a ring all-gather, we arrange the devices in a circle. In each step, each device sends its current chunk to the next device to its right, and stores the chunk it received from the device to its left. This process repeats, where each device passes the chunk it just received to the right, and receives a new chunk from the left. After $N-1$ steps, each device has the entire tensor.

In our idealized setting, each device simultaneously transmits a chunk of size $\frac{S}{N}$ in each step, with egress bandwidth W, and there are $N-1$ steps, so the ring all-gather takes $\frac{N-1}{N}\frac{S}{W}$ seconds.

Next, let's analyze the **ring reduce-scatter**. In a reduce-scatter, each device i starts with a full tensor $x^{(i)}$ of size S. We then want to compute the reduction $y = \sum_{i=0}^{N-1} x^{(i)}$, but where each device i ends up with just a chunk y_i of size $\frac{S}{N}$. Like the ring all-gather, we'll start by arranging the devices in a circle. Each device will first divide its tensor $x^{(i)}$ into N chunks $\left[x_0^{(i)}, ..., x_{N-1}^{(i)}\right]$, each of size $\frac{S}{N}$. We'll then pass chunks around just like the ring all-gather, except before passing the chunk on, each device adds its contribution to the chunk (which stores a partial sum). Specifically:

For step $t = 1, ..., N-1$, device i does the following:

- If $t = 1$, initialize $y \leftarrow x^{(i)}$, which stores the partial sum so far
- Send chunk $y_{(i-t) \bmod N}$ to device $(i+1) \bmod N$
- Receive chunk $z_{(i-t-1) \bmod N}$ from device $(i-1) \bmod N$
- Update your copy of the partial sum: $y_{(i-t-1) \bmod N} \leftarrow y_{(i-t-1) \bmod N} + z_{(i-t-1) \bmod N}$

After $N-1$ steps, device i then has the full sum for chunk y_i, so ring reduce-scatter takes $\frac{N-1}{N}\frac{S}{W}$ seconds, just like ring all-gather.

最后实现环形 all-reduce。每设备起初持有完整 S 字节张量 x^(i)，最终得到 y=Σx^(i)。先环形 reduce-scatter，再环形 all-gather，因此耗时为 2·((N-1)/N)·S/W 秒。

题目（alternate_ring_all_reduce）：另一种环形 all-reduce（1 分）

不用先 reduce-scatter 再 all-gather，而使用下列算法。对 t=1,...,N-1，设备 i 执行：

• 若 t=1，初始化 y←x^(i)，保存当前部分和。

• 向设备 (i+1) mod N 发送 x^((i-t+1) mod N)。

• 从设备 (i-1) mod N 接收 x^((i-t) mod N)。

• 更新部分和：y←y+x^((i-t) mod N)。

同样假设每设备出口带宽 W，各 x^(i) 大小 S，这个算法耗时多久？

提交内容：用 S、N、W 表示答案，附一句解释。

8.2 分析数据并行

了解通信原语后，可以分析并行策略。考虑单个 FFN 层，输入为 x，前向传播见公式（20）-（23）：x1=xW1，x2=xW2，z=f(x1)*x2，y=zW3。

x 形状 (B,D)，W1、W2 形状 (D,D_FF)，W3 形状 (D_FF,D)。f 是逐元素激活函数，如 SiLU；* 表示逐元素乘法。

显式写出反向传播也很有用。给定形状 (B,D) 的 dy，反向计算见公式（24）-（30），* 仍为逐元素乘法。

回顾使用 N_DP 个设备的数据并行：将 x 按批次分为 x^(i)，形状 (B/N_DP,D)。前向照常执行，无集合通信，产生同样形状的 y^(i)。反向传播在下一页接续。

Finally, let's implement **ring all-reduce**. In an all-reduce, each device i starts with a full tensor $x^{(i)}$ of size S, and ends up with the reduction $y = \sum_{i=0}^{N-1} x^{(i)}$. We'll implement all-reduce as a **ring reduce-scatter** followed by a **ring all-gather**, so the ring all-reduce takes $2\frac{N-1}{N}\frac{S}{W}$ seconds.

Problem (`alternate_ring_all_reduce`): Alternate ring all-reduce (1 point)

Instead of implementing all-reduce as a ring reduce-scatter followed by a ring all-gather, let's use the following algorithm:

For step $t = 1, ..., N-1$, device i does the following:

- If $t = 1$, initialize $y \leftarrow x^{(i)}$, which stores the partial sum so far
- Send $x^{((i-t+1) \bmod N)}$ to device $(i+1) \bmod N$
- Receive $x^{((i-t) \bmod N)}$ from device $(i-1) \bmod N$
- Update your copy of the partial sum: $y \leftarrow y + x^{((i-t) \bmod N)}$

In the same setting as above (W egress bandwidth per device, each $x^{(i)}$ is of size S), how long does this algorithm take?

Deliverable: An answer in terms of S, N, and W, along with a one-sentence justification.

8.2 Analyzing Data Parallel

Given our communication primitives, we are ready to analyze parallelism strategies. We'll analyze the parallelization of a single FFN layer. Recall that given input $\boldsymbol{x}$, our forward pass is given by the following:

$$\boldsymbol{x_1} = \boldsymbol{x}\boldsymbol{W_1} \tag{20}$$
$$\boldsymbol{x_2} = \boldsymbol{x}\boldsymbol{W_2} \tag{21}$$
$$\boldsymbol{z} = f(\boldsymbol{x_1}) * \boldsymbol{x_2} \tag{22}$$
$$\boldsymbol{y} = \boldsymbol{z}\boldsymbol{W_3}, \tag{23}$$

where $\boldsymbol{x}$ has shape (B, D), $\boldsymbol{W_1}$ and $\boldsymbol{W_2}$ have shape (D, D_{FF}), and $\boldsymbol{W_3}$ has shape (D_{FF}, D). f is our elementwise activation function (e.g. SiLU), and $*$ represents elementwise multiplication.

It will also be useful to explicitly write out the backward pass. Recall that given $\boldsymbol{dy}$ with shape (B, D), the backward pass is given by the following:

$$\boldsymbol{dz} = \boldsymbol{dy}\boldsymbol{W_3}^\top \tag{24}$$
$$\boldsymbol{dx_2} = \boldsymbol{dz} * f(\boldsymbol{x_1}) \tag{25}$$
$$\boldsymbol{dx_1} = \boldsymbol{dz} * f'(\boldsymbol{x_1}) * \boldsymbol{x_2} \tag{26}$$
$$\boldsymbol{dx} = \boldsymbol{dx_1}\boldsymbol{W_1}^\top + \boldsymbol{dx_2}\boldsymbol{W_2}^\top \tag{27}$$
$$\boldsymbol{dW_3} = \boldsymbol{z}^\top \boldsymbol{dy} \tag{28}$$
$$\boldsymbol{dW_2} = \boldsymbol{x}^\top \boldsymbol{dx_2} \tag{29}$$
$$\boldsymbol{dW_1} = \boldsymbol{x}^\top \boldsymbol{dx_1}, \tag{30}$$

where $*$ represents elementwise multiplication.

Recall that in data parallelism with N_{DP} devices, we shard our input $\boldsymbol{x}$ into shards $\boldsymbol{x^{(i)}}$ of size $\left(\frac{B}{N_{\text{DP}}}, D\right)$. The forward pass proceeds as usual without any collectives, producing activations $\boldsymbol{y^{(i)}}$ of size $\left(\frac{B}{N_{\text{DP}}}, D\right)$. In

中文译文 / 对应原文第 42 页

反向传播照常使用按批次分片的激活，设备 i 得到公式（31）-（33）的梯度。它们并未对全部 B 个外积求和，而只对本设备输入分片的 B/N_DP 个外积求部分和。需要跨设备 all-reduce，获得完整 dW3、dW2、dW1。

题目（data_parallel_calcs）：数据并行计算（3 分）

现在可以计算数据并行何时出现通信瓶颈。C（FLOP/s）为加速器速度，W（字节/秒）为每设备出口带宽，用以计算计算时间与通信时间。两者可重叠，因此通信时间超过计算时间时形成瓶颈。假设权重和激活均为 FP16，即每元素两字节。

(a) 使用 N_DP 数据并行时，反向需要多少 FLOPs？忽略非矩阵乘法操作。回顾 (A,B)×(B,C)→(A,C) 的矩阵乘法需要 2ABC FLOPs。

提交内容：用 B、D、D_FF、N_DP 表示答案，附一句解释。

(b) N_DP 数据并行的反向需要多少通信时间？

提交内容：用 B、D、D_FF、N_DP、W 的一部分变量表示答案，附一句解释。

(c) 固定其他参数，N_DP 最大能增加到多少而不被通信瓶颈限制？

提交内容：不等式，一侧为 N_DP，另一侧为 B、D、D_FF、C、W 的一部分变量构成的表达式，附一句解释。

8.3 分析全分片数据并行

FSDP 与 DP 一样，对输入和激活的批次维分片；为节省显存，进一步分片优化器状态、梯度与权重。可沿权重的任一维分片，使设备 i 持有 W1^(i)、W2^(i)、W3^(i)，每个大小 DD_FF/N_FSDP。

前向时，对按批次分片的输入执行 DP 前向。但必须先跨设备 all-gather 权重分片，见公式（34）、（35），下页接续。

公式、代码与图表见右侧对应原页

the backward pass, proceeding as usual with the batch-sharded activations, device i ends up with gradients

$$\boldsymbol{dW_3^{(i)}} = \boldsymbol{z^{(i)\top} dy^{(i)}} \tag{31}$$

$$\boldsymbol{dW_2^{(i)}} = \boldsymbol{x^{(i)\top} dx_2^{(i)}} \tag{32}$$

$$\boldsymbol{dW_1^{(i)}} = \boldsymbol{x^{(i)\top} dx_1^{(i)}}, \tag{33}$$

where instead of summing over all B outer products, we only have the partial sum over the $\frac{B}{N_{\text{DP}}}$ outer products for our shard of the input. Then, we need to do an all-reduce across devices to get the full gradients $\boldsymbol{dW_3}$, $\boldsymbol{dW_2}$, and $\boldsymbol{dW_1}$.

Problem (`data_parallel_calcs`): Data parallel calculations (3 points)

We now have everything we need to calculate when data parallelism becomes communication bottlenecked. Let C (in FLOP/s) denote the device accelerator speed, and W (in bytes per second) denote each device's egress bandwidth. We can then compute the computation time and communication time. Because computation and communication can be overlapped, we are bottlenecked when communication time becomes larger than computation time. We'll assume that all weights and activations are in FP16 (i.e. two bytes).

(a) How many FLOPs are required to compute the backward pass, with N_{DP} data parallelism? You can ignore all non-matmul operations. Recall that a matmul $(A, B)(B, C) \to (A, C)$ takes $2ABC$ flops.

Deliverable: An answer in terms of B, D, D_{FF}, and N_{DP}, along with a one-sentence justification.

(b) How much communication time is required in the backward pass, with N_{DP} data parallelism?

Deliverable: An answer in terms of a subset of B, D, D_{FF}, N_{DP}, and W, along with a one-sentence justification.

(c) Fixing the other parameters, how large can N_{DP} become before we're communication bottlenecked?

Deliverable: An inequality with N_{DP} on one side, and an expression in terms of a subset of B, D, D_{FF}, C, and W on the other, along with a one-sentence justification.

8.3 Analyzing Fully Sharded Data Parallel

Next, let's analyze FSDP. Recall that just like DP, FSDP shards the batch dimension of the inputs and activations. In addition, to save on memory, we also shard the optimizer states, gradients, and weights. We can shard the weights along either dimension, producing shards $\boldsymbol{W_1^{(i)}}$, $\boldsymbol{W_2^{(i)}}$, and $\boldsymbol{W_3^{(i)}}$ on device i, each of size $\frac{DD_{\text{FF}}}{N_{\text{FSDP}}}$.

In the forward pass, we will just do the data parallel forward pass on the batch-sharded input. But to do so, recall that we need to first all-gather the weight shards across devices:

$$\boldsymbol{W_1} = \text{all-gather}\left(\left\{\boldsymbol{W_1^{(i)}}\right\}_{i=0}^{N_{\text{FSDP}}-1}\right) \tag{34}$$

$$\boldsymbol{W_2} = \text{all-gather}\left(\left\{\boldsymbol{W_2^{(i)}}\right\}_{i=0}^{N_{\text{FSDP}}-1}\right) \tag{35}$$

右侧公式（36）完成 W3 的 all-gather；公式（37）表示“执行按批次分片的前向传播”。

all-gather 的时机可灵活安排：权重使用前汇集完成即可，使用后应丢弃以减少显存。为简化，本节将三个 all-gather 一起列出。

反向与前向一样，需先跨设备汇集权重。再执行 DP 反向，但不必 all-reduce，因为每设备只需要自己分片的梯度，所以改为 reduce-scatter。流程见公式（38）-（44），其中公式（41）表示“执行按批次分片的反向传播”。

注意上标 (i) 的分片符号具有两种含义：reduce-scatter 输入是完整权重上的部分和，输出是分片权重上的完整和。

题目（fsdp_calcs）：全分片数据并行计算（3 分）

使用与 DP 计算相同的设置，计算 FSDP 何时出现通信瓶颈。

(a) N_FSDP 个设备的 FSDP，反向需要多少 FLOPs？前向呢？

提交内容：用 B、D、D_FF、N_FSDP 表示两个答案，各附一句解释。

(b) 反向需要多少通信时间？前向呢？

提交内容：用 B、D、D_FF、N_FSDP、W 的一部分变量表示两个答案，各附一句解释。

(c) 固定其他参数，N_FSDP 最多多大，反向才不会成为通信瓶颈？前向呢？

提交要求在下一页接续。

$$\boldsymbol{W_3} = \text{all-gather}\left(\left\{\boldsymbol{W_3^{(i)}}\right\}_{i=0}^{N_{\text{FSDP}}-1}\right) \tag{36}$$

$$\text{(do batch-sharded forward pass)} \tag{37}$$

Note that we have some freedom in when we want to do the all-gather: we just need an all-gathered weight before it's used, and we should then discard it to keep memory costs low. For simplicity, in this section we just list the three all-gathers together.

Like the forward, in the backward pass we need to first all-gather the weight shards across devices. We can then do the data parallel backward pass, except we no longer need to do an all-reduce because each device only needs the gradient for its shard. Therefore, we do a reduce-scatter instead:

$$\boldsymbol{W_1} = \text{all-gather}\left(\left\{\boldsymbol{W_1^{(i)}}\right\}_{i=0}^{N_{\text{FSDP}}-1}\right) \tag{38}$$

$$\boldsymbol{W_2} = \text{all-gather}\left(\left\{\boldsymbol{W_2^{(i)}}\right\}_{i=0}^{N_{\text{FSDP}}-1}\right) \tag{39}$$

$$\boldsymbol{W_3} = \text{all-gather}\left(\left\{\boldsymbol{W_3^{(i)}}\right\}_{i=0}^{N_{\text{FSDP}}-1}\right) \tag{40}$$

$$\text{(do batch-sharded backward pass)} \tag{41}$$

$$\boldsymbol{dW_1^{(i)}} = \text{reduce-scatter}\left(\left\{\boldsymbol{dW_1^{(i)}}\right\}_{i=0}^{N_{\text{FSDP}}-1}\right) \tag{42}$$

$$\boldsymbol{dW_2^{(i)}} = \text{reduce-scatter}\left(\left\{\boldsymbol{dW_2^{(i)}}\right\}_{i=0}^{N_{\text{FSDP}}-1}\right) \tag{43}$$

$$\boldsymbol{dW_3^{(i)}} = \text{reduce-scatter}\left(\left\{\boldsymbol{dW_3^{(i)}}\right\}_{i=0}^{N_{\text{FSDP}}-1}\right) \tag{44}$$

Note that the sharding notation (superscript (i)) is overloaded: the inputs to the reduce-scatter are partial sums on the full weights, while the outputs are full sums on the sharded weights.

Problem (`fsdp_calcs`): Fully sharded data parallel calculations (3 points)

Under the same setting as the data parallel calculations, let's calculate when FSDP becomes communication bottlenecked.

(a) How many FLOPs are required to compute the backward pass, with N_{FSDP} FSDP? What about the forward pass?

Deliverable: Two answers in terms of B, D, D_{FF}, and N_{FSDP}, along with two one-sentence justifications.

(b) How much communication time is required in the backward pass, with N_{FSDP} FSDP? What about the forward pass?

Deliverable: Two answers in terms of a subset of B, D, D_{FF}, N_{FSDP}, and W, along with two one-sentence justifications.

(c) Fixing the other parameters, how large can N_{FSDP} become before the backward pass is communication bottlenecked? What about the forward pass?

中文译文 / 对应原文第 44 页

提交内容：两个不等式，一侧为 N_FSDP，另一侧为 B、D、D_FF、C、W 的一部分变量构成的表达式，各附一句解释。

8.4 分析张量并行

实际中 FSDP 经常与 TP 结合。TP 沿各权重矩阵的输入或输出维度跨设备分片。输入维度分片通常称"行并行"，输出维度分片称"列并行"。

具体而言，考虑 xW，x 形状 (B,D)，W 形状 (D,D_FF)。列并行时，W^(i) 形状 (D,D_FF/N_TP)，乘积通过 all-gather 组合，见公式（45）。

行并行时，W^(i) 形状 (D/N_TP,D_FF)，每设备还需在矩阵乘法前将输入截取为 x^(i)，形状 (B,D/N_TP)。乘积通过 all-reduce 求和，见公式（46）。

并行 FFN 时，使用特定组合：W1、W2 列并行，即输出维度分片；W3 行并行，即输入维度分片。行并行权重只需输入的一个切片，因此可省略列并行权重之后的 all-gather。给定形状 (B,D) 的 x，前向流程见公式（47）-（51）。W1^(i)、W2^(i) 形状 (D,D_FF/N_TP)，W3^(i) 形状 (D_FF/N_TP,D)。

题目（tp_calcs）：张量并行计算（4 分）

使用与 DP、FSDP 相同的设置，计算 TP 何时被通信限制。

(a) 给定形状 (B,D) 的 dy，写出上述 TP 策略的反向传播，分片权重形状如上。

提交内容：一系列反向传播等式，以 dy、分片权重 W1^(i)、W2^(i)、W3^(i)、前向保存的激活 x、x1^(i)、x2^(i)、z^(i)、y^(i)、通信原语，以及自行定义的中间变量表示。等式应产生每设备的梯度 dW1^(i)、dW2^(i)、dW3^(i)，以及反向输出 dx。可参考第 8.2 节未分片反向并修改。

(b) N_TP 个设备的 TP，前向需要多少 FLOPs？反向呢？

公式、代码与图表见右侧对应原页

Deliverable: Two inequalities with N_{FSDP} on one side, and an expression in terms of a subset of B, D, D_{FF}, C, and W on the other, along with two one-sentence justifications.

8.4 Analyzing Tensor Parallel

In practice, FSDP is often combined with a parallelism strategy called tensor parallelism (TP). In TP, we shard either the input or output dimension of each weight matrix across devices. Input dimension sharding is often called "row parallel," while output dimension sharding is often called "column parallel."

Specifically, suppose we want to shard the matmul $\boldsymbol{x}\boldsymbol{W}$, for $\boldsymbol{x}$ with shape (B, D) and $\boldsymbol{W}$ with shape (D, D_{FF}). In column parallel we have shards $\boldsymbol{W}^{(i)}$ of shape $\left(D, \frac{D_{\text{FF}}}{N_{\text{TP}}}\right)$, and we have

$$\boldsymbol{x}\boldsymbol{W} = \text{all-gather}\left(\left\{\boldsymbol{x}\boldsymbol{W}^{(i)}\right\}_{i=0}^{N_{\text{TP}}-1}\right). \tag{45}$$

On the other hand, for row parallel, we have shards $\boldsymbol{W}^{(i)}$ of shape $\left(\frac{D}{N_{\text{TP}}}, D_{\text{FF}}\right)$, and each device also narrows the input $\boldsymbol{x}$ into a slice $\boldsymbol{x}^{(i)}$ with shape $\left(B, \frac{D}{N_{\text{TP}}}\right)$ before doing the matmul. Then, we have

$$\boldsymbol{x}\boldsymbol{W} = \text{all-reduce}\left(\left\{\boldsymbol{x}^{(i)}\boldsymbol{W}^{(i)}\right\}_{i=0}^{N_{\text{TP}}-1}\right). \tag{46}$$

To parallelize our FFN, we'll use a specific tensor parallel configuration where $\boldsymbol{W_1}$ and $\boldsymbol{W_2}$ are column parallel (output-dimension-sharded), while $\boldsymbol{W_3}$ is row parallel (input-dimension-sharded). Since the row parallel weight only requires a slice of the input, this configuration lets us skip the all-gather after the column parallel weights. This strategy gives the following forward pass, given input $\boldsymbol{x}$ of size (B, D):

$$\boldsymbol{x_1}^{(i)} = \boldsymbol{x}\boldsymbol{W_1}^{(i)} \tag{47}$$

$$\boldsymbol{x_2}^{(i)} = \boldsymbol{x}\boldsymbol{W_2}^{(i)} \tag{48}$$

$$\boldsymbol{z}^{(i)} = f\left(\boldsymbol{x_1}^{(i)}\right) * \boldsymbol{x_2}^{(i)} \tag{49}$$

$$\boldsymbol{y}^{(i)} = \boldsymbol{z}^{(i)}\boldsymbol{W_3}^{(i)} \tag{50}$$

$$\boldsymbol{y} = \text{all-reduce}\left(\left\{\boldsymbol{y}^{(i)}\right\}_{i=0}^{N_{\text{TP}}-1}\right), \tag{51}$$

where $\boldsymbol{W_1}^{(i)}$ and $\boldsymbol{W_2}^{(i)}$ have shape $\left(D, \frac{D_{\text{FF}}}{N_{\text{TP}}}\right)$, and $\boldsymbol{W_3}^{(i)}$ has shape $\left(\frac{D_{\text{FF}}}{N_{\text{TP}}}, D\right)$.

Problem (`tp_calcs`): Tensor parallel calculations (4 points)

Under the same setting as the DP and FSDP calculations, let's calculate when TP becomes communication bottlenecked.

(a) Given input $\boldsymbol{dy}$ of size (B, D) write out the backward pass of the tensor parallel strategy described above (where $\boldsymbol{W_1}^{(i)}$ and $\boldsymbol{W_2}^{(i)}$ have shape $\left(D, \frac{D_{\text{FF}}}{N_{\text{TP}}}\right)$, and $\boldsymbol{W_3}^{(i)}$ has shape $\left(\frac{D_{\text{FF}}}{N_{\text{TP}}}, D\right)$).

Deliverable: A series of equations describing the backward pass, in terms of $\boldsymbol{dy}$, sharded weights $\left(\boldsymbol{W_1}^{(i)}, \boldsymbol{W_2}^{(i)}, \boldsymbol{W_3}^{(i)}\right)$, activations saved from the forward pass $\left(\boldsymbol{x}, \boldsymbol{x_1}^{(i)}, \boldsymbol{x_2}^{(i)}, \boldsymbol{z}^{(i)}, \boldsymbol{y}^{(i)}\right)$, communication primitives, and any intermediate variables you'd like to define. The equations should produce each device's gradients $\boldsymbol{dW_1}^{(i)}$, $\boldsymbol{dW_2}^{(i)}$, $\boldsymbol{dW_3}^{(i)}$, and the backward pass output $\boldsymbol{dx}$. Feel free to reference the non-sharded backward pass in Section 8.2 and modify it.

(b) How many FLOPs are required to compute the forward pass, with N_{TP} TP? What about the backward pass?

提交内容：用 B、D、D_FF、N_TP 表示两个答案，各附一句解释。

(c) N_TP 个设备的 TP，前向需要多少通信时间？反向呢？

提交内容：用 B、D、D_FF、N_TP、W 的一部分变量表示两个答案，各附一句解释。

(d) 固定其他参数，N_TP 最多多大，反向才不会成为通信瓶颈？前向呢？

提交内容：两个不等式，一侧为 N_TP，另一侧为 B、D、D_FF、C、W 的一部分变量构成的表达式，各附一句解释。

8.5 二维并行（FSDP + TP）

终于可以组合并行策略了！本节研究如何最优结合 FSDP 与 TP。提示：前几节应发现，批大小与模型规模限制可扩展设备数；让这些规模变大，可继续增加设备而不被通信限制。但批大小超过某个点后，梯度噪声显著减小，失去 SGD 的隐式正则化作用，性能开始下降；这个点常称"临界批大小"。扩展定律通常也决定希望采用的模型大小，这将是下次作业的任务。

本节假设有人提供全部问题参数：批大小、模型规模、带宽及加速器速度。你的任务是选择 FSDP 与 TP 配置，在保持计算受限而非通信受限的前提下，扩展到尽可能多的设备。

先说明组合机制。每设备有 TP rank i=0,...,N_TP-1 和 FSDP rank j=0,...,N_FSDP-1，形成二维网格，总设备数 N=N_TP N_FSDP。

按 TP，先沿输出维分片 W1、W2，沿输入维分片 W3，因此需在激活上加入 TP 风格的 all-reduce。再应用 FSDP：对输入批次维分片，并沿每权重矩阵未被 TP 分片的另一维进一步分片。在 TP 前向/反向之前，加入 FSDP 的权重 all-gather；TP 反向之后，加入权重梯度 reduce-scatter。

设备 (i,j) 最终持有 W1^(i,j)、W2^(i,j)，形状 (D/N_FSDP,D_FF/N_TP)；W3^(i,j) 形状 (D_FF/N_TP,D/N_FSDP)。给定按批次分片的输入 x^(j)，形状 (B/N_FSDP,D)，前向见公式（52）-（56），下页接续。

Deliverable: Two answers in terms of B, D, D_{FF}, and N_{TP}, along with two one-sentence justifications.

(c) How much communication time is required in the forward pass, with N_{TP} TP? What about the backward pass?

Deliverable: Two answers in terms of a subset of B, D, D_{FF}, N_{TP}, and W, along with two one-sentence justifications.

(d) Fixing the other parameters, how large can N_{TP} become before the backward pass is communication bottlenecked? What about the forward pass?

Deliverable: Two inequalities with N_{TP} on one side, and an expression in terms of a subset of B, D, D_{FF}, C, and W on the other, along with two one-sentence justifications.

8.5 2D Parallelism (FSDP + TP)

We're finally ready to combine parallelism strategies! In this section, we'll look at how to optimally combine FSDP and TP. Hint: in the previous sections, you should have found that your batch size and model size parameters limit how many devices you can scale to, where making everything larger allows you to keep scaling devices without being communication bottlenecked. Unfortunately, scaling batch size past a certain point starts to degrade performance because gradients noise shrinks significantly, losing the implicit regularization properties of SGD; this point is often called the "critical batch size." And scaling laws often tell us how large we want the model to be (that will be your job in the next assignment!).

In this section, we'll consider a simplified setting where someone comes to you with all of the problem parameters (batch size, model size, bandwidth, accelerator speed). Your job will be to choose a configuration of FSDP and TP that scales to as many devices as possible, while remaining compute-bound rather than communication-bound.

Let's first walk through the mechanics of combining FSDP with TP. Each device will have a TP rank $i = 0, \ldots, N_{\text{TP}} - 1$ and an FSDP rank $j = 0, \ldots, N_{\text{FSDP}} - 1$, forming a 2D grid with $N = N_{\text{TP}} N_{\text{FSDP}}$ devices total. Following TP, we'll first shard $\boldsymbol{W_1}$ and $\boldsymbol{W_2}$ along the output dimension, and $\boldsymbol{W_3}$ along the input dimension. As a result, we'll have to insert a TP-style all-reduce on the activations. Next, applying FSDP, we'll shard the batch dimension of the inputs, and we'll also further shard each weight matrix along whichever dimension wasn't sharded by TP. Then, we'll have to insert FSDP-style all-gathers on the weights before doing our TP-style forward/backward passes, and reduce-scatters on the weight gradients after our TP-style backward pass.

The result is that each device (i, j) holds the weight shards $\boldsymbol{W_1}^{(i,j)}$ and $\boldsymbol{W_2}^{(i,j)}$ with shape $\left(\frac{D}{N_{\text{FSDP}}}, \frac{D_{\text{FF}}}{N_{\text{TP}}}\right)$, and $\boldsymbol{W_3}^{(i,j)}$ with shape $\left(\frac{D_{\text{FF}}}{N_{\text{TP}}}, \frac{D}{N_{\text{FSDP}}}\right)$. We can then write out the forward pass as the following, given batch-sharded input $\boldsymbol{x}^{(j)}$ of size $\left(\frac{B}{N_{\text{FSDP}}}, D\right)$:

$$\boldsymbol{W_1}^{(i)} = \text{all-gather}\left(\left\{\boldsymbol{W_1}^{(i,j)}\right\}_{j=0}^{N_{\text{FSDP}}-1}\right) \tag{52}$$

$$\boldsymbol{W_2}^{(i)} = \text{all-gather}\left(\left\{\boldsymbol{W_2}^{(i,j)}\right\}_{j=0}^{N_{\text{FSDP}}-1}\right) \tag{53}$$

$$\boldsymbol{W_3}^{(i)} = \text{all-gather}\left(\left\{\boldsymbol{W_3}^{(i,j)}\right\}_{j=0}^{N_{\text{FSDP}}-1}\right) \tag{54}$$

$$\boldsymbol{x_1}^{(i,j)} = \boldsymbol{x}^{(j)} \boldsymbol{W_1}^{(i)} \tag{55}$$

$$\boldsymbol{x_2}^{(i,j)} = \boldsymbol{x}^{(j)} \boldsymbol{W_2}^{(i)} \tag{56}$$

右侧公式（57）-（59）接续二维并行前向，最终得到按批次分片的输出 y^(j)，形状 (B/N_FSDP,D)。为简洁，本节省略反向，只关注前向。但现在你应已有自行写出反向的全部信息。

题目（fsdp_tp_calcs）：二维并行计算（6 分）

采用此前相同设置，计算二维并行何时出现瓶颈。

(a) N_FSDP 的 FSDP 加 N_TP 的 TP，前向需要多少 FLOPs？

提交内容：用 B、D、D_FF、N_FSDP、N_TP 表示答案，附一句解释。

(b) 前向需要多少通信时间？假设各轴通信可以重叠，即 FSDP 集合通信可与 TP 集合通信重叠。

提交内容：用 B、D、D_FF、N_FSDP、N_TP、W 的一部分变量表示答案，附一句解释。提示：由于可以重叠，答案应为 FSDP 与 TP 两项集合通信成本的最大值。

(c) N_TP 和 N_FSDP 取最优配置时，N=N_TP N_FSDP 最多多大，前向才不会被通信限制？

提交内容：一个不等式，一侧为 N，另一侧为 B、D、D_FF、C、W 的一部分变量构成的表达式；用几句话和等式解释。

(d) 若 FSDP 与 TP 共享网络资源，两个轴的集合通信无法重叠，最优配置下 N 最多多大，前向才不会被通信限制？不必将 N_TP、N_FSDP 截取为整数。

提交内容：一个不等式，一侧为 N，另一侧为 B、D、D_FF、C、W 的一部分变量构成的表达式；用几句话和等式解释。

9 排行榜

作业 2 排行榜测试 8B 模型完整训练步骤的速度。挑战你测试代码，使用能想到的技巧优化显存与时间。

主要限制是不能改变模型输入/输出行为。你的实现将与 cs336_basics 目录中的模型对照测试。输入用 BF16 与因果掩码，需通过与常规实现相同的测试。实现必须由你自行编写，不得使用或复制已有实现。

在两块 B200 上，以批大小 2、序列长度 32,768 测量。题目故意让模型难以装入显存，完整配置见下页代码。我们将核验前 5-10 名提交的正确性和性能。

$$\boldsymbol{z}^{(i,j)} = f\left(\boldsymbol{x}_1^{(i,j)}\right) * \boldsymbol{x}_2^{(i,j)} \tag{57}$$

$$\boldsymbol{y}^{(i,j)} = \boldsymbol{z}^{(i,j)} \boldsymbol{W}_3^{(i)} \tag{58}$$

$$\boldsymbol{y}^{(j)} = \text{all-reduce}\left(\{\boldsymbol{y}^{(i,j)}\}_{i=0}^{N_{\text{TP}}-1}\right), \tag{59}$$

ending up with batch-sharded output $\boldsymbol{y}^{(j)}$ of size $\left(\frac{B}{N_{\text{FSDP}}}, D\right)$. We'll omit the backward pass for brevity in this section, and just focus on the forward pass. But at this point, you should have all the information you need to write it out yourself.

Problem (`fsdp_tp_calcs`): 2D parallelism calculations (6 points)

Under the same setting as the calculations so far, let's calculate when 2D parallelism becomes bottlenecked.

(a) How many FLOPs are required to compute the forward pass, with N_{FSDP} FSDP + N_{TP} TP?

Deliverable: An answer in terms of B, D, D_{FF}, N_{FSDP}, and N_{TP}, along with a one-sentence justification.

(b) How much communication time is required in the forward pass, with N_{FSDP} FSDP + N_{TP} TP? Assume that the communication along each axis can be overlapped (in other words, the collectives along the FSDP axis can be overlapped with the collectives along the TP axis).

Deliverable: An answer in terms of a subset of B, D, D_{FF}, N_{FSDP}, N_{TP}, and W, along with a one-sentence justification. Hint: The answer should be expressed as a max between two quantities (the FSDP and TP collective costs), since the two can be overlapped.

(c) Under the optimal setting of N_{TP} and N_{FSDP}, how large can $N = N_{\text{TP}}N_{\text{FSDP}}$ become before the forward pass is communication bottlenecked?

Deliverable: An inequality with N on one side, and an expression in terms of a subset of B, D, D_{FF}, C, and W on the other, along with a few sentences and equations as justification.

(d) Now suppose the FSDP-axis and TP-axis collectives cannot be overlapped because they share the same network resources. Under the optimal setting of N_{TP} and N_{FSDP}, how large can $N = N_{\text{TP}}N_{\text{FSDP}}$ become before the forward pass is communication bottlenecked? Don't worry about truncating N_{TP} and N_{FSDP} to be integers.

Deliverable: An inequality with N on one side, and an expression in terms of a subset of B, D, D_{FF}, C, and W on the other, along with a few sentences and equations as justification.

9 Leaderboard

Assignment 2's leaderboard will test the speed of a full training step for an 8B model. We challenge you to benchmark your code and to optimize memory and runtime, using any tricks you can come up with. The key restrictions are that you cannot change the input/output behavior of the model. Your implementation will be tested against the model in the `cs336_basics` directory. Your inputs will be tested at BF16 with causal masking, and they must pass the same tests as your regular implementation. The implementation must also be your own, and you cannot use or copy pre-existing implementations. Your timing should be measured on two B200 GPUs using a sample with batch size 2 and sequence length 32,768. It is intentionally difficult to fit the model in memory. See the code below for the full config. We

中文译文 / 对应原文第 47 页

排行榜使用的计时代码完整保留在右侧。

模型配置：ctx_len=32768，vocab_size=151936，d_model=4096，d_ff=11008，num_layers=34，num_heads=32，torch_dtype=torch.bfloat16，is_causal=True，batch_size=2。

测试生成随机 labels、targets，创建 BasicsTransformerLM 与 AdamW。train_step 依次清零梯度（set_to_none=True）、前向、计算交叉熵并求和、反向、优化器更新；用 triton.testing.do_bench 计时，rep=30_000、warmup=10_000，并打印结果。

为了测试，可以将重复执行及预热时长（单位毫秒）调短。

可尝试的优化及让模型装入显存的思路：

• 调整内核块大小，可以使用 Triton autotune。

• 调整其他 Triton/torch.compile 配置参数。

• 实现融合 AdamW。

• 基础实现会生成完整 logits，形状 [batch, seq_len, vocab_size]。编写融合 LM head 与交叉熵损失的内核，也可让它立即以融合方式计算反向传播。

• 改进 FlashAttention：

① 用 Triton 而不仅是 torch.compile 实现反向，见第 4.2.3 节。

② 反向遍历输入两次，一次用于 dQ，一次用于 dK、dV，避免原子操作或块间同步。

③ 因果掩码时让程序实例提前结束，跳过必然全零的块。

④ 区分无需掩码的块与对角块：前者不比较索引，后者仅比较一次。

其余建议在下页接续。

will verify the top 5-10 submissions for correctness and performance. The test we will run to time your implementation is the following:

```
class Config:
    ctx_len = 32768
    vocab_size = 151936
    d_model = 4096
    d_ff = 11008
    num_layers = 34
    num_heads = 32
    torch_dtype = torch.bfloat16
    is_causal = True
    batch_size = 2
cfg = Config()

def test_timing_forward_backward():
    labels, targets = torch.randint(high=cfg.vocab_size, size=(2, cfg.batch_size,
cfg.ctx_len))

    model = BasicsTransformerLM(Config())
    optimizer = AdamW(model.parameters())

    def train_step():
        optimizer.zero_grad(set_to_none=True)
        res = model(labels)
        loss = cross_entropy(res, targets).sum()
        loss.backward()
        optimizer.step()

    timing_results = triton.testing.do_bench(train_step, rep=30_000, warmup=10_000)
    print(timing_results)
```

For testing purposes, you can reduce the repetition and warmup time (given in ms) to something shorter.

Some ideas for improvement and for making sure your model fits:

- Tune the tile sizes for your kernel (use Triton autotune for this!)
- Tune additional Triton/`torch.compile` config parameters
- Implement fused AdamW
- The base implementation is materializing the full logits (`[batch, seq_len, vocab_size]`). Write a kernel that fuses the LM head and your cross-entropy loss. You can also have it compute the backward pass immediately in a fused manner
- Improve FlashAttention
  - Implement the backward pass in Triton, not just torch.compile (see Section 4.2.3)
  - Do two passes over your input for the backward pass, one for dQ and another for dK and dV, to avoid atomics or synchronization between blocks
  - Stop program instances early when doing causal masking, skipping tiles that are guaranteed to be all zero
  - Separate the non-masked tiles from the tile diagonals, computing the first without ever comparing indices, and the second with a single comparison

⑤ 在 Hopper 之后的架构上使用 TMA（Tensor Memory Accelerator，张量内存加速器），遵循与教程类似的模式。

• 只有确有需要时，才用激活检查点，以运行速度换取显存节省。

题目（leaderboard）：排行榜：最快训练步骤（10 分）

基准测试使用批大小 2、两块 B200，以完整训练步骤的实际经过时间评估：前向、损失、反向和 AdamW 更新。

从空的 PyTorch/Triton 缓存开始，整个基准运行必须在 10 分钟内完成，因此要谨慎使用过于激进的 torch.compile 和 Triton 自动调优。

提交内容：包含 AdamW 的完整前向与反向训练步骤的最佳实际经过时间。

我们期望排行榜提交超过耗时 10 秒的朴素基线。

将结果提交到：

github.com/stanford-cs336/assignment2-systems-leaderboard

参考文献

作者信息和原始出版记录完整保留在右侧；以下翻译标题并保留链接。

[1] T. Dao：《FlashAttention-2：通过更好的并行与工作划分加速注意力》。在线：

https://arxiv.org/abs/2307.08691

[2] T. Dao、D. Y. Fu、S. Ermon、A. Rudra、C. Re：《FlashAttention：感知 I/O 的快速、显存高效的精确注意力》。载《神经信息处理系统进展》，A. H. Oh、A. Agarwal、D. Belgrave、K. Cho 编，2022。在线：

https://openreview.net/forum?id=H4DqfPSibmx

[3] M. Milakov、N. Gimelshein：《softmax 的在线归一化因子计算》。在线：

https://arxiv.org/abs/1805.02867

[4] H. He：《从第一性原理出发，让深度学习飞快运行》，2022。在线：

https://horace.io/brrr_intro.html

[5] S. Rajbhandari、J. Rasley、O. Ruwase、Y. He：《ZeRO：面向万亿参数模型训练的显存优化》，2020。

[6] J. Austin 等：《如何扩展你的模型》，2025。

[7] H. Z. P. N. M. M. L. W. T. W. Nouamane Tazi Ferdinand Mom：《超大规模实战手册：在 GPU 集群上训练大语言模型》，2025。

- Use TMA (Tensor Memory Accelerator) functionality on architectures later than Hopper, following a similar pattern to our tutorial
- Use activation checkpointing to trade runtime speed for memory savings only if you need it

Problem (`leaderboard`): Leaderboard: fastest training step (10 points)

The benchmark will be run at batch size 2 on two B200 GPUs. Your submission will be evaluated on wall-clock time for a complete training step: forward pass, loss, backward pass, and AdamW update.

From an empty PyTorch/Triton cache, your benchmarking run must complete within 10 minutes, so be careful with overly aggressive `torch.compile` and Triton autotuning.

Deliverable: Your best wall-clock time for a full forward-and-backward training step with AdamW.

We expect leaderboard submissions to beat the naïve baseline of 10 seconds.

Submit your result to the leaderboard here:
`github.com/stanford-cs336/assignment2-systems-leaderboard`

Bibliography

[1] T. Dao, "FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning." [Online]. Available: https://arxiv.org/abs/2307.08691

[2] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Re, "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness," in *Advances in Neural Information Processing Systems*, A. H. Oh, A. Agarwal, D. Belgrave, and K. Cho, Eds., 2022. [Online]. Available: https://openreview.net/forum?id=H4DqfPSibmx

[3] M. Milakov and N. Gimelshein, "Online normalizer calculation for softmax." [Online]. Available: https://arxiv.org/abs/1805.02867

[4] H. He, "Making Deep Learning Go Brrrr From First Principles," 2022, [Online]. Available: https://horace.io/brrr_intro.html

[5] S. Rajbhandari, J. Rasley, O. Ruwase, and Y. He, "ZeRO: Memory Optimizations Toward Training Trillion Parameter Models." 2020.

[6] J. Austin *et al.*, "How to Scale Your Model," 2025.

[7] H. Z. P. N. M. M. L. W. T. W. Nouamane Tazi Ferdinand Mom, "The Ultra-Scale Playbook: Training LLMs on GPU Clusters." 2025.